technische universität dortmund

Institut für Energiesysteme, Energieeffizienz und Energiewirtschaft

Dimensionality Reduction in Reinforcement Learning

Desmond Fotock Fomelack

Examiners:
Dr.-Ing. Ulf Häger
Prof. Dr.-Ing. Timm Faulwasser

Master thesis
ie3
December 2024

technische universität dortmund

Institut für Energiesysteme, Energieeffizienz und Energiewirtschaft

Master thesis for Desmond Fotock Fomelack

Dimensionality Reduction in Reinforcement Learning

Motivation:

In Model Predictive Control (MPC), dimensionality reduction based on the input trajectory subspace has proven successful in reducing the computational complexity while maintaining the control performance. Starting with move blocking. The methods used to determine the subspace include move blocking schemes, SVD based decomposition ,orthonormal basis functions, or sensitivity based approaches.

In contrast to MPC, Reinforcement Learning (RL) learns control laws from closed-loop interactions with the controlled system with limited knowledge of the system model. Reinforcement learning has many parallels to closed-loop control and especially to MPC and several works have combined RL and MPC. For example, in MPC-RL a parameterized Optimal Control Problems (OCPs) is used as a function approximator in RL actor critic methods, where the parameters of the OCP are improved from closed loop feedback.

The aim of the this thesis is to combine the ideas of dimensionality reduction from NMPC in active subspace with MPC-RL. In that, the OCP dimensionality reduction should be improved by RL based on the closed loop interactions.

To this end, the following tasks are to be considered:

- Literature review on MPC with dimensionality reduction and on combinations of RL and MPC formulation of an algorithm to combine MPC and RL
- Formulation and comparison of optimal control problems for trajectory optimization and obstacle avoidance
- Implementation of the algorithm in Python
- Comparison of computational complexity and control performance between the algorithms

The results of the thesis have to be presented in a colloquium.

Starting date:	20. 06. 2024
Deadline:	20. 12. 2024
Examiners:	Ulf Häger
	Timm Faulwasser

Eidesstattliche Versicherung

(Affidavit)

Fomelack, Desmond Fotock — 243690

Name, Vorname (surname, first name) — Matrikelnummer (student ID number)

☐ Bachelorarbeit (Bachelor's thesis) ■ Masterarbeit (Master's thesis)

Titel (Title)

Dimensionality Reduction in Reinforcement Learning

Ich versichere hiermit an Eides statt, dass ich die vorliegende Abschlussarbeit mit dem oben genannten Titel selbstständig und ohne unzulässige fremde Hilfe erbracht habe. Ich habe keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate kenntlich gemacht. Die Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

I declare in lieu of oath that I have completed the present thesis with the above-mentioned title independently and without any unauthorized assistance. I have not used any other sources or aids than the ones listed and have documented quotations and paraphrases as such. The thesis in its current or similar version has not been submitted to an auditing institution before.

Wuppertal,20.12.2024

Ort, Datum (place, date) — Unterschrift (signature)

Belehrung:
Wer vorsätzlich gegen eine die Täuschung über Prüfungsleistungen betreffende Regelung einer Hochschulprüfungsordnung verstößt, handelt ordnungswidrig. Die Ordnungswidrigkeit kann mit einer Geldbuße von bis zu 50.000,00 € geahndet werden. Zuständige Verwaltungsbehörde für die Verfolgung und Ahndung von Ordnungswidrigkeiten ist der Kanzler/die Kanzlerin der Technischen Universität Dortmund. Im Falle eines mehrfachen oder sonstigen schwerwiegenden Täuschungsversuches kann der Prüfling zudem exmatrikuliert werden. (§ 63 Abs. 5 Hochschulgesetz - HG -).

Die Abgabe einer falschen Versicherung an Eides statt wird mit Freiheitsstrafe bis zu 3 Jahren oder mit Geldstrafe bestraft.

Die Technische Universität Dortmund wird ggf. elektronische Vergleichswerkzeuge (wie z.B. die Software „turnitin") zur Überprüfung von Ordnungswidrigkeiten in Prüfungsverfahren nutzen.

Die oben stehende Belehrung habe ich zur Kenntnis genommen:

Official notification:
Any person who intentionally breaches any regulation of university examination regulations relating to deception in examination performance is acting improperly. This offense can be punished with a fine of up to EUR 50,000.00. The competent administrative authority for the pursuit and prosecution of offenses of this type is the Chancellor of TU Dortmund University. In the case of multiple or other serious attempts at deception, the examinee can also be unenrolled, Section 63 (5) North Rhine-Westphalia Higher Education Act (*Hochschulgesetz, HG*).

The submission of a false affidavit will be punished with a prison sentence of up to three years or a fine.

As may be necessary, TU Dortmund University will make use of electronic plagiarism-prevention tools (e.g. the "turnitin" service) in order to monitor violations during the examination procedures.

I have taken note of the above official notification:*

Wuppertal,20.12.2024

Ort, Datum (place, date) — Unterschrift (signature)

***Please be aware that solely the German version of the affidavit ("Eidesstattliche Versicherung") for the Bachelor's/ Master's thesis is the official and legally binding version.**

Abstract

Nonlinear Model Predictive Control (NMPC) is a powerful optimization control technology widely used for stabilizing and controlling complex nonlinear dynamics systems. However, NMPC is computationally demanding, especially for systems that require long forecasting or real-time performance. Dimensional reduction techniques such as Singular Value Decomposition (SVD), Movement Blocking and Active Subspace Methods have been proposed to reduce this computational burden.

Among them, NMPC in active subspaces presents a promising approach by projecting the optimization problem into a lower-dimensional input space while maintaining recursive feasibility and stability. This approach depends on certain projector matrices to solve the optimization problem in the reduced-dimensional subspaces. However, there is no underlying concept for selecting these projector matrices in order to achieve closed-loop optimality.

This paper aims to address the gaps in NMPC in active subspaces through Reinforcement Learning (RL) by learning these projector matrices in a way that improves closed-loop performance. Our proposed method integrates NMPC in active subspaces as a function approximation to the state-action value function of RL.

The goal of the RL problem is to search for the suboptimal policy among our dimensionality-reduced policies that minimizes a given performance index in a closed-loop fashion. By so doing we are able to get a suboptimal projectors matrix with improve closed-loop performance.

The effectiveness of the proposed Model Predictive Control (MPC) RL-based framework (MPCQ-learning) is demonstrated by stabilizing the unstable equilibrium point of the nonlinear cart-pole system across a distribution of given initial conditions. The results show significant improvements in closed-loop performance and computational efficiency compared to traditional methods.

This work is a step forward in the synthesis of data-driven techniques with optimization control frameworks, which paved the way for more scalable and efficient NMPC solutions for nonlinear systems.

Contents

1 Introduction **1**

2 Literature Review **3**
2.1 Model Predictive Control (MPC) 3
2.1.1 Problem Formulation 4
2.1.2 Direct Single and Multiple Shooting in Non-Linear Program (NLP) Formulation 5
2.1.3 NMPC in Active Subspaces 7
2.2 Reinforcement Learning (RL) 14
2.2.1 Basics of RL 14
2.2.2 Q-learning framework 19
2.2.3 MPC as a function approximator for RL 21

3 MPCQ-Learning For Active Subspace **25**
3.1 Overview of Proposed Approach 25
3.2 Learning Active Subspace in MPC with RL 29
3.3 Algorithm Design 34

4 Simulation and Results **38**
4.1 Simulation 38
4.1.1 Model Dynamics 38
4.1.2 Design of the NMPC controller 41
4.2 Results 54

5 Conclusion **65**

Bibliography **66**

Nomenclature

Symbol

α	Learning rate
$\boldsymbol{u}_k$	Predicted control trajectory
$\boldsymbol{x}_k$	Predicted state trajectory
ℓ	Stage cost
η	Variance threshold
γ	Discount factor
$\hat{C}$	Estimated covariance matrix
κ	LQR state feedback law
$\mathbb{U}$	Set of feasible controls
$\mathbb{X}$	Set of feasible states
$\mathbb{X}_0$	Distribution of initial conditions
$\tilde{\mathbf{u}}$	Perturbed control input
$\mathcal{AS}$	Active subspace
$\mathcal{H}$	Constraint sets
$\mathcal{IS}$	Inactive subspace
$\mathcal{J}$	Cost performance index
$\mathcal{J}_N$	Cost performance index
$\mathcal{S}$	Feature vector
$\mathcal{U}$	Set of open-loop control trajectories
$\mathscr{P}$	Parametric program
$\Phi(s_N)$	Terminal cost
π	Policy
$\pi^\star$	Optimal policy
Σ	Eigenvalue matrix
$\tilde{w}$	Inactive subspace vector
a	Action
a_k	Closed-loop control
C	Covariance matrix
f	System dynamic function
h	Constraint function
K	Number of episodes
k	Discrete time index

L	Reward
N	Horizon length
n_H	Dimension of the constraints vector H
n_u, m	Dimension of input space
n_v	Active subspace dimension
n_w	Inactive subspace dimension
n_x, n	Dimension of state space
Q	State weight matrix
Q_N^*	Optimal state-action value function
Q_N^π	State-action value function
R	Input weight matrix
r	Reward
S	State space
s	Closed-loop state
s_0	Initial closed-loop state
s_k	Closed-loop state
T	Prediction horizon
t	Continuous time index
T_1	Active subspace projector
T_2	Inactive subspace projector
$u^\star$	Optimal control trajectory
u_k	Input vector
v	Active subspace vector
$V_N^\pi(s)$	Value function
$V^\star$	Optimal value function
V_f	Terminal cost
X_0	Distribution of initial conditions
x_0	Initial condition of the state
x_k	State vector

Acronyms

MPC Model Predictive Control . . . vii
NMPC Nonlinear Model Predictive Control . . . vii
OCP Optimal Control Problem . . . 1
NLP Non-Linear Program . . . ix
RL Reinforcement Learning . . . vii
SVD Singular Value Decomposition . . . 1
TD Temporal-Difference . . . 18
PCA Principal Component Analysis . . . 3
NNs Neural Networks . . . 21
LQR linear quadratic regulator . . . 45
LQR Linear Quadratic Regulator . . . 45
OC Optimal Control . . . 6
DP Dynamic Programming . . . 16
dOCP Discounted Optimal Control Problem . . . 16
MC Monte Carlo . . . 18
SARSA State-Action-Reward-State-Action . . . 18
LS Least-Square . . . 20

List of Figures

2.1 Full NMPC closed-loop framework 4
2.2 The agent-environment interaction in reinforcement learning 15

3.1 MPCQ-learning framework 34

4.1 Cart-pole system dynamics representation 39
4.2 Closed-loop trajectories of NMPC for different initial conditions 44
4.3 Closed-loop trajectories of NMPC with $C = H$ and $n_v + 1 = 61$, for different initial conditions 47
4.4 Closed-loop trajectories of NMPC with $C = I$ and $n_v + 1 = 61$, for different initial conditions 48
4.5 Closed-Loop trajectories for $x_0 = [1.296, 2.858, 0, 0]^\top$ $n_v + 1 = 61$ and $N \cdot n_u = 100$ 49
4.6 Closed-loop trajectories for $x_0 = [1.296, 2.858, 0, 0]^\top$ after PCA with $n_v + 1 = 6$ and $N \cdot n_u = 100$ 51
4.7 Closed-loop trajectories for $x_0 = [1.296, 2.858, 0, 0]^\top$ after RL with $n_v + 1 = 6$ and $N \cdot n_u = 100, K = 1300, C = H$ 54
4.8 Closed-loop trajectories for $x_0 = [1.296, 2.858, 0, 0]^\top$ after RL with $n_v + 1 = 6$ and $N \cdot n_u = 100, K = 1300, C = I$ 55
4.9 Closed-loop trajectories for different initial conditions with the RL Policy,n_v+1 = 6 and $N \cdot n_u = 100$,$C = I$ 57
4.10 Closed-loop trajectories for different initial conditions with the RL policy n_v+1 = 6 and $N \cdot n_u = 100$, $C = H$ 58
4.11 Closed loop trajectories of RL Policies for different training episodes $n_v + 1 = 6$ and $N \cdot n_u = 100$ 59
4.12 Closed-loop trajectories for $x_0 = [1.296, 2.858, 0, 0]\top$ after PCA with $n_v + 1 = 23, n_v + 1 = 10, n_v + 1 = 4$ and $N \cdot n_u = 100$, $C = H$ 60
4.13 Closed-loop trajectories for $x_0 = [1.296, 2.858, 0, 0]\top$ after RL with $n_v + 1 = 23, n_v + 1 = 10, n_v + 1 = 4$ and $N \cdot n_u = 100$, $C = H$ 61
4.14 Closed-loop trajectories for $x_0 = [1.296, 2.858, 0, 0]\top$ after PCA with $n_v + 1 = 23, n_v + 1 = 10, n_v + 1 = 4$ and $N \cdot n_u = 100$, $C = I$ 62
4.15 Closed-loop trajectories for $x_0 = [1.296, 2.858, 0, 0]\top$ after RL with $n_v + 1 = 23, n_v + 1 = 10, n_v + 1 = 4$ and $N \cdot n_u = 100$, $C = I$ 63

4.16 Computational complexity and relative closed -loop performance for $n_v = \{1, \cdots, 10\}$ with $n_v + 1 = 6$:RL Policy 64

List of Tables

4.1 Relative performance difference δ (3.26) with $N_b = 30$, $N_{X_0} = 130$, and $N \cdot n_u = 100$. 48
4.2 Mean computation time of different NMPC schemes with 130 initial conditions 49
4.3 Relative performance difference δ (3.26) with $N_b = 30$, $N_{x_0} = 130$, and $N \cdot n_u = 100$ after PCA. 51
4.4 Mean computation time of different NMPC schemes with 130 initial conditions after PCA 52
4.5 Relative performance difference δ (3.26) with $N_b = 30$, $N_{x_0} = 130$, and $N \cdot n_u = 100$ after RL. 56
4.6 Mean computation time of different NMPC schemes with 130 initial conditions after RL 56

1 Introduction

MPC, including its nonlinear variant NMPC, is a powerful optimization tool widely used for controlling complex systems. However, MPC is computationally expensive, as it typically requires a large prediction horizon to ensure system stability for highly nonlinear dynamics. This computational burden has driven research into dimensionality reduction techniques aimed at making MPC more efficient. These techniques reduce the computational load by solving the Optimal Control Problem (OCP) within a lower-dimensional input subspace. Various approaches to dimensionality reduction, such as Singular Value Decomposition (SVD) [1], move-blocking strategies [2], and orthonormal basis functions [3], have been proposed to address this issue.

For instance, the sub-optimal receding horizon control strategy for input-constrained linear systems is based on an SVD of the Hessian of the quadratic performance index generally considered in MPC. The singular vectors are employed to generate a basis function expansion of the unconstrained solution to the finite-horizon OCP. At each sampling time, a feasible control sequence is determined by selecting a variable subset of the basis representation. No solution to the associated quadratic program is needed. For cases in which the Hessian is poorly conditioned, the proposed strategy can provide a sub-optimal solution with minimal performance degradation.

Move-blocking is a widely used strategy to reduce the degrees of freedom of the OCP arising in receding horizon control. The size of the OCP is reduced by forcing the input variables to be constant over multiple discretization steps [4]. Instead of optimizing at every time step, the control input or its derivatives are assumed to be constant over several time steps, simplifying the problem. In [5], a method is proposed for solving the blocked optimization using highly parallelizable algorithms, which significantly improves the computation times of the OCP.

Orthonormal basis functions refer to a set of functions that are orthogonal to each other and have a unit norm. This method uses orthonormal basis functions, such as Laguerre polynomials or wavelets, to represent the state or control trajectories. By projecting the dynamics onto a lower-dimensional subspace defined by these basis functions, the problem size is reduced.

Active subspace approaches for NMPC [6] offer a promising solution to the dimensionality reduction problem while ensuring recursive feasibility. These methods project the OCP onto a lower-dimensional space using orthogonal projectors derived from sensitivity functions. How-

ever, a key challenge remains: there is currently no established theory on how to select these projectors to ensure closed-loop optimality, which is critical for stable and effective control.

In contrast to MPC, RL learns control laws from closed-loop interactions with the controlled system, with limited knowledge of the system model. Reinforcement learning has many parallels to closed-loop control, especially to MPC, and several works have combined RL and MPC [7]. For example, in MPC-RL, a parameterized OCP is used as a function approximator in RL actor-critic methods, where the parameters of the OCP are improved from closed-loop feedback.

This thesis aims to address the gap in projector selection for active subspace NMPC by leveraging RL to learn the optimal projectors. The goal is to improve closed-loop performance for the underlying dimensionality-reduced OCP by learning an effective projection that captures the optimal policy of the full OCP, thereby reducing the dimensionality without compromising control quality. This approach combines the strengths of RL and active subspace methods to enhance the efficiency and stability of NMPC for nonlinear systems.

The structure of this thesis is as follows: Chapter 2 presents the fundamental theoretical background, covering essential concepts in MPC, RL, and the integration of active subspace methods within these frameworks, setting the stage for the work that follows. Chapter 3 details the core contributions of this thesis, outlining the development and implementation of the MPCQ-learning algorithm tailored to active subspaces, as well as a comprehensive description of the algorithmic structure, design, and underlying approach. Chapter 4 describes the simulation setup, including the test scenarios and evaluation metrics used to validate the proposed approach, and finally discusses the results obtained, evaluating the performance and effectiveness of the proposed methods. Concluding remarks are provided in Chapter 5, where the key outcomes are summarized, and potential avenues for future research are outlined.

2 Literature Review

In this chapter, we present the theoretical foundations of our work. We begin by introducing the core principles of MPC. Following this, we delve into the concept of MPC with active subspaces, highlighting its key principles and extending the concept of active subspaces using Principal Component Analysis (PCA). Finally, we explore the fundamentals of RL, starting from basic concepts and progressing toward our innovative approach. This section forms the core of the theoretical framework of the study.

2.1 Model Predictive Control (MPC)

MPC is an optimal control technique that originated in the chemical process industry, and has since been used in a wide range of applications [8]. The basic concept of MPC is to use a dynamic model to forecast system behavior, and optimize the forecast to produce the best decision/the control move at the current time that minimize a given cost performance. MPC differs, therefore, from conventional control in which the control law is precomputed offline. Models are therefore central to every form of MPC.

MPC is a form of control in which the control action is obtained by solving online, at each sampling instant, a finite horizon OCP in which the initial state is the current state of the plant. Optimization yields a finite control sequence, and the first part of the control action in this sequence is applied to the plant. Open-loop optimal control problems often can be solved rapidly enough, using standard mathematical programming algorithms, to permit the use of MPC even though the system being controlled is nonlinear, and constraints on states and controls must be satisfied [9].

As shown in Figure 2.1, at each time step k, for a given estimate/measurement x_k of the current state of the plant, a prediction over a given time horizon is computed by OCP using a model of the plant. The OCP choose the control input yielding the "best" prediction (i.e. the prediction closest to the desired behavior which involved the minimization of some cost performance).

The first part of the control is apply to the plant and the entire optimization process is repeated again in the next time step $k + 1$ using the updated information. This approach enables the NMPC to dynamically adjust to disruptions and mismatches by integrating system feedback at every stage.

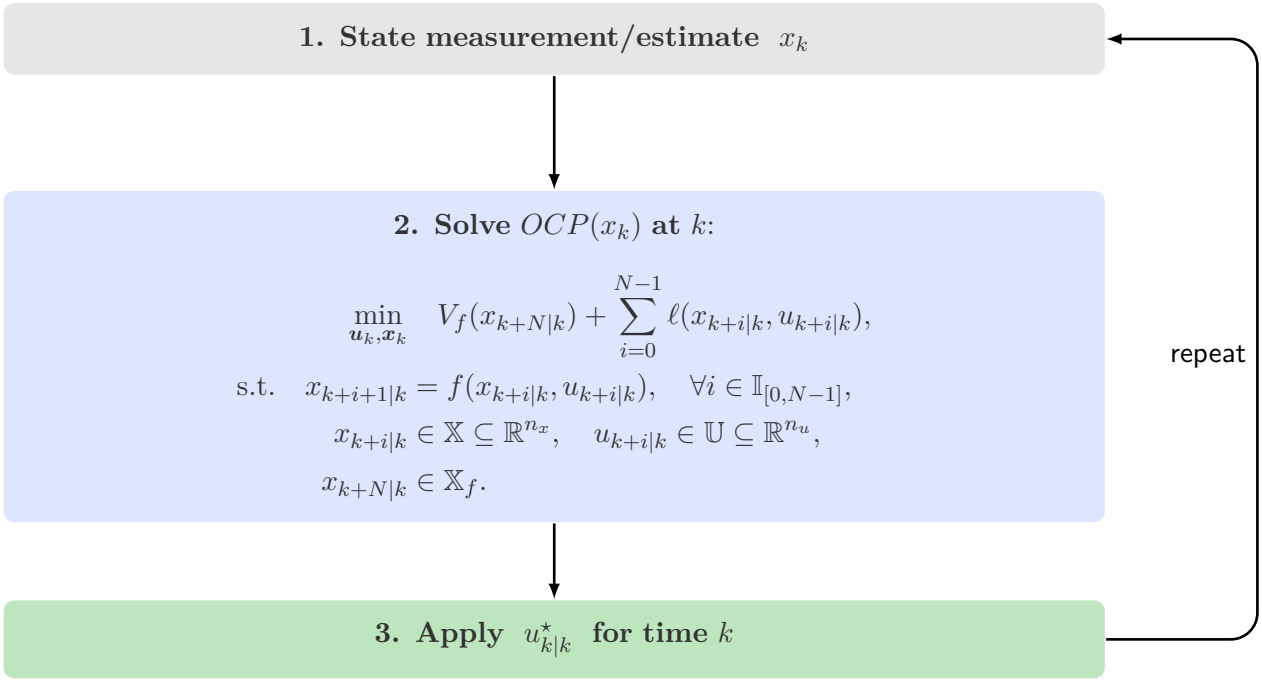

Figure 2.1: Full NMPC closed-loop framework

So, overall, MPC is inherently a multivariable control algorithm that employs an internal model of the process. MPC's main strength is its ability to work with large state-spaces, where constraints may be imposed on both the manipulated and control variables [10]. Due to its simplicity and flexibility, MPC has been successfully applied in a wide range of applications [11, 12].

2.1.1 Problem Formulation

Having introduced the MPC and its nonlinear variant NMPC framework, let us now focus on its central component the OCP. Optimal control is the optimization of an objective function with underlying nonlinear differential equations including time-dependent controls that govern the modeled process that is described by so-called state variables. In general, the controls are the unknowns in this problem class.

The OCP can be formulated for both continuous-time and discrete-time dynamic systems. In this thesis, we will focus exclusively on the discrete-time formulation. Specifically, we present the OCP in its discrete-time form, which is used to stabilize a system governed by the discrete-time dynamics $x_{k+1} = f(x_k, u_k)$ at time instant $k \in \mathbb{N}$, by minimizing a cost performance as in:

$$V(x_k) = \min_{\boldsymbol{u}_k,\boldsymbol{x}_k} V_f(x_{k+N|k}) + \sum_{i=0}^{N-1} \ell(x_{k+i|k}, u_{k+i|k}), \tag{2.1a}$$

$$\text{s.t.} \quad x_{k+i+1|k} = f(x_{k+i|k}, u_{k+i|k}), \quad \forall i \in \mathbb{I}_{[0,N-1]}, \tag{2.1b}$$

$$x_{k+i|k} \in \mathbb{X} \subseteq \mathbb{R}^{n_x}, \quad u_{k+i|k} \in \mathbb{U} \subseteq \mathbb{R}^{n_u}, \tag{2.1c}$$

$$x_{k+N|k} \in \mathbb{X}_f. \tag{2.1d}$$

where $\ell(x_{k+i|k}, u_{k+i|k})$ and $V_f(x_{k+N|k})$ are the stage and terminal costs respectively, x_k is the current system state, while the nominal MPC predicted trajectories are $\boldsymbol{x}_k = \begin{bmatrix} x_{k|k}^\top & \cdots & x_{k+N|k}^\top \end{bmatrix}^\top \in \mathbb{R}^{(N+1)\cdot n_x}$, $\boldsymbol{u}_k = \begin{bmatrix} u_{k|k}^\top & \cdots & u_{k+N-1|k}^\top \end{bmatrix}^\top \in \mathbb{R}^{N\cdot n_u}$. The sets of admissible solutions for the states and inputs are denoted by $\mathbb{X}$ and $\mathbb{U}$ respectively, n_x and n_u represent the input and the state dimensions, N the horizon length and $\mathbb{I}_{[0,N-1]} = \{k \in \mathbb{Z} \mid 0 \leq k \leq N-1\}$.

Note that, formally, the decision variables of the OCP (2.1) are the predicted inputs $\boldsymbol{u}_k$ and the predicted states $\boldsymbol{x}_k$. However, the predicted states $x_{k+i|k}$, $i \in \mathbb{I}_{[1,N]}$, are functions of the initial state x_k and the chosen inputs $\boldsymbol{u}_k$. They can be computed recursively as

$$x_{k+i|k} = f\left(f\left(f\left(x_k, u_{k|k}\right), u_{k+1|k}\right), \ldots, u_{k+i-1|k}\right). \tag{2.2}$$

After the elimination of state variables, which in the context of linear quadratic MPC is also known as condensing, one can reformulate (2.1) as follows [6]

$$\mathscr{P}(x_k): \quad \min_{\boldsymbol{u}_k} \mathcal{J}(x_k, \boldsymbol{u}_k) \quad \text{s.t.} \quad \mathcal{H}(x_k, \boldsymbol{u}_k) \leq 0 \tag{2.3}$$

$$\mathcal{J}: \mathbb{X} \times \mathbb{U}^N \to \mathbb{R}; \mathcal{H}: \mathbb{X} \times \mathbb{U}^N \to \mathbb{R}^{n_H}$$

where in $\mathcal{H}$ we collect all input and state constraints over the horizon N. We call $\mathscr{P}(x_k)$ a *nominal parametric program* with parameter $x_k \in \mathbb{X}$ (the initial state) and decision variable $\boldsymbol{u}_k \in \mathbb{U}^N$.

Typical algorithms used to handle (2.1) and (2.3) are NLP solvers, a class of optimization problems in which some of the constraints or the objective function are nonlinear. In the next section, we will introduce some numerical schemes used to transform the OCP into an NLP.

2.1.2 Direct Single and Multiple Shooting in NLP Formulation

There are many different methods to solve such OCPs numerically. In this next section, we focus on direct methods, namely the direct single and direct multiple shooting approach. We omit the widely used direct collocation method that approximates the control and state space using polynomial expressions. In contrast to that, direct single shooting relies on state-of-the-art numerical integrators to alter between simulating the ODE and iterating the optimization. Direct multiple shooting is a hybrid but nevertheless simultaneous approach, where the state trajectory is split into several independent parts by introducing new artificial initial values. Both direct single and multiple shooting techniques are carried out in order to express the OCP as a NLP. In order to detail the technique behind direct single and multiple shooting, let us formulate a simplified (OCP) as shown below:

Consider the simplified Optimal Control (OC)-NLP defined as follow:

$$\min_{\boldsymbol{u}_k, \boldsymbol{x}_k} \sum_{k=0}^{N-1} \ell(x_k, u_k) + V_f(x_N) \tag{2.4a}$$

$$\text{s.t.} \quad x_{k+1} = f(x_k, u_k), \quad \forall k \in \mathbb{I}_{[0,N-1]}, \tag{2.4b}$$

$$x_0 = x^0, \tag{2.4c}$$

$$0 \leq h(x_k, u_k), \tag{2.4d}$$

$$0 \leq r(x_N). \tag{2.4e}$$

In direct multiple shooting the horizon N is divided into N smaller intervals $[k, k+1]$ called shooting interval. On each shooting interval the OCP is transformed into NLP. The main idea is to solve (2.4) unmodified using any generic NLP solver. The NLP formulation in direct multiple shooting read as follow:

$$\min_{\mathbf{x}} F(\mathbf{x}) = \sum_{k=0}^{N-1} \ell(x_k, u_k) + V_f(x_N) \tag{2.5a}$$

$$\text{s.t. } G(\mathbf{x}) = 0, \tag{2.5b}$$

$$\mathcal{H}(\mathbf{x}) \leq 0, \tag{2.5c}$$

where:

$$\mathbf{x} = \begin{pmatrix} x_0 \\ u_0 \\ \vdots \\ x_N \\ u_{N-1} \end{pmatrix}, \quad G(\mathbf{x}) = \begin{pmatrix} x_0 - x^0 \\ x_1 - f(x_0, u_0) \\ \vdots \\ x_N - f(x_{N-1}, u_{N-1}) \end{pmatrix}, \quad \mathcal{H}(\mathbf{x}) = \begin{pmatrix} h(x_0, u_0) \\ \vdots \\ h(x_{N-1}, u_{N-1}) \\ r(x_N) \end{pmatrix}.$$

with the following mappings:

$$G : \mathbb{R}^{(N+1)n_x + Nn_u} \to \mathbb{R}^{(N+1)n_x}, \quad \mathcal{H} : \mathbb{R}^{(N+1)n_x + Nn_u} \to \mathbb{R}^{n_H}.$$

In direct mutilple shooting constraints are enforced at discretization points only. This approach can lead to infeasible path, i.e., equality constraints might be violated. Multiple shooting keeps the initial states of all shooting intervals as optimization variables, resulting in a total number of decision variables equal to $(N+1) \cdot n_x + N \cdot n_u + N \cdot n_x$. Although the size of the NLP is large, its Jacobians and Hessians are block-sparse, which can be exploited in the numerical solution procedure.

With direct single shooting the OCP is transformed into NLP on one shooting interval. The main idea is to eliminate $(x_0, x_1, \ldots, x_N)$ such that $G(\mathbf{x}) \equiv 0$. Consider the following definitions:

$$\tilde{x}_0 = x^0, \tag{2.6a}$$

$$\tilde{x}_{k+1} = f(\tilde{x}_k, u_k), \quad \forall k \in \mathbb{I}_{[0,N-1]}, \tag{2.6b}$$

$$\tilde{\mathbf{x}} : \mathbb{R}^{N \cdot n_u} \to \mathbb{R}^{(N+1)\cdot n_x + N \cdot n_u}, \quad \boldsymbol{u} = (u_0, \ldots, u_{N-1}) \to \tilde{\mathbf{x}}(\boldsymbol{u}) = \begin{pmatrix} \tilde{x}_0(\boldsymbol{u}) \\ u_0 \\ \vdots \\ \tilde{x}_{N-1}(\boldsymbol{u}) \\ u_{N-1} \\ x_N(\boldsymbol{u}) \end{pmatrix}. \tag{2.6c}$$

By substituting $\tilde{\mathbf{x}}(\boldsymbol{u})$ in (2.5) we are able to eliminate 2.5b and obtain the following NLP formulation in direct single shooting:

$$\min_{\boldsymbol{u}} \tilde{F}(\boldsymbol{u}) = \sum_{k=0}^{N-1} \ell(\tilde{x}_k(\boldsymbol{u}), u_k) + V_f(\tilde{x}_N(\boldsymbol{u})), \tag{2.7a}$$

$$\text{s.t. } \tilde{\mathcal{H}}(\boldsymbol{u}) \leq 0. \tag{2.7b}$$

Where:

$$\tilde{\mathcal{H}}(\boldsymbol{u}) = \begin{pmatrix} h(\tilde{x}_0(\boldsymbol{u}), u_0) \\ \vdots \\ h(\tilde{x}_{N-1}(\boldsymbol{u}), u_{N-1}) \\ r(\tilde{x}_N(\boldsymbol{u})) \end{pmatrix}.$$

The total number of decision variables in single shooting is $N \cdot n_u$ making it to have less variables compare to its multiple shooting counter part, but its Hessians and Jacobian is dense. Single shooting will be workhorse method for solving our OCP in this project.

2.1.3 NMPC in Active Subspaces

MPC and NMPC are powerful optimization tools for complex systems but are computationally expensive, as they typically require a large prediction horizon N to guarantee stability and recursive feasibility of the underlying (OCP) [9]. This becomes particularly computationally demanding for systems with complex nonlinear dynamics. Dimensionality reduction of decision variables is a practical and well-established method to alleviate the computational burden. Existing approaches range from early move-blocking techniques [2] to singular value decomposition [1] and NMPC in active subspaces [6].

By decomposing the space of decision variables related to the inputs into active and inactive complements, NMPC in active subspaces reduces the computational load, allowing real-time control with minimal lose in closed-loop performance. Using global sensitivity analysis, the active subspace can be identified dynamically, enabling the control scheme to adapt to changes in system conditions. This framework not only simplifies the optimization process but also maintains recursive feasibility, ensuring stable and feasible control in high-dimensional, nonlinear environments [6].

Baseline Approach:
Consider the parametrisation of the control input vector $\boldsymbol{u}_k$ defined by

$$T\hat{\mathbf{u}}_k = \begin{bmatrix} T_1 & T_2 \end{bmatrix} \begin{bmatrix} \mathbf{v}_k \\ \mathbf{w}_k \end{bmatrix} = \mathbf{u}_k \tag{2.8}$$

where $T \in \mathbb{R}^{Nm \times Nm}$ such that $TT^\top = I$ is an orthonormal matrix of some generic feature vector $\mathcal{S}$ that map $\boldsymbol{u}_k \in \mathbb{R}^{Nm}$ to new coordinates $(\mathbf{v}_k, \mathbf{w}_k)$. Here $\mathbf{v}_k \in \mathbb{R}^{n_v}$,$\mathbf{w}_k \in \mathbb{R}^{n_w}$ are the active and inactive subspace vectors. For the sake of simplicity and without loss of generality, we consider the case where $T_1 \in \mathbb{R}^{Nn_u \times n_v}$ and $T_2 \in \mathbb{R}^{Nn_u \times (Nn_u - n_v)}$ correspond to the first n_v columns and the remaining $Nn_u - n_v$ columns of T, respectively. We refer to the space spanned by T_1 as active subspace $\mathcal{AS} = \text{col}(T_1)$. If $q \ll N_m$, $\mathcal{AS}$ is a low-dimensional subspace of $\mathbb{R}^{Nn_u}$. The complement spanned by T_2 is denoted as the inactive subspace $\mathcal{IS} = \text{col}(T_2)$.

By performing the optimization on the active subspace vector $\mathbf{v}_k$ together with $\tilde{\mathbf{w}}_k = (T_2)^\top \tilde{\mathbf{u}}_k$ of the inactive subspace vector, the dimensionality reduce OCP with active subspace read as follow [6]:

$$\begin{aligned} \mathscr{P}(x_k, \tilde{\mathbf{w}}_k): \quad & \min_{\mathbf{v}_k, \mu_k} \ \mathcal{J}(x_k, T_1\mathbf{v}_k + \mu_k T_2 \tilde{\mathbf{w}}_k) \\ \text{s.t.} \quad & \mathcal{H}(x_k, T_1\mathbf{v}_k + \mu_k T_2 \tilde{\mathbf{w}}_k) \leq \mathbf{0} \\ \text{where} \quad & \mathcal{J} : \mathbb{X} \times \mathbb{R}^{n_\mathbf{v}} \times \mathbb{R}^{n_\mathbf{w}} \times \mathbb{R} \to \mathbb{R} \\ & \mathcal{H} : \mathbb{X} \times \mathbb{R}^{n_\mathbf{v}} \times \mathbb{R}^{n_\mathbf{w}} \times \mathbb{R} \to \mathbb{R}^{n_\mathcal{H}} \end{aligned} \tag{2.9}$$

With $\mu_k \in \mathbb{R}$ introduced as an extra scaler decision variable to improves the performance of the closed-loop since the suboptimal guess $\tilde{\mathbf{w}}$ can be scaled to zero [6]. We call $\mathscr{P}(x_k, \tilde{\mathbf{w}}_k)$ a *reduced parametric program* with parameter $x_k \in \mathbb{X}$ (the initial state), $\tilde{\mathbf{w}}_k \in \mathbb{R}^{n_w}$ and decision variables $\boldsymbol{v}_k \in \mathbb{R}^{n_v}$ and $\mu_k \in \mathbb{R}$. The dimensionality reduce nonlinear programming problem is solved using single shooting.

Using (2.9) in conjunction with the update rule for the inactive subspace vector $\tilde{\mathbf{w}}_k = (T_2)^\top \tilde{\mathbf{u}}_k$ to ensure recursive feasibility, the algorithm for the NMPC in the active subspace is formulated as follows:

Algorithm 1 Active-subspace NMPC with $\mathscr{P}(x_k, \widetilde{\mathbf{w}}_k)$

Input: T_1, T_2, κ, $x_0 \in X_0$

1: Solve $\mathscr{P}(x_0)$ for $\widetilde{\mathbf{u}}_0$, set $\widetilde{\mathbf{w}}_0 \leftarrow T_2^\top \widetilde{\mathbf{u}}_0$, $k \leftarrow 0$
2: Estimate x_k, solve $\mathscr{P}(x_k, \widetilde{\mathbf{w}}_k)$ from (2.9) for $(\mathbf{v}_k^*, \mu_k^*)$
3: **if** $\mathcal{J}(x_k, T_1\mathbf{v}_k^* + \mu_k^* T_2 \widetilde{\mathbf{w}}_k) \leq \mathcal{J}(x_k, \widetilde{\mathbf{u}}_k)$ holds **then**
4: $\quad \mathbf{u}_k \leftarrow T_1\mathbf{v}_k^* + \mu_k^* T_2 \widetilde{\mathbf{w}}_k$
5: **else**
6: $\quad \mathbf{u}_k \leftarrow \widetilde{\mathbf{u}}_k$
7: **end if**
8: Apply the first step of $\mathbf{u}_k$
9: Update $\widetilde{\mathbf{u}}_k$ by $\widetilde{\mathbf{u}}_k = \left[u_{k|k-1}^\top, \quad \ldots, \quad u_{k+N-2|k-1}^\top, \quad \kappa(x_{k+N-1|k-1})^\top\right]$, $\widetilde{\mathbf{w}}_k \leftarrow T_2^\top \widetilde{\mathbf{u}}_k$
10: $k \leftarrow k+1$, and go to Step 2.

To ensure stability guarantee of Algorithm 1, we compute $\tilde{\mathbf{u}}_\mathbf{k}$ by appending a state feedback control law $\kappa(x_{k+N-1|k-1})$ at the end of the previous predicted trajectory $\mathbf{u}_{k-1}$.

$$\tilde{\mathbf{u}}_k = \left[\mathbf{u}_{k-1}, \kappa(x_{k+N-1|k-1})^\top\right] = \left[u_{k|k-1}^\top, \ldots, u_{k+N-2|k-1}^\top, \kappa(x_{k+N-1|k-1})^\top\right]$$

Moreover the computation of $\tilde{\mathbf{w}}_k$ using $\tilde{\mathbf{w}}_k = (T_2)^\top \tilde{\mathbf{u}}_k$ follows the classic construction of feasible solutions of Algorithm 1 [13, 14].One of the benefits of using Algorithm 1 is that, for any choice of the generic feature vector $\mathcal{S}$, we can guarantee both stability and recursive feasibility of the dimensionality-reduced MPC scheme(see [6] for more technical detail).

Computation of active subspaces via sensitivities analysis

So far, we have introduced the concept of NMPC in active subspaces and explained how it uses the active and inactive subspace projectors, T_1 and T_2, derived from a given generic feature vector $\mathcal{S}$, to guarantee recursive feasibility and asymptotic stability. To this end, we introduce a scheme for computing computing T_1 and T_2 from a generic feature vector $\mathcal{S}$.

Let a generic feature vector $\mathcal{S} : \mathbb{X}_0 \to \mathbb{R}^{N\times m}$ related to the original optimization problems (2.1), respectively, (2.9) be given. We consider:

$$C = \frac{1}{|\mathbb{X}_0|} \int_{\mathbb{X}_0} \mathcal{S}(x)\mathcal{S}(x)^\top \, dx, \tag{2.10}$$

i.e., C is the covariance matrix of $\mathcal{S}$ over the considered set of initial conditions $\mathbb{X}_0$, and $|\cdot|$ denotes the Lebesgue measure of a set. We note that C is positive semi-definite and symmetric. Thus, C admits the non-negative real eigendecomposition.

$$\begin{aligned} &C = T\Sigma T^\top, \\ &\Sigma = \operatorname{diag}(\sigma_1, \ldots, \sigma_{Nm}), \quad \sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_{Nm} \geq 0. \end{aligned} \tag{2.11}$$

where T is the $Nm \times Nm$ orthonormal matrix spanned by the normalized eigenvectors of C, and $\sigma_1 \geq \cdots \geq \sigma_{Nm}$ are the corresponding eigenvalues. The dimensionality reduction means to consider the first q eigenvectors which span the active subspace $\mathcal{AS}$. Naturally, the choice of the dimension $q \in \mathbb{I}_{[1,Nm]}$ is problem-specific and relies on the spectrum of C. If the dimension of the active subspace q is chosen such that $\sum_{i=1}^{q} \sigma_i \geq \sum_{i=q+1}^{N_m} \sigma_i$, then, on average, the performance varies much more in the $\mathcal{AS}$ than in the inactive subspace $\mathcal{IS}$.

Computing the elements of C from (2.10) becomes challenging, as one must evaluate integrals over the high-dimensional space $\mathbb{X}$. To address this, we opt for a numerically tractable approximation of C given by:

$$\hat{C} = \frac{1}{M} \sum_{j=1}^{M} \mathcal{S}(x^j)\mathcal{S}(x^j)^\top \tag{2.12}$$

Where $M = |\mathbb{X}_{\text{train}}|$ and $\mathbb{X}_{\text{train}}$ consist of a randomly chosen set of samples defined by

$$\mathbb{X}_{\text{train}} = \{x^j \in \mathbb{X}_0 \mid j = 1, \ldots, M\}$$

Using 2.10 together 2.11, we can directly deduce that the projector matrix T can be computed either directly from a suitable choice of the C matrix or from the reconstruction of the C matrix using a generic feature vector $\mathcal{S}(x)$.

At this point, we will discuss both methods in detail. Specifically, we will first explore the method of computing the projector matrix T directly from a suitable choice of the C matrix, and then examine the alternative approach of reconstructing the C matrix using a generic feature vector $\mathcal{S}(x)$.

First, we focus on the method of directly computing the projector matrix T from a suitable choice of the C matrix, we assume that there exists a suitable choice of C that can be obtained directly, without using (2.10). In this case, we can directly perform the eigendecomposition defined in (2.11) to obtain the eigenvectors T and eigenvalues Σ of the matrix C. We illustrate this concept by considering the condensed, parametrized Linear Quadratic MPC scheme:

$$\mathscr{P}(x_k): \quad \min_{\mathbf{u}} \ \mathcal{J}_N(x_k, \mathbf{u})$$

$$\text{s.t.} \quad \mathcal{H}(x_k, \mathbf{u}) \leq 0$$

where:

$$\begin{aligned}
\mathcal{J}_N(x_k, \mathbf{u}) &= x_k^\top Y x_k + \mathbf{u}^\top H \mathbf{u} + 2\mathbf{u}^\top F x_k \\
Y &= Q + \Lambda^\top \mathbf{Q} \Lambda \in \mathbb{R}^{n_x \times n_x}, \\
H &= \mathbf{R} + \Gamma^\top \mathbf{Q} \Gamma \in \mathbb{R}^{N n_u \times N n_u}, \\
F &= \Gamma^\top \mathbf{Q} \Lambda \in \mathbb{R}^{N n_u \times n_x}, \\
\mathbf{Q} &= \operatorname{diag}(Q, \ldots, Q, P) \in \mathbb{R}^{n_x \times n_x}, \\
\mathbf{R} &= \operatorname{diag}(R, \ldots, R) \in \mathbb{R}^{n_u \times n_u}.
\end{aligned}$$

and

$$\Lambda = \begin{bmatrix} A \\ A^2 \\ \vdots \\ A^N \end{bmatrix} \in \mathbb{R}^{Nn_x \times n_x}, \qquad \Gamma = \begin{bmatrix} B & 0 & \dots & 0 \\ AB & B & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ A^{N-1}B & A^{N-2}B & \dots & B \end{bmatrix} \in \mathbb{R}^{Nn_u \times n_x},$$

$$\mathcal{H}(x_k, \mathbf{u}) = \begin{pmatrix} h(\tilde{x}_k(\mathbf{u}), u_{k|k}) \\ \vdots \\ h(\tilde{x}_{k+N-1|k}(\mathbf{u}), u_{k+N-1|k}) \\ r(\tilde{x}_{k+N|k}(\mathbf{u})) \end{pmatrix} \in \mathbb{R}^{n_{\mathcal{H}}}$$

The matrices Q, R, and P are positive semi-definite weight matrices for the state, control input, and terminal state, respectively. A and B represent the state and input matrices. Matrices Λ and Γ define the relationship between the vector of control moves $\mathbf{u}$ and the vector $\mathbf{x}$ which contains the future time evolution of the state vector of the system.

The Hessian of the Lagrangian of the constrained optimization problem is computed as:

$$\nabla^2 \mathcal{L}(x_k, \mathbf{u}, \lambda_0, \mu) = \lambda_0 \nabla^2 \mathcal{J}_N(x_k, \mathbf{u}) + \sum_{i=0}^{m} \mu_i \nabla^2 \mathcal{H}_i(x_k, \mathbf{u}) \tag{2.13}$$

For unconstrained problem and in cases where no constraints are active, (2.13) is reduce to:

$$\nabla^2 \mathcal{L}(x_k, \mathbf{u}, \lambda_0, \mu) = \lambda_o \nabla^2 \mathcal{J}_N(x_k, \mathbf{u}) = H, \quad \lambda_0 = 1$$

The Hessian of the objective function in this case is a suitable candidate for the C matrix, i.e., $C = H$ [1]. In the case of non-linear dynamics, the Hessian of the objective function for the linearized dynamics can be considered an approximation of the C matrix. However, this approximation may lead to a loss of information if the system operates far from its equilibrium point. If one or more constraints are active then the choice of $C = H$ might not likely provide sufficient information.

Another possible choice for the C matrix is given by $I_{n\times p}$, where $n = p$. Here, $I_{n\times p}$ is a block identity matrix defined as:

$$I_{n\times p} = \begin{bmatrix} \delta_{11} & \delta_{12} & \cdots & \delta_{1p} \\ \delta_{21} & \delta_{22} & \cdots & \delta_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ \delta_{n1} & \delta_{n2} & \cdots & \delta_{np} \end{bmatrix},$$

where δ_{ij} is the Kronecker delta function defined as:

$$\delta_{ij} = \begin{cases} 1 & \text{if } i = j, \\ 0 & \text{if } i \neq j. \end{cases}$$

The choice of $C = I_{n\times p}$ is closely linked-yet not identical-to a move blocking structure, i.e., the first few control steps are considered most important.

Next, we turn our attention to the alternative method, which involves reconstructing the C matrix using a generic feature vector $\mathcal{S}(x)$. This approach enables the computation of the projector matrix T indirectly, relying on the relationship defined by (2.12). The design of $\mathcal{S}(x)$ which in turn determines T is a degree of freedom. Since recursive feasibility of the proposed active-subspace NMPC does not depend on the actual choice of the subspace. For example, with $\mathcal{S}(x) = u^\star(x)$, our generic active-subspace framework includes SVD with respect to $u^\star(x)$ as suggested in [15].

Moreover, we remark that for unconstrained problems and for cases in which no constraint is active, optimality implies $\nabla_u J(x, u^\star) = 0$. Hence, if only a few sampled OCPs admit active constraints, the choice $\mathcal{S}(x) = \nabla_u J(x, u^\star(x))$ likely does not provide sufficient information. In this case, one may include the subspace spanned by corresponding local LQR solutions obtained for the linearized system at the equilibrium in the active subspace $\mathcal{AS}$.

Computing active subspaces with PCA

Here, we detail the concept of the PCA technique for dimensionality reduction and use this idea to evaluate equation (2.12) with our choice of feature vector $\mathcal{S}(x) = u^\star(x)$.

PCA is a very popular technique for dimensionality reduction. Given a set of data in n-dimensions, represented as an $n \times t$ matrix X, where each column corresponds to an n-dimensional observation, PCA aims to find a linear subspace of dimension d, lower than n, such that the data points lie mainly on this linear subspace [16]. Such a reduced subspace attempts to maintain most of the variability of the data.

The linear subspace can be specified by d orthogonal vectors that form a new coordinate system, called the 'principal components'. The principal components are orthogonal, linear transformations of the original data points, so there can be no more than n of them. However, the expectation is that only $d < n$ principal components are necessary to approximate the space spanned by the original n-dimensional axes.

In order to capture as much of the variability as possible, let us choose the first principal component, denoted by U_1, to have maximum variance. Suppose that all centered observations are stacked into the columns of X. Let the first principal component be a linear combination of X defined by coefficients (or weights) $w = [w_1 \ \dots \ w_n]$. In matrix form:

$$U_1 = w^\top X$$

$$\mathrm{var}(U_1) = \mathrm{var}(w^\top X) = w^\top C w$$

where C is the $n \times n$ sample covariance matrix of X. Clearly, $\text{var}(U_1)$ can be made arbitrarily large by increasing the magnitude of w. Therefore, we choose w to maximize $w^\top Cw$ while constraining w to have unit length [16]:

$$\begin{aligned} &\max w^\top Cw \\ &\text{subject to} \quad w^\top w = 1 \end{aligned} \tag{2.14}$$

From this point onward, we aim to deduce that by solving (2.14), we can directly obtain the projector T_1. To achieve this, let us consider the Lagrangian of the constrained optimization problem defined by:

$$\mathcal{L}(w, \lambda_1) = w^\top Cw - \lambda_1(w^\top w - 1) \tag{2.15}$$

Where λ_1 is the Lagrange multiplier . Differentiating with respect to w gives n equations,

$$Cw = \lambda_1 w$$

Premultiplying both sides by $w^\top$, we have:

$$w^\top Cw = \lambda_1 w^\top w = \lambda_1$$

$\text{var}(U_1)$ is maximized if λ_1 is the largest eigenvalue of C. Clearly, λ_1 and w are an eigenvalue and an eigenvector of $\mathcal{S}$. Differentiating (2.15) with respect to the Lagrange multiplier λ_1 gives us back the constraint:

$$w^\top w = 1$$

This shows that the first principal component is given by the normalized eigenvector with the largest associated eigenvalue of the sample covariance matrix C. A similar argument can show that the d dominant eigenvectors of the covariance matrix C determine the first d principal components. The d dominant vectors then form the active subspace projector T_1 for the given choice of covariance matrix C, while the remaining $t-d$ eigenvectors of C constitute the inactive subspace projector T_2.

PCA is a data-driven method that relies on the given dataset to compute the sample covariance matrix C. We have deduced that, with a chosen sample covariance matrix C, it is possible to determine the active subspace projector T_1. However, we have not yet explained how to derive the sample covariance matrix from a given dataset. Therefore, from this point onward, we will illustrate the method of computing C using our chosen feature vector $u^*(x)$.

Consider the set of data points U defined by $U = \left\{u_j^\star(x)\right\}_{j=1}^{M}$ where $u_j^\star(x)$ consist of Nn_u control data points starting from some initial condition x. Suppose that all centered observations of U are stacked into the columns of an $Nn_u \times M$ matrix X, where each column corresponds to an Nn_u-dimensional observation and there are M observations.

The sample covariance C is computed as:

$$C = XX^\top = \frac{1}{M}\sum_{j=1}^{M} u_j^\star(x)u_j^\star(x)^\top \tag{2.16}$$

C is an $Nn_u \times Nn_u$ sample covariance matrix of $u^\star(x)$. By setting $\mathcal{S}(x) = u^\star(x)$ as our feature vector in (2.10), a suitable C matrix can be derived from (2.16). By combining the concept of dimensionality reduction using PCA with our specific design of $\mathcal{S}(x)$, we are able to achieve (2.12).

There are a number of assumptions that were both implicitly and explicitly made in order to motivate and justify the PCA method that is described above.

Linear change of basis: PCA consists essentially of a change of basis, from the Euclidean basis (in which we measure our predictors) to an orthonormal basis of eigenvectors of $w^\top w$. Thus, PCA assumes that such a linear change of basis is sufficient for identifying degrees of freedom and conducting dimensionality reduction.

Mean/variance is sufficient: In applying the PCA technique to our data, we are only using the means (for standardizing) and the covariance matrix that are associated with our predictors. Thus, the method assumes that such statistics are sufficient for describing the distributions of the predictor variables.

High variance indicates importance: Another fundamental assumption that we made in describing the PCA procedure is that the eigenvalues λ_i, which represent the variability in the data and are associated with the i-th principal component, measure the importance of that component. This is intuitively reasonable, since components corresponding to low variability likely say little about the data, which is not always true.

2.2 Reinforcement Learning (RL)

In this section, we begin by introducing the concept of RL. We then proceed to formulate RL for a finite-horizon deterministic system, which constitutes a core part of the framework for our project. Next, we present the Q-learning framework tailored for deterministic systems and, finally, establish the concept of using MPC as a function approximator within the context of our RL problem.

2.2.1 Basics of RL

Reinforcement learning problems involve learning what to do-how to map situations to actions-so as to minimize a numerical reward signal. In an essential way they are closed-loop problems because the learning system's actions influence its later inputs. Moreover, the learner is not

told which actions to take, as in many forms of machine learning, but instead must discover which actions yield the most reward by trying them out.

The reinforcement learning problem is meant to be a straightforward framing of the problem of learning from interaction to achieve a goal (see Figure 2.2). The learner and decision-maker is called the *agent*. The thing it interacts with, comprising everything outside the agent, is called the *environment*. These interact continually, the agent selecting actions and the environment responding to those actions and presenting new situations to the agent. The environment also gives rise to rewards, special numerical values that the agent tries to maximize over time.

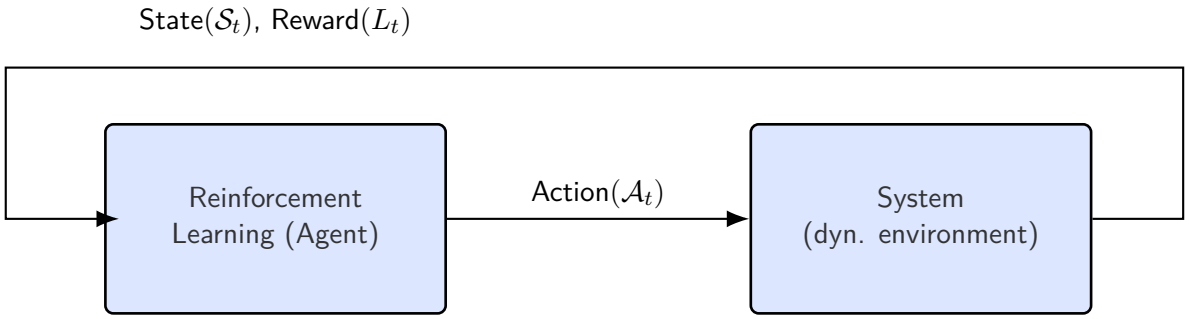

Figure 2.2: The agent-environment interaction in reinforcement learning

We consider a class of reinforcement learning problems in which an agent interacts with a known deterministic system over a finite time horizon. Each interaction is referred to as an *episode*, and the agent's objective is to minimize the expected discounted cumulative reward over episodes.

The system is identified by a sextuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, N, f, L, X_0)$, where $\mathcal{S}$ is the continuous state space, $\mathcal{A}$ is the continuous action space, N is the horizon, f is a discrete time system function (*environment*), L is a reward function and X_0 is a distribution of initial conditions.

If action $a_k \in \mathcal{A}$ is selected while the system is in state $s_k \in \mathcal{S}$ at time instant $k = 0, 1, \ldots, N-1$, a reward of $L(s_k, a_k)$ is realized; furthermore, if $k < N-1$, the state transitions to next state s_{k+1} which is determine by f defined by:

$$s_{k+1} = f(s_k, a_k), \quad s_0 = s^0 \sim X_0 \tag{2.17}$$
$$s_k \in \mathcal{S} \subseteq \mathbb{R}^n, \quad a_k \in \mathcal{A} \subseteq \mathbb{R}^m$$

Each episode terminates at period $N-1$, and then a new episode begins. The initial state of episode j is randomly drawn from the distribution X_0.

To represent the history of actions and observations over multiple episodes, we will often index variables by both episode and period. For example, $s_{j,k}$ and $a_{j,k}$ denote the state and action at period k of episode j, where $j = 0, 1, \ldots, K$ and $k = 0, 1, \ldots, N-1$.

From here onward we will formulate our RL problem as an OCP. Consider the following deterministic Discounted Optimal Control Problem (dOCP) defined by:

$$V_N(s_k) = \min_{a_0,\ldots,a_{N-1}} \sum_{k=0}^{N-1} \gamma^k L(s_k, a_k) + \Phi(s_N), \tag{2.18a}$$

$$\text{s.t.} \quad s_{k+1} = f(s_k, a_k), \quad \forall k \in \mathbb{I}_{[0,N-1]}, \tag{2.18b}$$

$$s_0 = s^0 \sim X_0, \tag{2.18c}$$

$$s_k \in \mathcal{S}, \quad a_k \in \mathcal{A}. \tag{2.18d}$$

where $\gamma \in (0, 1]$ is the discount factor.

A (deterministic) policy $\pi = (\pi_0, \ldots, \pi_K)$ is a sequence of functions, each mapping $\mathcal{S}$ to $\mathcal{A}$. The main objective of our RL problem is to seek a deterministic control policy π that minimizes the following closed-loop performance J:

$$J(\pi) = \mathbb{E}_{s_0 \sim \mathbf{X}_0} \left[\sum_{k=0}^{K} \gamma^k L(s_k, \pi(s_k)) \right] \tag{2.19}$$

For each policy π define a value function V_N^π that provides the discounted sum of cumulative reward for policy π, starting from given initial conditions $s \sim \mathbf{X}_0$:

$$V_N^\pi(s) = \sum_{k=0}^{N-1} \gamma^k L(s_k, \pi(s_k)) \tag{2.20}$$

The optimal value function is defined by $V^\star(s) = \min_\pi V_\pi(s)$. A policy $\pi^\star$ is said to be optimal if $V_{\pi^\star} = V^\star$.

It is also useful to define an action-value function Q_π that provide the expected cost for policy π, starting from given initial conditions $s \sim \mathbf{X}_0$, and using action a as first input then follow the policy π thereafter:

$$Q_N^\pi(s, a) = L(s, a) + \sum_{k=1}^{N-1} \gamma^k L(s_k, \pi(s_k)). \tag{2.21}$$

So far, we have reformulated our RL problem as a closed-loop OCP and expressed the objective of our RL problem in terms of the closed-loop performance of the OCP. Before proceeding further, it is essential to discuss the requirements for achieving optimality in our closed-loop OCP. To this end, we introduce Bellman's Dynamic Programming (DP) principle, which serves as a fundamental requirement for the optimality of the closed-loop OCP. According to this principle, the optimal value function for our deterministic dOCP, where $N \in \mathbb{R}$ and $S = \mathbb{R}^{n_x}$, satisfies [17, 18]:

$$V_N^\star(s_k) = \min_{a_k \in \mathcal{A}} (L(s_k, a_k) + \gamma V_{N-1}^\star(f(s_k, a_k))). \tag{2.22}$$

with boundary condition $V_0(s_k) = \Phi(s_N)$.

The optimal state-action valued function $Q_N^* : \mathbb{R}^{n_x} \times \mathbb{R}^{n_u} \to \mathbb{R}$ is defined as:

$$Q_N^*(s_k, a_k) = L(s_k, a_k) + V_{N-1}^\star(f(s_k, a_k)), \tag{2.23}$$

where $\mathbb{R}^{n_u} = \mathcal{A}$ Thus, the optimum value function V_N^* and the optimum policy π^* satisfy:

$$V_N^\star(s_k) = \min_{a_k \in \mathcal{A}} Q_N^\star(s_k, a_k), \tag{2.24}$$

$$\pi^\star(s) = \arg\min_{a_k \in \mathcal{A}} Q_N^\star(s_k, a_k) \tag{2.25}$$

The, the optimal action-value function $Q^\star$ can rewritten as:

$$Q_N^\star(s_k, a_k) = L(s_k, a_k) + \gamma \min_{a_{k+1}} Q_{N-1}^\star(f(s_k, a_k), a_{k+1}). \tag{2.26}$$

So far we have introduced our RL framework from an optimal control perspective, as such aim to learn the optimal feedback policy. Conceptually this can be achieved via learning the optimal value function, or via the optimal state-action value function or learning the policy directly. These learning approaches vary based on the considered settings, such as model knowledge, model structure, reward structure, etc,. Indeed it is possible to explain these approaches using policy, model and value function.

One of the most crucial aspects of an RL algorithm is the question of whether the agent has access to or learns the model of the environment: this element enables the agent to predict state transitions and rewards. In model-based: exact models of f, L are available in closed form and the transition probabilities are available whereas in model-free: the model f and l can be evaluated through black-box simulators (or experiments), the transition probabilities (and expectations) are approximated through sampling.

In this project we are going to considers a reinforcement learning framework in which the agent initially knows the state space $\mathcal{S}$, the action space $\mathcal{A}$, the exact system function f in closed form, the reward function L, and prior information about the value function but does not know anything else about the the sequence of the initial states S.

The use of policy or value function as the central part of the method represents another essential distinction between RL algorithms. The optimal input depends on the knowledge of the value function and solving the DP equations become intractable as the state space become large, hence approximation methods are needed. Here we distinguish two methods, approximation done in value space: where we learn either the value function $V : \mathbb{R}^{n_x} \to \mathbb{R}$ or the state-action value function $Q : \mathbb{R}^{n_x} \times \mathbb{R}^{n_u} \to \mathbb{R}$ and approximation done in policy space: where we learn the policy $\mathbb{R}^{n_x} \to \mathbb{R}^{n_u}$.

The learning setting could be online or offline. In offline, one learns the approximation of V/Q before the current state x_k is known and computes the control action via the DP minimization

(2.26). Whereas online, learns the approximation of V/Q once the current state x_k is known and compute the control actions via the DP minimization.

For our project, we will focus exclusively on learning in the value space, specifically within the family of model-free RL algorithms known as Temporal-Difference (TD) methods. TD methods play a significant role in reinforcement learning tasks. As detailed in [Sutton and Barto, 1998], three primary approaches are identified for solving such tasks: DP, Monte Carlo (MC) methods, and TD Learning, with the latter being described as the most central and innovative concept in reinforcement learning.

MC [19] can learn from episodes of experience the simple idea that averaging sample returns provide the value. This leads to the main caveat of these methods: they work only with episodic control systems because the episode has to terminate before it is possible to calculate any returns. The total reward accumulated in an episode and the distribution of the visited states is used to calculate the value function while the improvement step is carried out by making the policy greedy concerning the value function.

TD learning is an approach made by combining ideas from both MC methods and Dynamic Programming (DP). TD uses bootstrapping to make updates as in dynamic programming. The central distinction from MC approaches is that TD methods calculate a *temporal error* instead of using the total accumulated reward. The temporal error is the difference between the new estimate of the value function and the old one. Furthermore, they calculate this error considering the reward received at the current time step and use it to update the value function: this means that these approaches can work with continuing (non-terminating) environments. This type of update reduces the variance compared to Monte Carlo one but increases the bias in the estimate of the value function because of bootstrapping.

The fundamental update equation for the value function is shown in (2.27), where TD *error* and TD *target* are in evidence.

$$V(s_k) \leftarrow V(s_k) + \alpha \overbrace{\left(\underbrace{L(s_k, a_k) + \gamma V(s_{k+1})}_{\text{TD target}} - V(s_k) \right)}^{\text{TD error}(\delta_k)} \tag{2.27}$$

where $\alpha \in (0, 1]$ is the step size.

Two TD algorithms for the control problem which are worth quoting because of their extensive use to solve RL problems are *State-Action-Reward-State-Action (SARSA)* and *Q-learning.*

SARSA (on-policy) and *Q-learning* (off-policy) are TD algorithms that first aim to learn an action-value function rather than a state-value function. The main difference between SARSA and Q-learning lies in the update rule for the Q-function: SARSA updates the Q-function based on the current policy the agent is following, whereas Q-learning updates the Q-function under

the assumption that the agent always takes the best possible action (e.g., an ϵ-greedy action) in the next state, regardless of the policy being followed during exploration.

From this point onward, Q-learning will serve as the primary method for addressing our RL problem. It constitutes a central component of our MPCQ-learning algorithm, which will be discussed in detail in a later chapter. Our decision to employ Q-learning is based on the specific design of our RL problem and is motivated by the considerations outlined in [20, 21]. Next, we will delve into the details of Q-learning, specifically in relation to the design of our RL problem.

2.2.2 Q-learning framework

Q-learning [18] is an off-policy TD control algorithm which represents one of the early revolution and advance in reinforcement learning. As mentioned above, its primary goal is to learn an optimal policy by approximating the action-value function.

Here, we discuss the Q-learning framework for nonlinear, deterministic state-space models. In this problem setting, we consider an agent interacting with an environment E with the goal of minimizing the discounted sum of expected future rewards. We focus on discrete-time deterministic systems, where the core concepts are most clearly illustrated. In this context, s and a represent the closed-loop state and control trajectories, respectively.

We consider a real systems that can be described as discrete time systems with continuous state and action spaces. For a given action a_k in state s_k, the function f in (2.17) determine the next state s_{k+1}.

The Q-learning update rule is given in equation (2.28), while algorithm 2 summarizes its pseudocode.

$$Q(s_k, a_k) \leftarrow Q(s_k, a_k) + \alpha[L(s_k, a_k) + \gamma \min_{a_{k+1}} Q(f(s_k, a_k), a_{k+1}) - Q(s_k, a_k)] \tag{2.28}$$

The strategies presented in Algorithm 2 work smoothly with systems that have well-defined states and actions, i.e., finite discrete state spaces. In this context, it is reasonable to use lookup tables to represent the action-value function, where each entry in the table consists of a state-action pair. It is easy to understand how this setting cannot scale up with very large control systems: problems regarding the availability of memory arise as it becomes difficult to manage the storage of a large number of states and actions. Also, there may be obstacles concerning the slowness of learning the value of each state individually. Furthermore, the tabular form could lead to expensive computation in linear lookup and can not work with continuous action and state spaces.

Function approximators represent the solution to overwhelm this problem. The underlying concept is as follow: Consider a parameterized family $\{Q_\theta : \theta \in \mathbb{R}^d\}$; each a real-valued function on $\mathcal{S} \times \mathcal{A}$. For example, the vector θ might represent weights in a neural network. The aim is to approximate the optimal state-action value function $Q^\star$ within this parameterized

Algorithm 2 Q-learning (off-policy TD control) for estimating $\pi \approx \pi^*$

Input: Model f, action space $\mathcal{A}$, state space $\mathcal{S}$, step size $\alpha \in (0, 1]$, discount factor $\gamma \in [0, 1]$, distribution of initial conditions X_0

Initialize: $Q(s, a)$ arbitrarily $\forall s \in \mathcal{S}, a \in \mathcal{A}$, except that $Q(\text{terminal}, \cdot) = 0$

for each j in K episodes **do**
  initialize $s_k : s_0 = s^0 \sim X_0$
  Choose a_k from s_k using a policy derived from Q (e.g., ϵ-greedy)
  while s_k is not terminal **do**
    Take action a_k, evaluate reward $L(s_k, a_k)$ and next state $s_{k+1} = f(s_k, a_k)$
    Update the Q-function:

$$Q(s_k, a_k) \leftarrow Q(s_k, a_k) + \alpha \left[L(s_k, a_k) + \gamma \min_{a_{k+1}} Q(s_{k+1}, a_{k+1}) - Q(s_k, a_k) \right]$$

    Update $s_k \leftarrow s_{k+1}$
    Choose a_k from s_k using a policy derived from Q (e.g., ϵ-greedy)
  end while
end for

family by tuning the parameter vector using TD. We assumed that , there exists a parameter vector θ^* such that:

$$Q_{\theta^\star}(s, a) = Q^\star(s, a) \tag{2.29}$$

We acknowledge that assumption (2.29) is strong. If it is not possible to capture $Q^\star$, RL will find the best parameters among the set of functions provided by the selected parametrization (see, e.g.,[22]).

Learning process

The learning process aims to seek a set of parameters θ that results in the best possible function approximation for a specific objective. For deterministic systems, Q-learning uses the following Least-Square (LS) problem for the parameters θ (see e.g.[23]) in the discounted setting:

$$\min_{\boldsymbol{\theta} \in \Theta} \mathbb{E}_{s_0 \sim X_0} \left[\frac{1}{K} \sum_{k=0}^{K-1} \left(Q_{\boldsymbol{\theta}}(s_k, a_k) - Q^\star(s_k, a_k) \right)^2 \right], \tag{2.30}$$

where $\mathbb{E}_{s_0 \sim X_0}$ denotes the expectation taken over the trajectory distribution induced by the parametrize policy π_θ with possible addition of small random exploration. This formulation ensures that the parameterized action-value function $Q_\theta(s_k, a_k)$ closely approximates the optimal Q-function $Q^\star(s_k, a_k)$, thereby enabling the derivation of an optimal policy $\pi^\star$. Note that the random initial condition increases the visited state-input pair domain and results in a better approximation in the Q-learning [24]. The solution of θ in (2.30) yields the optimal parameters $\theta^\star$ that asymptotically capture the optimal action-value function $Q^\star(s_k, a_k)$.

We should note that our condition for fully capturing the optimal cost $Q^\star$ is based on the assumption that the parametrization provided by the universal cost Q_θ is sufficiently rich and that the number of data points K is large enough [25]. Moreover, the data must cover the state input space sufficiently, e.g., using random initial state. However, these conditions may not hold in practice.

TD learning is a common way to tackle (2.30) [23]. More specifically, a basic TD-based learning step uses the following update rule for the parameters θ at time instance k in the discounted setting:

$$\delta_k = L(s_k, a_k) + \gamma V_\theta(s_{k+1}) - Q_\theta(s_k, a_k), \tag{2.31a}$$

$$\theta \leftarrow \theta + \alpha \delta_k \nabla_\theta Q_\theta(s_k, a_k) \tag{2.31b}$$

The gradient $\nabla_\theta Q_\theta(s_k, a_k)$ is calculated at θ_k. This algorithm generates a sequence of the parameters θ_k that converge to the parameters that have the best estimation of the exact optimal action-value function. The convergence conditions for the Q-learning method can be found in e.g [18].

So far, we have examined the concept of learning the optimal state-action value function $Q^\star$ using the function approximator Q_θ. The objective of doing this is base on the fact that if we can use Q_θ to infer $Q^\star$ then the optimal policy $\pi^\star$ can be obtain from 2.25. This approach is known as *value base* learning methods. The preponderance of optimizations belonging to this category is *off-policy*: this means that the optimization step is done using all data collected during the whole training, not only with the most recent policy available. In this configuration, the information used for the learning phase could also come from exploration decision, apart from ones obtained with the most recent policy. Value-based approaches are more sample efficient because they can reuse data more efficiently, but they are considered less stable than policy gradient ones.

2.2.3 MPC as a function approximator for RL

Most modern RL research uses function approximators in a deep configuration. Specifically, Deep Reinforcement Learning methods use Neural Networks (NNs) as function approximators for the Q-function [26]. However, the use of MPC to support the approximations of V_θ and Q_θ has been proposed and justified in [7]. In our project we, aim to use the MPC scheme described in (2.9) as function approximator for our MPCQ-learning algorithm which will be discuss in the later chapter. It is worthwhile to explain the concept behind using MPC as a function approximator for Q-learning.

In the remainder of this section, we parameterize the stage cost, and dynamic of a tracking MPC scheme. The parameterized tracking MPC scheme is then utilized as a function approximator to estimate the optimal action-value function.

Let us consider the following finite-horizon MPC-scheme parameterized by θ:

$$V_\theta(s) = \min_{\boldsymbol{u},\boldsymbol{x}} \; V_f(x_N) + \sum_{k=0}^{N-1} \ell_\theta(x_k, u_k) \tag{2.32a}$$

$$\text{s.t.} \quad x_{k+1} = f_\theta(x_k, u_k) \quad \forall k = 0, \ldots, N-1, \tag{2.32b}$$

$$x_0 = s \tag{2.32c}$$

$$h(u_k) \leq 0 \tag{2.32d}$$

where $\boldsymbol{x} = \{x_0, \ldots, x_N\}$, $\boldsymbol{u} = \{u_0, \ldots, u_{N-1}\}$ are the primal decision variables, N is the prediction,ℓ and V_f are the stage and terminal costs, respectively.

By incorporating an additional initial input constraint into (2.32), the parameterized action-value function of the MPC scheme is defined as:

$$Q_\theta(s, a) = \min_{\boldsymbol{u},\boldsymbol{x}} \; V_f(x_N) + \sum_{k=0}^{N-1} \ell_\theta(x_k, u_k) \tag{2.33a}$$

$$\text{s.t.} \quad x_{k+1} = f_\theta(x_k, u_k) \quad \forall k = 0, \ldots, N-1, \tag{2.33b}$$

$$x_0 = s \tag{2.33c}$$

$$\boxed{u_0 = a} \tag{2.33d}$$

$$h(u_k) \leq 0 \tag{2.33e}$$

We propose to use the action-value function approximation $Q_\theta \approx Q^*$ obtained from (2.32)[7]. The parameter vector θ can be modified by the Q-learning algorithm using the TD update rule in (2.31) to shape the action-value function. Under some mild assumptions (see [7] for the technical details), if the parametrization is rich enough, the MPC scheme is able to capture the true optimal action-value function Q^*, value function V^*, and policy π^* jointly, even if the MPC model f does not capture the real system dynamics.

The parameterized deterministic policy π_θ reads as follows:

$$\pi_\theta(s) = u_0^\star(s, \theta) \tag{2.34}$$

Where $u_0^\star(s, \theta)$ is the first element of $\boldsymbol{u}^\star$ solution of the MPC scheme (2.32).

Therefore, the value function $V_\theta(s)$ can be acquired together with the policy $\pi_\theta(s)$ by solving the classic MPC scheme in (2.32), while the action-value function results from solving the same MPC scheme with its first input constrained to a specific value a.

The sensitivity $\nabla_\theta Q_\theta(s, a)$ required in (2.31b) is given by [7]:

$$\nabla_\theta Q_\theta(s, a) = \nabla_\theta \mathcal{L}_\theta(s, a, y^\star) \tag{2.35}$$

where $\mathcal{L}$ is the Lagrange function associated with the MPC (2.33), i.e.,

$$\mathcal{L}_\theta(y) = \Phi_\theta + \lambda^\top G_\theta + \mu^\top H_\theta \tag{2.36}$$

where Φ_θ is the cost (2.33a), G_θ gathers the equality constraints (2.33b), (2.33c), (2.33d), and H_θ collects the inequalities (2.33e). The dual variables λ and μ are associated with the constraints. The argument y is defined as $y = \{x, u, \lambda, \mu\}$, and $y^\star$ is the primal dual solution to the constrained OCP (2.33).

Algorithm 3 summarizes the pseudocode for Q-learning with an MPC-based function approximator.

Algorithm 3 MPC-based Q-learning method

Input: Model f, objective function, initial parameter θ_0, step size $\alpha \in (0, 1]$, discount factor $\gamma \in [0, 1]$, distribution of initial conditions X_0
Output:locally optimal policy $\pi_{\theta^\star}$

repeat
  for each j in K episodes **do**
    initialize $s_k : s_0 = s^0 \sim X_0$
    while not done **do**
      solve the MPC in (2.32) and get $y^\star$
      calculate and record the RL stage cost $L(s_k, a_k)$ and $s_{k+1} = f(s_k, a_k)$
      calculate and record $V_\theta(s_{k+1})$, $Q_\theta(s_k, a_k)$ using (2.32) and (2.33)
      calculate and record the sensitivity $\nabla_\theta Q_\theta(s_k, a_k)$ according to (3.9)
      calculate and record the TD error δ_k using (2.31a)
    end while
    Update the θ according to (2.31b)
  end for
until *convergence*

Experience Replay

RL can be unstable or even diverge when a nonlinear function approximator such as a neural network or MPC is used to represent the value function [27]. This instability has several causes. A source of instability is the correlations present in the sequence of observations. Additionally, the fact that small updates to Q_θ may significantly change the policy and therefore change the data distribution. Finally, the correlations between Q_θ and its target values can cause the learning to diverge. [28], [26] address these instabilities using a mechanism called experience replay that randomizes over the data, thereby removing correlations in the observation sequence and smoothing over changes in the data distribution.

In experience replay, the agent's experiences at each time step k are stored as tuples (s_k, a_k, r_k, s_{k+1}) into the replay memory $\mathcal{M}$. A minibatch of independent samples is then drawn from $\mathcal{M}$ to train the function approximator via stochastic gradient descent.

This approach demonstrates several improvements over standard Q-learning, as detailed in Algorithm 4, which incorporates the experience replay buffer mechanism. First, each step of experience is potentially used in many weight updates, thus allowing for greater data efficiency. Second, learning directly from consecutive samples is inefficient due to the strong correlations between the samples. Randomizing the samples breaks these correlations and reduces the update variance. Last, when learning on-policy the current parameters determine the next data sample that the parameters are trained on. For instance, if the minimizing action is to move left then the training samples will be dominated by samples from the left-hand side; if the minimizing action then changes to the right then the training distribution will also change. Unwanted feedback loops may therefore arise and the method could get stuck in a poor local minimum or even diverge.

Algorithm 4 MPC-based Q-learning method with Experience replay

Input: Model f, objective function, initial parameter θ_0, step size $\alpha \in (0, 1]$, discount factor $\gamma \in [0, 1]$, distribution of initial conditions X_0, empty replay buffer $\mathcal{D}$

Output:locally optimal policy $\pi_{\theta^\star}$

repeat
  for each j in K episodes **do**
    initialize $s_k : s_0 = s^0 \sim X_0$
    while not done **do**
      solve the MPC in (2.32) and get $y^\star$
      calculate and record the RL stage cost $L(s_k, a_k)$ and $s_{k+1} = f(s_k, a_k)$
      Store the transition $(s_k, a_k, s_{k+1}, L(s_k, a_k))$ in replay buffer $\mathcal{D}$
    end while
    for t in range(however many updates) **do**
      Randomly sample a batch of transitions, $\mathcal{M} = \{(s_k, a_k, s_{k+1}, L(s_k, a_k))\}$ from $\mathcal{D}$
      Calculate $V_\theta(s_{k+1})$, $Q_\theta(s_k, a_k)$ according to (2.32) and (2.33)
      Calculate the sensitivity $\nabla_\theta Q_\theta(s_k, a_k)$ according to (3.9)
      Set $\delta_k = L(s_k, a_k) + \gamma V_\theta(s_{k+1}) - Q_\theta(s_k, a_k)$
      Update the θ according to (2.31b)
    end for
  end for
until *convergence*

3 MPCQ-Learning For Active Subspace

In this chapter, we go through the main work of this master's thesis. We begin by providing an overview of the proposed approach for integrating MPC with active subspaces and RL. Next, we discuss the process of learning active subspaces within the context of MPC using RL. The final part of the chapter elaborates on the implementation of the MPCQ-Learning algorithm, including a detailed description and the specific methods developed and employed in this thesis.

3.1 Overview of Proposed Approach

Dimensionality reduction of decision variables is a practical and classic method to reduce the computational burden in linear and Nonlinear MPC.

In section 2.1.3, we observed that NMPC in active subspaces provides a general framework for dimensionality reduction in NMPC while guaranteeing recursive feasibility. The method relies on the active and inactive projector matrices T_1 and T_2 to solve the OCP in a lower dimensional subspace. However, so far, there is no underlying theory on how to choose these projectors to guarantee closed-loop optimality.

RL on the other hand is a powerful paradigm for learning-based control that provides effective solutions for optimal control in complex, nonlinear process systems. Section 2.2.3 demonstrates that using a generic parametrize MPC scheme as function approximator for Q-learning and under some mild assumption we are able to learn optimal policy.

Our proposed approach is to use RL specifically Q-learning to learn the optimal active subspace projector T_1 of the dimensionality reduced MPC (2.9) in active subspaces. We parameterize the stage cost, dynamics and the input constraints of the dimensionality reduced OCP and show that it can be use as function approximator for Q-learning to learn the state-action value function of the full problem.

Let us begin by introducing our concept of the optimal active subspace projector. To this end, consider the discounted finite-horizon OCP defined by:

$$V_N(x_k) = \min_{u_{k|k},\ldots,u_{k+N-1|k}} \sum_{i=0}^{N-1} \gamma^k \ell(x_{k+i|k}, u_{k+i|k}) + V_f(x_{k+N|k}), \tag{3.1a}$$

$$\text{s.t.} \quad x_{k+i+1|k} = f(x_{k+i|k}, u_{k+i|k}), \quad \forall i \in \mathbb{I}_{[0,N-1]}, \tag{3.1b}$$

$$x_k = s \tag{3.1c}$$

$$x_{k+i} \in \mathbb{X}, \tag{3.1d}$$

$$u_{k+i} \in \mathbb{U}. \tag{3.1e}$$

With the input vector $\boldsymbol{u}_k = \left[u_{k|k}^\top, \cdots, u_{k+N-1|k}^\top\right]^\top$. Suppose that $T_1^\star$ is the optimal active subspace projector , and $T_2^\star = ker(T_1^\star)$, with active subspace vector $\boldsymbol{v}_k$ and inactive subspace vector $\boldsymbol{w}_k$, such that

$$\begin{bmatrix} T_1^\star & T_2^\star \end{bmatrix} \begin{bmatrix} \boldsymbol{v}_k \\ \boldsymbol{w}_k \end{bmatrix} = \boldsymbol{u}_k$$

From (2.9), we defined the dimensionality reduced OCP $\mathscr{P}(x_k, \tilde{\boldsymbol{w}}_k)$:

$$V(x_k, \tilde{\boldsymbol{w}}_k) = \min_{\boldsymbol{v}_k, \mu_k} \mathcal{J}(x_k, \tilde{T}_1 \boldsymbol{v}_k + \mu_k \tilde{T}_2 \tilde{\boldsymbol{w}}_k) \tag{3.2}$$

$$\forall k \in \mathbb{I}_{[0,N-1]}, \quad x_k = s$$

$$\text{s.t.} \quad \mathcal{H}(x_k, \tilde{T}_1 \boldsymbol{v}_k + \mu_k \tilde{T}_2 \tilde{\boldsymbol{w}}_k) \leq \boldsymbol{0}$$

where $\tilde{T}_1$ and $\tilde{T}_2$ are some suboptimal active and inactive subspace projectors from some given generic feature vector $\mathcal{S}$. Theory says:

$$V_N(x_k, \tilde{\boldsymbol{w}}_k) = V_N(x_k) + \Delta V(x_k, \tilde{\boldsymbol{w}}_k) \tag{3.3}$$

Our approach relies on the fact that:

$$\Delta V(x_k) \to 0 \quad \text{as} \quad \tilde{T}_1 \to T_1^* \quad \text{and} \quad \tilde{T}_2 \to T_2^* \tag{3.4}$$

In order to achieve (3.4), we use Q-learning with the dimensionality reduced MPC scheme in (2.9) as function approximator to learn the state-action value function of the full problem in (3.1). By applying the Bellman equation defined in (2.24), we obtain $T_1^\star$ as shown:

$$\pi^{MPC} = T_1^\star \boldsymbol{v} + T_2^\star \boldsymbol{w} = \arg\min_{u \in \mathcal{A}} Q_N(x, u) \tag{3.5}$$

where π^{MPC} is the MPC policy of the full problem and $Q_N(x, u)$ is the state-action value function.

MPC in active subspaces as function approximator for RL

Here, we aim to extend the concept presented in Section 2.2.3 to MPC in active subspaces. For the sake of clarity and understanding, we will use multiple shooting for the OCP discussed here. Nevertheless, the concept can easily be transferred to the condensed OCP.

Let us consider for our given MPC scheme in (2.9) the following finite-horizon dimensionality reduced OCP in multiple shooting parameterized by θ:

$$V_\theta(s, \tilde{\boldsymbol{w}}) = \min_{\boldsymbol{v},\boldsymbol{x},\mu} \; V_f(x_N) + \sum_{k=0}^{N-1} \ell_\theta(x_k, p_k(T_1\boldsymbol{v} + \mu T_2\tilde{\boldsymbol{w}})) \tag{3.6a}$$

$$\text{s.t.} \quad x_{k+1} = f_\theta(x_k, p_k(T_1\boldsymbol{v} + \mu_k T_2\tilde{\boldsymbol{w}})) \quad \forall k = 0, \dots, N-1, \tag{3.6b}$$

$$x_0 = s \tag{3.6c}$$

$$\underline{U} \leq p_{k,\theta}(T_1 v + \mu T_2\tilde{\boldsymbol{w}}) \leq \overline{U} \tag{3.6d}$$

$$\underline{X} \leq x_k \leq \overline{X} \tag{3.6e}$$

$$x_N \in \mathbb{X}_f \tag{3.6f}$$

where $\boldsymbol{x} = \{x_0, \dots, x_N\}$, $\boldsymbol{v} = \{v_0, \dots, v_{n_v}\}$ and $\mu \in \mathbb{R}$ are the primal decision variables, N is the prediction horizon, T_1 the active subspace projector matrix, T_2 the inactive subspace projector matrix, $\tilde{\boldsymbol{w}}$ the inactive subspace vector, f_θ is the model dynamics, ℓ_θ and V_f are the stage and terminal costs, respectively. Constraint (3.6d) represents the input inequality constraints, and $p : \mathbb{R}^{Nn_u} \to \mathbb{R}^{Nn_u}$, with $p(T_1\boldsymbol{v} + \mu T_2\tilde{\boldsymbol{w}}) = \{p_0, \dots, p_{N-1}\}$, represents the set of predicted input trajectories.

By incorporating an additional initial input constraint into (3.6), the parameterized state-action value function of the MPC scheme is defined as:

$$Q_\theta(s, a, \tilde{\boldsymbol{w}}) = \min_{\boldsymbol{v},\boldsymbol{x},\mu} \; V_f(x_N) + \sum_{k=0}^{N-1} \ell_\theta(x_k, p_k(T_1 v + \mu T_2\tilde{\boldsymbol{w}})) \tag{3.7a}$$

$$\text{s.t.} \quad x_{k+1} = f_\theta(x_k, p_k(T_1 v + \mu T_2\tilde{\boldsymbol{w}})) \quad \forall k = 0, \dots, N-1, \tag{3.7b}$$

$$x_0 = s \tag{3.7c}$$

$$\boxed{p_0(T_1\boldsymbol{v} + \mu T_2\tilde{\boldsymbol{w}}) = a} \tag{3.7d}$$

$$\underline{U} \leq p_{k,\theta}(T_1\boldsymbol{v} + \mu T_2\tilde{\boldsymbol{w}}) \leq \overline{U} \tag{3.7e}$$

$$\underline{X} \leq x_k \leq \overline{X} \tag{3.7f}$$

$$x_N \in \mathbb{X}_f \tag{3.7g}$$

We propose to use the action-value function approximation $Q_\theta \approx Q_N$ obtained from (3.7). The parameter vector θ can be modified by the Q-learning algorithm using the TD update rule in (2.31) to shape the action-value function. Under same assumptions as in section 2.2.3 , the MPC scheme is able to capture the action-value function Q_N of the full problem even if the model f does not capture the real system dynamics.

The parameterized deterministic policy π_θ reads as follows:

$$\pi_\theta(s) = p_{0,\theta}(T_1 \boldsymbol{v}^\star(s) + \mu^\star T_2 \tilde{\boldsymbol{w}}) \tag{3.8}$$

Where $p_{0,\theta}(T_1 \boldsymbol{v}^\star(s) + \mu^\star T_1 \tilde{\boldsymbol{w}})$ is the first element of $\boldsymbol{p}^\star$ and $\boldsymbol{v}^\star, \mu^\star$ being the solution of the MPC scheme in (3.6).

Therefore, the value function $V_\theta(s)$ can be acquired together with the policy $\pi_\theta(s)$ by solving the classic MPC scheme in (3.6), while the action-value function results from solving the same MPC scheme with its first input constrained to a specific value a.

As defined by the TD update in (2.31b) we need to evaluate the gradient of the state-action value function Q_θ. To this end let us define the Lagrange function associated to the MPC scheme in (3.7) as follow:

$$\begin{aligned}\mathcal{L}_\theta(s, y) = V_f(x_N) + \chi_0^\top (x_0 - s) + \eta_N^\top h_f(x_N) + \sum_{k=0}^{N-1} \Big(\chi_{k+1}^\top (f_\theta(x_k, p_k(T_1 \boldsymbol{v} + \mu T_2 \tilde{\boldsymbol{w}})) - x_{k+1}) \\ + \eta_k^\top h_\theta(x_k, p_k(T_1 \boldsymbol{v} + \mu T_2 \tilde{\boldsymbol{w}})) + \ell_\theta(x_k, p_k(T_1 \boldsymbol{v} + \mu T_2 \tilde{\boldsymbol{w}})) \Big) + \zeta^\top (p_0(T_1 \boldsymbol{v} + \mu T_2 \tilde{\boldsymbol{w}}) - a)\end{aligned} \tag{3.9}$$

Where χ, η, and ζ are the multipliers associated with the equality constraints, state-input constraints, and initial condition of input constraints, respectively. We will denote $y = (x, v, \mu, \chi, \eta, \zeta)$ as the primal-dual variable associated with the MPC problem, and $\mathcal{E} = (x, v, \mu)$ as the primal decision variable of the MPC problem.

Note that for $\zeta = 0$, $\mathcal{L}_\theta(s, y)$ is the Lagrange function associated with the MPC for the value function $V_\theta(s, \tilde{\boldsymbol{w}})$.

Now to simplify the Lagrange expressions in (3.9), let consider the following:

$$\Omega_\theta = V_f(x_N) + \sum_{k=0}^{N-1} \ell_\theta(x_k, p_k(T_1 \boldsymbol{v} + \mu T_2 \tilde{\boldsymbol{w}}))$$

$$\boldsymbol{\chi}^\top \boldsymbol{G}_\theta = \sum_{k=0}^{N-1} \chi_{k+1}^\top (f_\theta(x_k, p_k(T_1 \boldsymbol{v} + \mu T_2 \tilde{\boldsymbol{w}})) - x_{k+1}) + \chi_0^\top (x_0 - s) + \zeta^\top (p_0(T_1 \boldsymbol{v} +, \mu T_2 \tilde{\boldsymbol{w}}) - a)$$

$$\boldsymbol{\eta}^\top \boldsymbol{H}_\theta = \sum_{k=0}^{N-1} \eta_k^\top h_\theta(x_k, p_k(T_1 \boldsymbol{v} + \mu T_2 \tilde{\boldsymbol{w}})) + \eta_N^\top h_f(x_N)$$

The Lagrange of the constrained dimensionality reduced OCP can then be written as:

$$\mathcal{L}_\theta(s, y) = \Omega_\theta + \boldsymbol{\chi}^\top \boldsymbol{G}_\theta + \boldsymbol{\eta}^\top \boldsymbol{H} \tag{3.10}$$

where Ω_θ is the cost, $\mathbf{G}_\theta$ gathers the equality constraints (3.7b), (3.7c),(3.7d), and $\mathbf{H}_\theta$ collects the inequalities constraints (3.7e), (3.7f), (3.7g).

Now we have:

$$\nabla_\theta Q_\theta(s, a, \tilde{\boldsymbol{w}}) = \nabla_\theta \mathcal{L}_\theta(s, y^*) \tag{3.11}$$

where y^* is the primal-dual solution of the MPC problem defined by $Q_\theta(s, a, \tilde{\boldsymbol{w}})$.

$$\nabla_\theta V_\theta(s, \tilde{w}_k) = \nabla_\theta \mathcal{L}_\theta(s, y^*) \tag{3.12}$$

where y^* is the primal-dual solution of the MPC problem defined by $V_\theta(s, \tilde{\boldsymbol{w}})$ with $\zeta^* = 0$.

Using the dimensionality-reduced MPC scheme in (2.9), with its stage cost, dynamics, and input constraints parameterized by θ, we have seen that its state-action value function $Q_\theta(s, a, \tilde{w})$, parameterized by θ, can be used as a function approximator in Q-learning to approximate the state-action value Q_N of the full problem in (3.1). In the next section, we will discuss how the dimensionality-reduced MPC function approximator can be used in Q-learning to learn the optimal active subspace projector.

3.2 Learning Active Subspace in MPC with RL

The core idea of our proposed approach is to use the parameterized dimensionality reduced MPC-scheme in (3.6) and (3.7) with parameter vector θ as value and state action value approximation function, and apply the TD update rule in (2.31a) to update the parameters so as to improve the closed-loop performance.

In the context of RL, we seek an optimal parametrized dimensionality reduced OCP policy $\tilde{\pi}_\theta^*$ that minimizes the closed-loop performance $\mathcal{J}(\tilde{\pi}_\theta)$ over a distrubution of initial conditions $\mathbf{X}_0$ with:

$$\mathcal{J}(\tilde{\pi}_\theta) = \mathbb{E}_{x_0 \sim \mathbf{X}_0} \left[\sum_{k=0}^{K} \gamma^k L_\theta(x_k, \tilde{\pi}_\theta(x_k)) \right] \tag{3.13a}$$

$$\tilde{\pi}_\theta^* = \arg\min_{\tilde{\pi}} \mathcal{J}(\tilde{\pi}_\theta) \tag{3.13b}$$

Here Overall, the adjustable parameter vector θ is the active subspace projector T_1:

$$\theta = \{T_1\} \tag{3.14}$$

The idea is to learn both T_1 and T_2 but since they are orthonormal projectors of some feature vector $\mathcal{S}$, and given that T_2 belongs to the null space of T_1, the learned parameter vector can then be reduced to just learning T_1.

Notice that the parameterized value function and state-action value function of the dimensionality-reduced MPC, as presented in (3.6) and (3.7) respectively, were introduced in a generic manner without specifying any particular choice of learnable parameters. Furthermore, the parameter-

ized reduced OCP scheme was expressed using a multiple shooting approach to simplify the explanation of the underlying concepts.

Since we are using the single shooting method as the primary approach for solving the OCP in this project, it is worthwhile to reformulate the parameterized reduced value, V_θ function and state-action value function, Q_θ (3.6) and (3.7) using single shooting, while incorporating our specific choice of parameterization vector, $\theta = \{T_1\}$.

To this end, the parameterized reduced value function V_θ and state-action value function Q_θ, with the parameterization vector $\theta = \{T_1\}$, are given by:

$$\mathscr{P}_\theta(x_k, \tilde{\boldsymbol{w}}_k) : V_\theta(x_k, \tilde{\boldsymbol{w}}_k) = \min_{\boldsymbol{v}_k, \mu_k} \; \mathcal{J}_\theta(x_k, \boldsymbol{v}_k, \mu_k, \tilde{\boldsymbol{w}}_k), \tag{3.15a}$$

$$\text{s.t.} \quad \mathcal{H}_\theta(x_k, \boldsymbol{v}_k, \mu_k, \tilde{\boldsymbol{w}}_k) \leq \mathbf{0}, \tag{3.15b}$$

$$x_k = s, \tag{3.15c}$$

and

$$Q_\theta(x_k, u_k, \tilde{\boldsymbol{w}}_k) = \min_{\boldsymbol{v}_k, \mu_k} \; \mathcal{J}_\theta(x_k, \boldsymbol{v}_k, \mu_k, \tilde{\boldsymbol{w}}_k), \tag{3.16a}$$

$$\text{s.t.} \quad \mathcal{H}_\theta(x_k, \boldsymbol{v}_k, \mu_k, \tilde{\boldsymbol{w}}_k) \leq \mathbf{0}, \tag{3.16b}$$

$$x_k = s, \tag{3.16c}$$

$$u_k = a \tag{3.16d}$$

Where $\boldsymbol{v}_k$ and μ_k are the primal decision variables, $\mathcal{J}_\theta$ the cost performance index and in $\mathcal{H}_\theta$ we collect all input and state constraints.

For a rich parametrization, we can characterize the optimal parameters as those that minimize the following least-squares problem:

$$\theta^\star = \arg\min_{\theta \in \Theta} \mathbb{E}_{s_0 \sim X_0} \left[\frac{1}{K} \sum_{k=0}^{K-1} \left(Q_\theta(x_k, u_k, \tilde{\boldsymbol{w}}_k) - Q_N(x_k, u_k) \right)^2 \right] \tag{3.17}$$

As the state action-value function of the full problem Q_N is in general unknown in our case, (3.17) cannot be solved directly. Using first order Q-learning method, we can try to achieved (3.17) by updating the learnable parameters using TD error as define in (2.31a).

The Q-learning update rules from equations (2.31a) and (2.31b) can then be rewritten for our MPC in active subspace as follows:

$$\delta = L(x, u) + \gamma V_\theta(x, \tilde{w}) - Q_\theta(x, \tilde{w}, u) \tag{3.18a}$$

$$\theta \leftarrow \theta + \alpha \delta \nabla_\theta Q_\theta(x, \tilde{w}, u) \tag{3.18b}$$

RL update rule for stable learning:

In order to prevent our MPC agent from receiving parameter updates that could cause abnormal behavior or constraint violations, one needs to adjust the general update rule to allow for a more stable update. The classical RL update steps read as follows:

$$\theta \leftarrow \theta - \alpha dJ \tag{3.19}$$

which can also be rewritten as:

$$\theta \leftarrow \theta + \Delta\theta \tag{3.20a}$$

$$\Delta\theta = \arg\min_{\Delta\theta} \left(\frac{1}{2} \|\Delta\theta\|^2 + \alpha dJ^\top \Delta\theta \right) \tag{3.20b}$$

$$dJ = \mathbb{E}\left[\delta_k \nabla_\theta Q_\theta(s_k, a_k, \tilde{w}_k) \right] \tag{3.20c}$$

From equations (3.20), in order to guarantee a stable RL update rule , the following additional requirements have to be met:

- **Constraints on the change in the learnable parameter**: The change in the learnable parameter $\Delta\theta$ should be in some compact set $\mathcal{I}$. The set is chosen in a way as to prevent large changes in θ during RL update which might result in abnormal behavior of the agent, loss in recursive feasibility or constraints violation of the MPC scheme.
- **Preservation of the orthonormality of the learnable parameter**: The orthonormal property of the learnable parameter θ must not be lost after each RL update step, as this could lead to abnormal behavior of the agent, loss recursive feasibility and in the search for the optimal active subspace projector.

Stiefel Manifold Optimization and Gram-Schmidt Orthogonalization

Constraints on changes in the learnable parameter can easily be handle by adding a hard constraint on the convex optimization problem defined in (3.20b). From this point onward, we will explore two approaches for handling the orthonormality of the learnable parameter so as guarantee stability and recursive feasibility during RL update steps.

1. Stiefel Manifold Optimization

   Suppose we wish to find p orthonormal vectors in $\mathbb{R}^n$ that are optimal with respect to an objective function F. We pose this problem as follows: Let X be any $n \times p$ matrix satisfying $X^\top X = I$. We take the columns of X to be p orthonormal vectors in $\mathbb{R}^n$, and we assume that F is a function from the space of $n \times p$ matrices to the real line [29]. We form the optimization problem:

   $$\min_{X \in \mathbb{R}^{n \times p}} F(X) \quad \text{s.t.} \quad X^\top X = I. \tag{3.21}$$

   - The constraint set $X^\top X = I$ is a submanifold of $\mathbb{R}^{n \times p}$ called the Stiefel manifold.

- The algorithm works by finding the gradient of F in the tangent plane of the manifold at the point $X^{[k]}$ of the current iterate. A curve is found on the manifold that proceeds along the projected negative gradient, and a curvilinear search is made along the curve for the next iterate $X^{[k+1]}$.
- The search curve is not a geodesic, but is instead constructed using a Cayley transformation (see [30] for technical detail). This has the advantage that matrix exponentials are not required. The algorithm only requires inversion of a $2p \times 2p$ matrix. The algorithm is especially suitable for use in high-dimensional spaces when $p \ll n$.

From the optimization problem in (3.20), we proposed the following reformulation of the RL update rule on the Stiefel manifold as follow:

$$\theta^* = \arg\min_{\theta} \left(0.5\|\theta - \theta_j\|_F^2 + \alpha \ \mathrm{tr}\left(dJ^\top(\theta - \theta_j)\right)\right) \tag{3.22a}$$

$$\text{s.t.} \quad \theta^\top \theta = I \tag{3.22b}$$

$$dJ = \mathbb{E}\left[\delta_k \nabla_\theta Q_\theta(s_k, a_k)\right] \tag{3.22c}$$

Where $\theta \in \mathbb{R}^{Nm \times n_v}$, $\|.\|_F$ denote the Frobinius norm and tr(.) the euclidean inner product of the vector space defined by $n \times p$ matrices.

The reformulation of the RL update rule in (3.22) is incomplete, as it lacks constraints on the change in the learnable parameter. In the case of Stiefel manifold optimization, these constraints cannot be added as hard constraints because optimization on the Stiefel manifold only allows orthonormality constraints to be passed as hard constraints to the optimization problem.

A standard approach to handle constraints on Stiefel Manifold is the so-called exact penalty method. As a replacement for the constraints, the method supplements the cost function with a L_1 penalty for violating the constraints [31]. This leads to a non-smooth, unconstrained optimization problem. Using this approach (3.22a) modify to:

$$\theta^* = \arg\min_{\theta} \left(0.5\|\theta - \theta_j\|_F^2 + \alpha \,\mathrm{tr}(dJ^\top(\theta - \theta_j)) + \rho \sum_{i=1}^{2} \max\{0, g_i(\theta)\}\right) \tag{3.23}$$

Where $g_i : \mathbb{R}^{Nm \times n_v} \to \mathbb{R}^{Nm \times n_v}$ collect the inequalities constraints of the change in the learnable parameter and ρ denote the penalty weight needed for the exact satisfaction of these constraints.

Notice that the resulting penalized cost function in (3.23) is non-smooth: such problems may be challenging in general. In-order to smooth the cost function we proposed to use log-sum-exp function [32]: $\max\{a, b\} \approx u \log(e^{a/u} + e^{b/u})$, with smoothing parameter

$u > 0$. This yields the following smooth, unconstrained problem on a manifold, where ρ and u are fixed constants:

$$\theta^* = \arg\min_{\theta} \left(0.5\|\theta - \theta_j\|_F^2 + \alpha \operatorname{tr}(dJ^\top(\theta - \theta_j)) + \rho \sum_{i \in I} u \log(1 + e^{g_i(x)/u}) \right) \tag{3.24}$$

Optimizing on the Stiefel manifold brings us many benefits, such as naturally handling the orthonormality constraints, improved numerical stability, and efficient use of the parameter space. But the downside of this is that optimization on the Stiefel Manifold is generally non-convex and there is no guarantee of finding a global minimizer for the problem.

2. Gram-Schmidt Orthogonalization

   Gram-Schmidt process is a method of constructing an orthonormal basis from a set of vectors in an inner product space, most commonly the Euclidean space $\mathbb{R}^n$ equipped with the standard inner product. The Gram-Schmidt process takes a finite, linearly independent set of vectors $v = \{\mathbf{v}_1, \ldots, \mathbf{v}_k\}$ for $k \leq n$, and generates an orthogonal set $w = \{\mathbf{w}_1, \ldots, \mathbf{w}_k\}$ that spans the same k-dimensional subspace of $\mathbb{R}^n$ as v.

   Using the concept of Gram-Schmidt, we propose a new approach to preserve the orthonormality of the learnable parameters after each RL update. By doing so, we can perform the optimization on the parameter updates in Euclidean space rather than on a matrix manifold.

   Our approach involved the following steps:

   - Vectorization of the learnable parameter θ as in (3.14)
   - We solve the following optimization problem for the RL update rule:

$$\Delta\theta^* = \arg\min_{\Delta\theta} \left(0.5\|\Delta\theta\|^2 + \alpha\, dJ^\top \Delta\theta \right) \tag{3.25a}$$

$$\text{subject to} \quad -0.2 \leq \Delta\theta \leq 0.2 \tag{3.25b}$$

$$\theta = \theta + \Delta\theta \tag{3.25c}$$

   - We perform Gram-Schmidt process on the updated θ to preserved the orthonormality constraint.

   By performing the optimization in Euclidean space, we benefit from a convex program (3.25), and we can also add constraints on changes in the learnable parameters as hard constraints. However, a drawback of re-orthonormalizing after every RL update is that the structure of the problem may not be preserved; descent directions can be altered, which may disrupt the natural flow of gradient descent and potentially slow convergence. This could require more iterations to reach an optimal point.

3.3 Algorithm Design

The final objective of our work is to design a RL algorithm that uses the NMPC in active subspaces scheme, as presented in Algorithm 1, to learn the optimal active subspace projector T_1, which can produce the best closed-loop performance for the NMPC active controller in the given lower-dimensional input space. To this end we propose an MPC-based Q-learning (MPCQ-learning) algorithm which is an extension of Algorithm 3.

MPCQ-learning uses NMPC in active subspace as a function approximator for Q-learning method and also incorporates safe RL update by reformulating the Q-learning TD update rule in (3.18) as an optimization on the Stiefel manifold (3.24) to ensure that the orthonormality of the learn active subspace projector matrix T_1 is not lost during RL update. This ensures stable update and guarantee recursive feasibility of the NMPC agent. Furthermore, MPCQ-learning employs the dimensionality reduction technique using PCA, as presented in Section 2.1.3, to determine the optimal active subspace dimension for any choice of covariance matrix C.

The following sections we present schematic of the proposed setup for implementing the MPCQ-learning algorithm. Next we detail in pseudocode the PCA algorithm to determine the optimal active subspace dimension for any choice of covariance matrix. Finally, the detailed structure of the MPCQ-learning algorithm is presented in pseudocode.

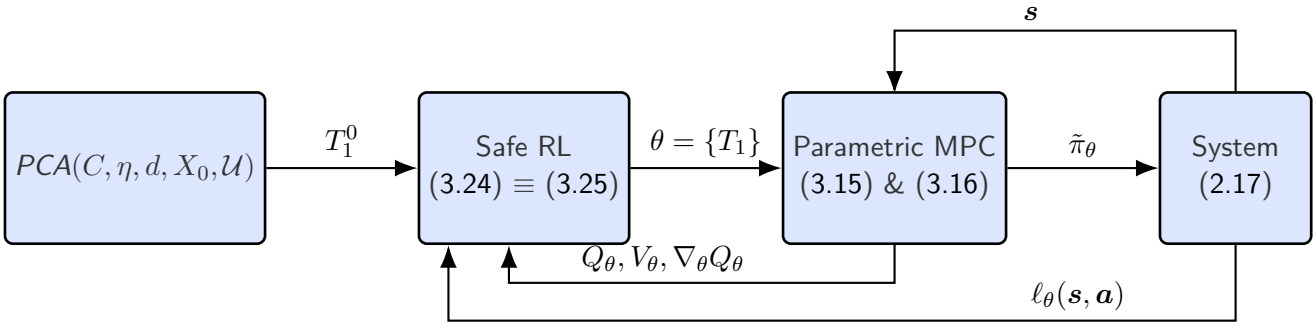

Figure 3.1: MPCQ-learning framework

For a given choice of covariance matrix C (e.g., candidate matrices include the Hessian matrix H, block identity matrix I, etc.), Algorithm 5 computes the suboptimal active subspace dimension and initializes the active subspace projector T_1^0. The RL method then updates the projector T_1^0 by evaluating the cost in equation (3.24). This cost depends on Q_θ, V_θ obtained from the parametrize dimensionality reduced MPC and l_θ. The dimensionality reduced MPC controls the system.

In Algorithm 5, it should be noted that the parameters d, the number of dominant eigenvalues of the covariance matrix C, and the threshold η, which is needed for determining the direction of the control input vector u, must be chosen carefully to ensure that PCA captures the best suboptimal active subspace dimension. It is advisable to select d as high as possible, at least more than half of the total number of eigenvalues of C. The threshold η in PCA always

Algorithm 5 Computation of active subspace dimension n_v with $PCA(C, \eta, d, X_0, \mathcal{U})$

Input: Variance threshold η, covariance matrix $C \in \mathbb{R}^{Nn_u \times Nn_u}$, distrubution of initial conditions X_0, $\mathcal{U} = \emptyset$

Output: Active subspace matrix T_1^0 and suboptimal active subspace dimension $\tilde{n}_v$

1: Eigenvalue decomposition on C to obtain eigenvalues σ_i and eigenvectors v_i: $C = T\Sigma T^\top$
2: Select the top d dominant eigenvectors T_1 corresponding to the largest d eigenvalues.
3: **for** each j in N_{X_0} **do**
4: get $x^j \in X_0$
5: solve active-subspace NMPC with $\mathscr{P}(x^j, \widetilde{\mathbf{w}}^j)$ and get open loop control trajectories $\boldsymbol{u}_{ol}^\star(x^j)$
6: Append the optimal trajectories $\mathcal{U} \leftarrow \mathcal{U} \cup \{\boldsymbol{u}_{ol}^\star(x^j)\}$
7: **end for**
8: Construct the data matrix X with each row corresponding to a vector u_i from $\mathcal{U}$.
9: Compute the mean vector μ for X: $\mu = \frac{1}{m}\sum_{i=1}^{m} u_i$
10: Center the data matrix: $X_{\text{centered}} = X - \mu$
11: Calculate $\mathcal{S}$: $\mathcal{S} = \frac{1}{K-1} X_{\text{centered}}^\top X_{\text{centered}}$
12: Eigenvalue decomposition on $\mathcal{S}$ to obtain eigenvalues λ_i and eigenvectors w_i: $\mathcal{S} = V\Lambda V^\top$
13: Compute cumulative variance contribution $var(\lambda_i) = \frac{\lambda_i}{\sum_j \lambda_j}$ for each λ_i.
14: Set $\tilde{n}_v$ as the number of eigenvalues meeting or exceeding η
15: Select the top $\tilde{n}_v$ eigenvectors T_1^0 corresponding to the largest $\tilde{n}_v$ eigenvalues.
16: Form T_1^0 using the top $\tilde{n}_v$ eigenvectors: $T_1^0 = [w_1, w_2, \ldots, w_{\tilde{n}_v}]$

represents a trade-off between the size of the reduced dimension and the amount of variance captured.

MPCQ-learning method:

Algorithm 6 shows MPCQ-learning method in pseudocode which use Q-learning to approximate the dimensionality reduce state-action value function with the aim of implicitly obtaining the dimensionally reduced suboptimal policy. Given $J_c(x^j)$ the reference MPC closed-loop performance starting from an initial condition x^j and $\tilde{J}_c(x^j)$ the closed-loop performance of dimensionality reduce MPC scheme. We denote by δ, the relative closed-loop performance difference over a distribution of the initial condition X_0.

$$\delta = \frac{1}{N_{X_0}} \sum_{j=0}^{N_{X_0}} \left(\left(J_c(x^j) - \tilde{J}_c(x^j) \right) J_c(x^j)^{-1} \right) \tag{3.26}$$

The main concept of the MPCQ-learning Method is that $\forall\ \varepsilon > 0$, we can always find a suboptimal closed-loop policy $\tilde{\pi}$ such that the absolute value of the relative closed-loop performance difference $|\delta| \leq \varepsilon$. In Algorithm 7, we summarize in pseudocode the MPCQ-learning method with experience replay.

Algorithm 6 MPCQ-Learnings Method

Input: Objective function J, system model f, $k = 1$, X_0, γ, α, ε, η, reference closed loop performance Δ_{ref}, initial closed loop performance Δ_0, C, d, $\mathcal{U} = \emptyset$, $\mathcal{U}_{cl}^{\star} = \emptyset$, $u_{cl}^{\star} = \emptyset$

Output:locally optimal policy $\pi_{\theta^{\star}}$

Solve $T_1^0, \tilde{n}_v = PCA(C, \eta, d, X_0, \mathcal{U})$
while $|(\Delta_{ref} - \Delta_0) \cdot \Delta_{ref}^{-1}| > \varepsilon$ **do**
  Initialise RL parameters $\boldsymbol{\theta}_0 = \{T_1^0\}$ and compute $T_2^0 \in \text{Null}(T_1^0)$
  for each j in K episodes **do**
    Pick $x_0 \in X_0$ and solve $\mathscr{P}(x_0)$ for $\tilde{\boldsymbol{u}}_0$, set $\tilde{\boldsymbol{w}}_0 = (T_2^j)^{\top} \tilde{\boldsymbol{u}}_0$
    while not done **do**
      Estimate x_k, solve $\mathscr{P}_{\theta_j}(x_k, \tilde{\boldsymbol{w}}_k)$ for $y^* = (\boldsymbol{v}_k^*, \mu_k^*)$
      Compute the optimal control trajectory: $\boldsymbol{u}_k^{\star} = T_1^j \boldsymbol{v}_k^* + \mu_k^* T_2^j \tilde{\boldsymbol{w}}_k$
      Append the first control input of the optimal trajectory $u_{cl}^{\star} \leftarrow u_{cl}^{\star} \cup \{\boldsymbol{u}_0^{\star}\}$
      Calculate and record $L(s_k, a_k)$,$V_\theta(s_{k+1})$, $Q_\theta(s_k, a_k)$, $s_{k+1} = f(s_k, a_k)$
      Calculate the sensitivity $\nabla_\theta Q_\theta(s_k, a_k)$
      Set $\tau_k = L(s_k, a_k) + \gamma V_\theta(s_{k+1}) - Q_\theta(s_k, a_k)$
      Update $\tilde{\boldsymbol{u}}_k$ by: $\tilde{\boldsymbol{u}}_k = [u_{k|k-1}^{\top}, \ldots, u_{k+N-2|k-1}^{\top}, u_{k+N-1|k-1}^{\top}]^{\top}$
      Update $\tilde{\boldsymbol{w}}_k$ by: $\tilde{\boldsymbol{w}}_k = (T_2^j)^{\top} \tilde{\boldsymbol{u}}_k$
    end while
    Append the optimal trajectories $\mathcal{U}_{cl}^{\star} \leftarrow \mathcal{U}_{cl}^{\star} \cup \{\boldsymbol{u}_{cl}^{\star}\}$
    Update θ according to the RL update rule in (3.24)
    Update T_2^j such that $T_2^j \in \text{Null}(T_1^j)$
    Update $u_{cl}^{\star} = \emptyset$
  end for
  Update Δ_0 from a distribution of initial conditions X_0:

$$\Delta_0 = \frac{1}{N_{X_0}} \sum_{t=0}^{N_{X_0}} J(x^{(t)}, T_1^j v_k^* + \mu_k^* T_2^j \tilde{w}_k)$$

  Update initial active subspace projector, $T_1^0 = T_1^j$
  Set $\mathcal{U}_{cl}^{\star} = \emptyset$
end while

Algorithm 7 MPCQ-Learnings Method with Experience replay

Input: Objective function J, system model f, $k = 1$, X_0, γ, α, ε, η, reference closed loop performance Δ_{ref}, initial closed loop performance Δ_0, C, d, $\mathcal{U} = \emptyset$, $\mathcal{U}_{cl}^{\star} = \emptyset$, $\boldsymbol{u}_{cl}^{\star} = \emptyset$, empty replay buffer $\mathcal{D}$

Output:locally optimal policy $\pi_{\theta^{\star}}$

Solve $T_1^0, \tilde{n}_v = PCA(C, \eta, d, X_0, \mathcal{U})$
while $|(\Delta_{ref} - \Delta_0) \cdot \Delta_{ref}^{-1}| > \varepsilon$ **do**
  Initialise RL parameters $\boldsymbol{\theta}_0 = \{T_1^0\}$ and compute $T_2^0 \in \text{Null}(T_1^0)$
  for each j in K episodes **do**
    Pick $x_0 \in X_0$ and solve $\mathscr{P}(x_0)$ for $\tilde{\boldsymbol{u}}_0$, set $\tilde{\boldsymbol{w}}_0 = (T_2^j)^{\top} \tilde{\boldsymbol{u}}_0$
    while not done **do**
      Estimate x_k, solve $\mathscr{P}_{\theta_j}(x_k, \tilde{\boldsymbol{w}}_k)$ for $y^* = (\boldsymbol{v}_k^*, \mu_k^*)$
      Compute the optimal control trajectory: $\boldsymbol{u}_k^{\star} = T_1^j \boldsymbol{v}_k^* + \mu_k^* T_2^j \tilde{\boldsymbol{w}}_k$
      Append the optimal trajectory $\boldsymbol{u}_{cl}^{\star} \leftarrow \boldsymbol{u}_{cl}^{\star} \cup \{u_0^{\star}\}$
      Calculate and record s_{k+1}, $L(s_k, a_k)$
      Store the transition $(s_k, a_k, s_{k+1}, L(s_k, a_k))$ in replay buffer $\mathcal{D}$
      Update $\tilde{\boldsymbol{u}}_k$ by: $\tilde{\boldsymbol{u}}_k = [u_{k|k-1}^{\top}, \ldots, u_{k+N-2|k-1}^{\top}, u_{k+N-1|k-1}^{\top}]^{\top}$
      Update $\tilde{\boldsymbol{w}}_k$ by: $\tilde{\boldsymbol{w}}_k = (T_2^j)^{\top} \tilde{\boldsymbol{u}}_k$
    end while
    Append the optimal trajectories $\mathcal{U}_{cl}^{\star} \leftarrow \mathcal{U}_{cl}^{\star} \cup \{\boldsymbol{u}_{cl}^{\star}\}$
    Update $u_{cl}^{\star} = \emptyset$
    for t in range(however many updates) **do**
      Randomly sample a batch of transitions, $\mathcal{B} = \{(s_k, a_k, s_{k+1}, L(s_k, a_k))\}$ from $\mathcal{D}$
      Calculate $V_{\theta}(s_k)$, $Q_{\theta}(s_k, a_k)$
      Calculate the sensitivity $\nabla_{\theta} Q_{\theta}(s_k, a_k)$
      Set $\tau_k = L(s_k, a_k) + \gamma V_{\theta}(s_{k+1}) - Q_{\theta}(s_k, a_k)$
      Update θ according to the RL update rule in (3.24)
      Update T_2^j such that $T_2^j \in \text{Null}(T_1^j)$
    end for
  end for
  Update Δ_0 from a distribution of initial conditions X_0:

$$\Delta_0 = \frac{1}{N_{X_0}} \sum_{t=0}^{N_{X_0}} J(x^{(t)}, T_1^j v_k^* + \mu_k^* T_2^j \tilde{w}_k)$$

  Update initial active subspace projector, $T_1^0 = T_1^j$
  Set $\mathcal{U}_{cl}^{\star} = \emptyset$
end while

4 Simulation and Results

This chapter provides an in-depth discussion of the implementation of the MPCQ-learning algorithm. It begins with an introduction to the simulation environment, detailing the software tools and frameworks used for optimization and the definition of the MPC scheme, CasADi. The chapter then moves on to describe the test scenarios used to evaluate the performance of the MPCQ-learning implementation. Finally, it presents the evaluation metrics and discusses the results obtained from the simulations, highlighting the implementation details, workflow, and results of simulation.

4.1 Simulation

In this section, we consider a simulation example motivated by the well-known cart-pole system, illustrated in Figure 4.1. The system consists of a pole of length l and mass m_p attached to an under-actuated joint on a cart of mass m_c that moves along a frictionless track. The mass of the pole is assumed to be concentrated at its end. This benchmark is widely used in both classical control design and machine learning, particularly in RL. The control tasks involves both swing-up and balancing. The swing-up control assumes that the pole starts from the bottom (with angle $\theta = \pi$) and the aim is to push the cart back and forth so that the pole will reach the desired upright position (with $\theta = 0$) as quickly as possible, subject to constraints on the available control force. The balancing control assumes that the pole starts around $\theta = 0$ and aims to maintain the pole upright while bringing the cart position back to its origin.

The Figure 4.1 shows our parametrization of the system. x is the horizontal position of the cart, θ is the clockwise angle of the pendulum. We collect $[\theta, x, \dot{\theta}, \dot{x}]^\top$ in the state vector $\boldsymbol{x}$. The task is to stabilize the unstable fixed point at $\boldsymbol{x} = [0, 0, 0, 0]^\top$.

4.1.1 Model Dynamics

The dynamics of the cart pole system are derived using Lagrangian Mechanics. The Lagrangian of the cart pole system uses the general energy equation $\mathcal{L} = \mathcal{T} - \mathcal{V}$ where $\mathcal{L}$ is the Lagrangian term generalizing the total energy in the system, $\mathcal{T}$ is the kinetic energy of the system, and $\mathcal{V}$ is the potential energy of the system.

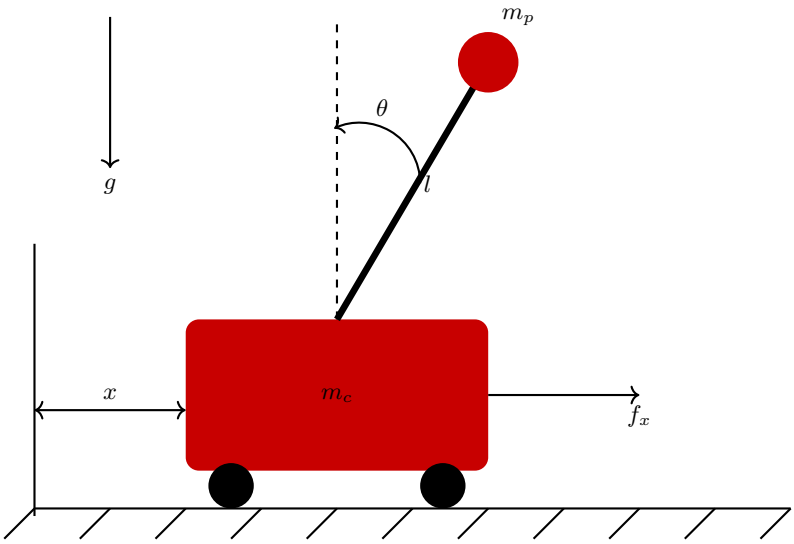

Figure 4.1: Cart-pole system dynamics representation

The energy is given by

$$\begin{aligned} \mathcal{T} &= \frac{1}{2}(m_c + m_p)\dot{x}^2 + m_p\dot{x}\dot{\theta}l\cos\theta + \frac{1}{2}m_p l^2\dot{\theta}^2, \\ \mathcal{V} &= m_p g l\cos\theta. \end{aligned} \tag{4.1}$$

So the Lagrangian is:

$$\mathcal{L} = \frac{1}{2}(m_c + m_p)\dot{x}^2 + m_p\dot{x}\dot{\theta}l\cos\theta + \frac{1}{2}m_p l^2\dot{\theta}^2 - m_p g l\cos\theta \tag{4.2}$$

Next, for each generalized coordinate, $\boldsymbol{q} = [q_1, q_2]^\top = [x, \theta]^\top$, and its associated actuating force F_j (if any), we can derive an equation of motion:

$$F_j = \frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{q}_j}\right) - \frac{\partial \mathcal{L}}{\partial q_j}, \quad j = 1, 2 \tag{4.3}$$

By further simplying the Lagrangian yields the following equations of motion:

$$\begin{aligned} (m_c + m_p)\ddot{x} + m_p l\ddot{\theta}\cos\theta - m_p l\dot{\theta}^2\sin\theta &= f_x, \\ m_p l\ddot{x}\cos\theta + m_p l^2\ddot{\theta} - m_p g l\sin\theta &= 0. \end{aligned} \tag{4.4}$$

In standard, manipulator equation form:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} = \boldsymbol{\tau}_g(\mathbf{q}) + \mathbf{B}\mathbf{u},$$

Using $\boldsymbol{q} = \Big[x, \theta\Big]^\top$, $\boldsymbol{u} = f_x$, we have:

$$\boldsymbol{M}(\boldsymbol{q}) = \begin{bmatrix} m_c + m_p & m_p l\cos\theta \\ m_p l\cos\theta & m_p l^2 \end{bmatrix}, \quad \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}}) = \begin{bmatrix} 0 & -m_p l\dot{\theta}\sin\theta \\ 0 & 0 \end{bmatrix},$$

$$\boldsymbol{\tau_g}(\boldsymbol{q}) = \begin{bmatrix} 0 \\ m_p g l\sin\theta \end{bmatrix}, \quad \boldsymbol{B} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

In this case, it is particularly easy to solve directly for the accelerations:

$$\ddot{x} = \frac{1}{m_c + m_p \sin^2\theta}\Big[f_x - m_p \sin\theta\left(l\dot{\theta}^2 - g\cos\theta\right)\Big] \tag{16}$$

$$\ddot{\theta} = \frac{1}{l\left(m_c + m_p\sin^2\theta\right)}\Big[f_x\cos\theta - m_p l\dot{\theta}^2\cos\theta\sin\theta + (m_c + m_p)g\sin\theta\Big] \tag{17}$$

The nonlinear dynamics can then be represented as:

$$\begin{bmatrix} \dot{\theta} \\ \dot{x} \\ \ddot{\theta} \\ \ddot{x} \end{bmatrix} = \begin{bmatrix} \dot{\theta} \\ \dot{x} \\ \frac{-m_p l\cos(\theta)\sin(\theta)\dot{\theta}^2 + f_x\cos(\theta) + m_p g\sin(\theta) + m_c g\sin(\theta)}{l\cdot(m_c + m_p\sin^2\theta)} \\ \frac{-m_p l\sin(\theta)\dot{\theta}^2 + f_x + m_p g\cos(\theta)\sin(\theta)}{m_c + m_p\sin^2\theta} \end{bmatrix}$$

The simplify state space form of the nonlinear dynamics can be written as:

$$\begin{aligned} \dot{\boldsymbol{x}} &= f_c(\boldsymbol{x}, \boldsymbol{u}), \\ \text{where} \quad \boldsymbol{x} &= [x_1, x_2, x_3, x_4]^\top = [\theta, x, \dot{\theta}, \dot{x}]^\top \end{aligned} \tag{4.5}$$

and $m_c = 1kg$, $m_p = 0.1kg$, $l = 0.8m$, and $g = 9.8m/s^2$. Equation (4.5) represents the continuous-time nonlinear dynamics of the cart-pole system, however, we are interested in its discrete-time counterpart. Consider the following discrete-time nonlinear system:

$$x(k+1) = f(x(k), u(k))$$

To find $f(x(k), u(k))$, we use Euler's method. Let $x_k = x(k\Delta T)$, $u_k = u(k\Delta T)$, where ΔT is the sampling rate.

$$\frac{x((k+1)\Delta T) - x(k\Delta T)}{\Delta T} \approx f_c(x(k\Delta T), u(k\Delta T))$$

$$\frac{x_{k+1} - x_k}{\Delta T} \approx f_c(x_k, u_k)$$

So

$$x_{k+1} \approx x_k + f_c(x_k, u_k)\Delta T = f(x_k, u_k) \tag{4.6}$$

Linearization of the Cart-Pole Dynamics

We also need to linearize the nonlinear dynamics of the continuous-time Cart-Pole system at the unstable equilibrium as part of the requirement for designing the MPC-Q learning controller. To this end, the system can be approximated using a Taylor series and takes the form:

$$\dot{x} = Ax + Bu \tag{4.7}$$

where

$$A = \frac{\partial f}{\partial x}(0,0) = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ \frac{(m_p+m_c)g}{lm_c} & 0 & 0 & 0 \\ -\frac{m_p g}{m_c} & 0 & 0 & 0 \end{bmatrix}$$

$$B = \frac{\partial f}{\partial u}(0,0) = \begin{bmatrix} 0 \\ 0 \\ -\frac{1}{lm_c} \\ \frac{1}{m_c} \end{bmatrix}$$

After discretization and linearization, we obtain :

$$x_{k+1} = x_k + (Ax_k + Bu_k)\Delta T \tag{4.8}$$

Where A and B represent the state and the input matrices respectively.

4.1.2 Design of the NMPC controller

We start by formulating the OCP, with the main objective of stabilizing the unstable equilibrium point of the cart-pole pendulum, starting from a set of random initial positions. So, generally the OCP for the dynamic system can be stated as follows: Find the set of admissible control histories $u = \{u_0, u_1, \ldots, u_{N-1}\} \in \mathbb{U}^N$ generating the corresponding state trajectory $x = \{x_0, x_1, \ldots, x_N\} \in \mathbb{X}^{N+1}$ that minimizes the cost function $\mathcal{J}$ while adhering to some design constraints.

It should be noted that the design of $\mathcal{J}$ is problem-specific. For our problem, we aim to stabilize the cart-pole pendulum at its unstable equilibrium points, starting from a random initial position, with minimal control effort, and subject to constraints on the travel distance of the cart. Therefore, we use a quadratic cost (L_2 cost) function as our design choice, which penalizes in a quadratic manner the deviation of the cart-pole from the desired equilibrium position (the origin $x_s = [0, 0, 0, 0]^T$) that we aim to stabilize.

As usual in NMPC, at time instant $k \in \mathbb{N}$, our OCP read as follow:

$$\min_{\boldsymbol{u}_k, \boldsymbol{x}_k} \quad x_{k+N|k}^\top P x_{k+N|k} + \sum_{i=0}^{N-1} \left(x_{k+i|k}^\top Q x_{k+i|k} + u_{k+i|k}^\top R u_{k+i|k} \right), \tag{4.9a}$$

$$\forall i \in \mathbb{I}_{[0,N-1]}, \tag{4.9b}$$

$$\text{s.t.} \quad x_{k+i+1|k} = f(x_{k+i|k}, u_{k+i|k}), \quad x_{k|k} = x_k \sim X_0 \in \mathbb{X} \subseteq \mathbb{R}^{n_x} \tag{4.9c}$$

$$-10 \leq x_{k+i|k}^{(2)} \leq 10, \quad -80 \leq u_{k+i|k} \leq 80. \tag{4.9d}$$

where Q, P, R are some positive semi definite weight matrices, $x_{k+i|k}$ is the predicted state at step $k+i$ of the system while starting at state x_k and the index j in $x_{k+i|k}^{(j)}$ represent the jth entry of the state vector. X_0 is a distribution of initial conditions drawn from a random uniform distribution as shown below:

$$X_0 = \left\{ (\theta, x, \dot{\theta}, \dot{x}) \,\middle|\, \theta \sim \mathcal{U}(0, \pi),\ x \sim \mathcal{U}(0, 3),\ \dot{\theta} = 0,\ \dot{x} = 0 \right\} \tag{4.10}$$

Due to the difficulties in tuning the weight matrices, the parameters of the weight matrices were chosen based on a sample example from the provided GitHub repository ([33]), which had a similar problem structure. The weight matrices where given as :

$$Q = P = \text{diag}(1, 1, 0.1, 0.1), \quad R = \text{diag}(0.001)$$

Moreover, due to the absence of a terminal set constraint in our problem, we use a horizon length of $N = 100$ to guarantee recursive feasibility.

By using the concept in (2.3), the condense full problem of the NMPC scheme read:

$$\mathscr{P}(x_k): \quad \min_{\boldsymbol{u}_k} \mathcal{J}(x_k, \boldsymbol{u}_k) \tag{4.11a}$$

$$\mathcal{J}(x_k, \boldsymbol{u}_k) = \boldsymbol{g}(x_k, \boldsymbol{u}_k)^\top \boldsymbol{Q} \boldsymbol{g}(x_k, \boldsymbol{u}_k) + \boldsymbol{u}_k^\top \boldsymbol{R} \boldsymbol{u}_k \tag{4.11b}$$

$$\boldsymbol{Q} = \text{diag}\{Q, \ldots, Q, P\}, \quad \boldsymbol{R} = \text{diag}\{R, \ldots, R\} \tag{4.11c}$$

$$\boldsymbol{u}_k = \begin{bmatrix} u_{k|k}^\top & \cdots & u_{k+N-1|k}^\top \end{bmatrix}^\top \tag{4.11d}$$

$$\boldsymbol{g}(x_k, \boldsymbol{u}_k) = \begin{pmatrix} x_k \\ f(x_k, u_{k|k}) \\ f(f(x_k, u_{k|k}), u_{k+1|k}) \\ f(f(f(x_k, u_{k|k}), u_{k+1|k}), u_{k+2|k}) \\ \vdots \\ f(f(f(x_k, u_{k|k}), u_{k+1|k}), \ldots, u_{k+N-1|k}) \end{pmatrix} \tag{4.11e}$$

$$-10 \leq g_i^{(2)} \leq 10, \quad -80 \leq u_{k+i|k} \leq 80. \quad \forall i \in \mathbb{I}_{[0,N-1]}, \tag{4.11f}$$

where g_i represent the ith position in vector $\boldsymbol{g} = \left[g_0^\top, g_1^\top, \ldots, g_N^\top\right]^\top$ and g_i^j is the jth element of the state vector g_i. The total number of decision variables for problem (4.9) is 504. By condensing the problem to (4.11), we reduce the number of decision variables to 100. The structure of the problem becomes easy to code but the Hessian is dense so not much is gain in term of computation complexity compare to (4.9).

Validation of the NMPC in simulation

To evaluate the performance of the NMPC, we utilized a Python-based simulation environment. Python was chosen due to its extensive set of control-oriented libraries such as CasADi [34]. CasADi provides a framework for efficient implementation of algorithms for nonlinear numerical optimization.

In CasADi, the OCP in (4.11) is constructed symbolically using symbolic expressions consisting of dense matrix-valued operations. These expressions are then differentiated in forward or reverse mode, after which an appropriate solver is employed to solve the optimization problem. CasADi supports several open-source nonlinear optimization solvers, such as IPOPT [35] ,SNOPT[36], WORHP[37].

In order to validate the design of our NMPC controller, we implemented the discrete-time dynamics of the Cart-Pole system, as described in Section (4.6), within the Python environment, using a sampling rate of $\Delta T = 0.01$. We used the CasADi framework within the Python environment to formulate the NMPC scheme as presented in Equation (4.11). The NMPC was then run for seven different initial conditions $x_0^j, \quad j = \{1, 2, \cdots, 7\}$, randomly drawn from the distribution of initial conditions X_0, as defined in (4.10) using a simulation time of $3s$.

In Figure 4.2, the closed-loop trajectories over the distribution of initial conditions are presented, and the NMPC controller is shown to have stabilized the unstable equilibrium point from all starting positions. Next, we turn toward the design of the NMPC controller using active subspaces.

Design of NMPC in active subspaces controller

As we have already seen in Section 2.1.3, NMPC in active subspaces uses the active subspace T_1 and inactive subspace T_2 orthogonal projector matrices to reduce the control input space. Additionally, the choice of T_1 and T_2 is generic and depends on the design of some feature vector $\mathcal{S}(x)$. We also observed that, instead of choosing $\mathcal{S}$, we could directly select the covariance matrix C. Our approach for reducing the input space will involve directly using suitable choices of covariance matrices. To this end, we present the Hessian H of the objective function for the linearized dynamics and the block identity matrix $I_{n\times n}$ as our choice of covariance matrices.

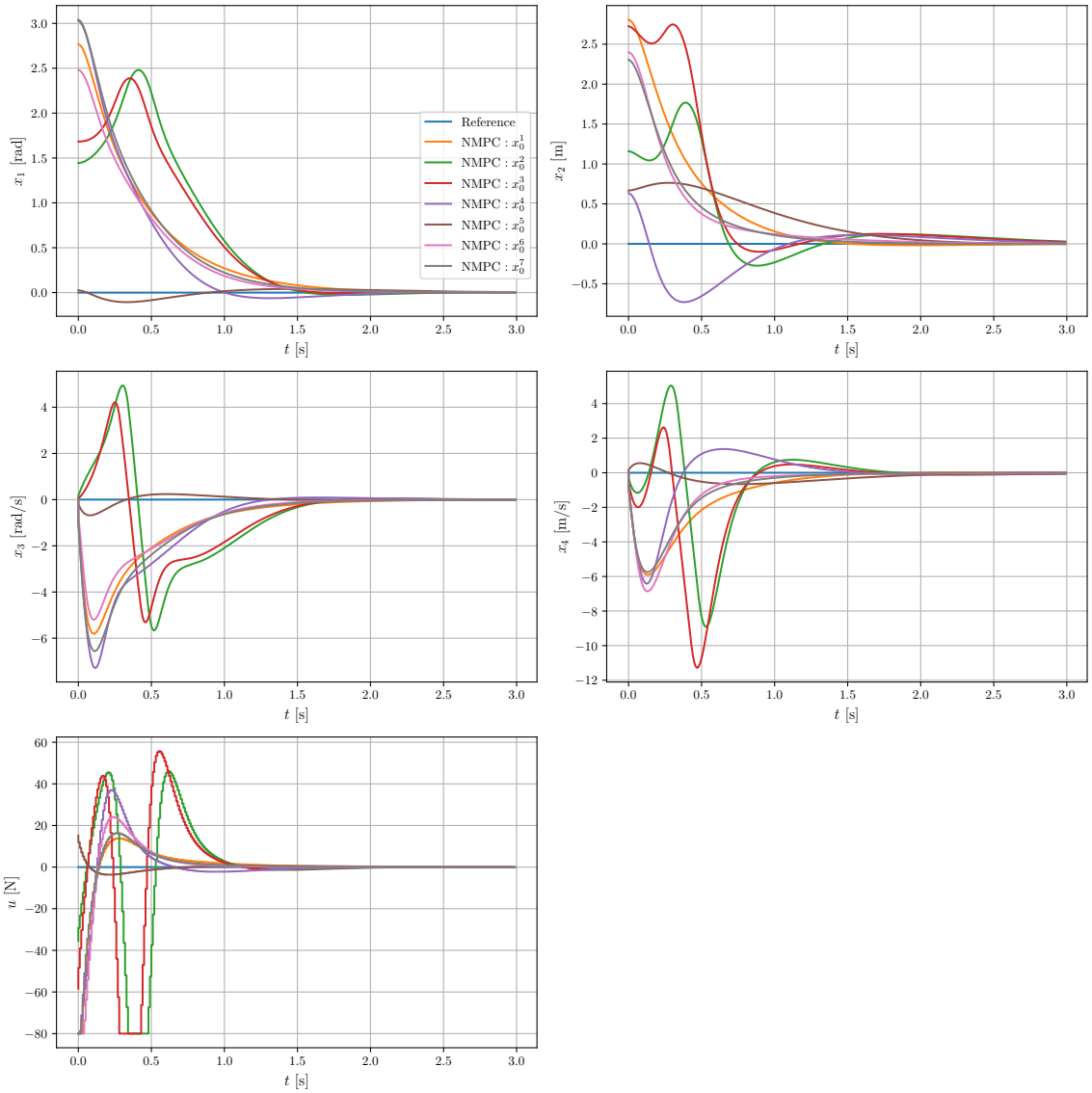

Figure 4.2: Closed-loop trajectories of NMPC for different initial conditions

The block identity matrix is defined as:

$$I_{n\times n} = \begin{bmatrix} \delta_{11} & \delta_{12} & \cdots & \delta_{1p} \\ \delta_{21} & \delta_{22} & \cdots & \delta_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ \delta_{n1} & \delta_{n2} & \cdots & \delta_{nn} \end{bmatrix}, \quad n = N \cdot n_u \tag{4.12}$$

where δ_{ij} is the Kronecker delta function defined as:

$$\delta_{ij} = \begin{cases} 1 & \text{if } i = j, \\ 0 & \text{if } i \neq j. \end{cases}$$

It should be noted that our interest in the block identity matrix as a choice of covariance matrix lies in its sparse nature. However, this method has its roots in the move-blocking strategy.

Using the linearized Cart-Pole dynamics in (4.8), the Hessian H of the linearized objective function for the condensed OCP is given as follows:

$$H = \boldsymbol{R} + \Gamma^\top \boldsymbol{Q} \Gamma \in \mathbb{R}^{N n_u \times N n_u} \tag{4.13}$$

where:

$$\Gamma = \begin{bmatrix} B & 0 & \dots & 0 \\ AB & B & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ A^{N-1}B & A^{N-2}B & \dots & B \end{bmatrix} \in \mathbb{R}^{N \cdot n_u \times N \cdot n_x}$$

$\boldsymbol{Q}$ and $\boldsymbol{R}$ are block diagonal matrices defined in (4.11). Having obtain the Hessian and block identity matrix , we then formulate the dimensionality reduced OCP of our condense OCP in (4.11) as follow:

$$\mathscr{P}(x_k, \tilde{\boldsymbol{w}}_k) : \min_{\boldsymbol{v}_k, \mu_k} \; \mathcal{J}(x_k, T_1 \boldsymbol{v}_k + \mu_k T_2 \tilde{\boldsymbol{w}}_k), \tag{4.14a}$$

$$\begin{aligned} \mathcal{J}(x_k, T_1 \boldsymbol{v}_k + \mu_k T_2 \tilde{\boldsymbol{w}}_k) = {} & \boldsymbol{g}(x_k, \boldsymbol{p}(T_1 \boldsymbol{v}_k + \mu_k T_2 \tilde{\boldsymbol{w}}_k))^\top \boldsymbol{Q} \boldsymbol{g}(x_k, \boldsymbol{p}(T_1 \boldsymbol{v}_k + \mu_k T_2 \tilde{\boldsymbol{w}}_k)) \\ & + \boldsymbol{p}(T_1 \boldsymbol{v}_k + \mu_k T_2 \tilde{\boldsymbol{w}}_k)^\top \boldsymbol{R} \boldsymbol{p}(T_1 \boldsymbol{v}_k + \mu_k T_2 \tilde{\boldsymbol{w}}_k). \end{aligned}$$

$$-10 \leq g_i^{(2)} \leq 10, \quad -80 \leq p_i \leq 80. \tag{4.14b}$$

$$x_k = s, \tag{4.14c}$$

Specifically, $\boldsymbol{p}$ represents the predicted input trajectory, given by $\boldsymbol{p} = \begin{bmatrix} p_0^\top & p_1^\top & \cdots & p_{N-1}^\top \end{bmatrix}^\top$, where $p_i \in \mathbb{R}^{n_u} \quad \forall i \in \mathbb{I}_{[0,N-1]}$. Note that the active and inactive subspace projection matrices, T_1 and T_2, can be directly obtained using (2.11), where the covariance matrix is chosen as the Hessian. Alternatively, T_1 and T_2 can be derived using the block identity matrix in (4.12), defined as:

$$T_1 = I_{N \cdot n_u \times n_v}, \quad T_2 = \text{null}(T_1) = I_{N \cdot n_u \times (N \cdot n_u - n_v)}.$$

Here, n_v represents the dimension of the active subspace vector v. Using Algorithm 1 with our dimensionally reduced OCP in (4.16), along with our update rule for the inactive subspace vector $\tilde{\boldsymbol{w}}_k$, we successfully designed the NMPC in active subspaces.

It is important to note that our update rule for $\tilde{\boldsymbol{w}}_k$ was designed slightly differently from the approach in [6]. Specifically, we did not use the state-feedback Linear Quadratic Regulator (LQR) controller for updating $\tilde{\boldsymbol{w}}_k$ as proposed in [6], since this would require the design of a gain-scheduling controller.

This necessity arises because our system starts from different initial positions, and for initial states far from the set point, the desired linear feedback controller might produce a poor

approximation of the infinite horizon cost-to-go. To avoid designing a gain-scheduling controller on top of our NMPC, we opted to use the concept of warm-starting for updating the inactive subspace vector $\tilde{\boldsymbol{w}}_k$. Our update rule is defined as follows:

$$\tilde{\boldsymbol{u}}_k = \left[\boldsymbol{u}_{k-1}, u_{k+N-1|k-1}^\top\right] = \left[u_{k|k-1}^\top, \ldots, u_{k+N-2|k-1}^\top, u_{k+N-1|k-1}^\top\right],$$

with

$$\tilde{\boldsymbol{w}}_k = T_2^\top \tilde{\boldsymbol{u}}_k, \tag{4.15}$$

where $\tilde{\boldsymbol{u}}_k$ is obtained from the last predicted input trajectory by shifting and appending $u_{k+N-1|k-1}^\top$.

It should be noted that this method could lead to a loss of recursive feasibility for highly nonlinear dynamics, as the optimization may easily get stuck in a local minimum.

Validation of the NMPC in active subspaces in simulation

After successfully designing our NMPC in active subspaces using a Python-based simulation together with the CasADi framework, we evaluated the performance of the controller. To ensure a fair comparison, we used the same distribution of initial conditions as those used for testing the controller of the full condensed problem.

We present two NMPC in active subspaces controllers. The first is the NMPC in active subspaces with the Hessian of the objective function for the linearized dynamics chosen as the covariance matrix for obtaining the projector matrices T_1 and T_2. The second is the NMPC in active subspaces with the block identity matrix chosen as the covariance matrix for obtaining the projector matrices T_1 and T_2. From here on, without any loss of generality, we will refer to the full condensed NMPC scheme simply as NMPC. For both dimensionality-reduced NMPC schemes, i.e., NMPC with $C = H$ and NMPC with $C = I$, we will mostly refer to them using their respective covariance matrices. The total number of decision variables for reduced problem is given by $n_v + 1$ with n_v being the dimension of the active subspace vector.

For the reduce problem we choose the same dimension for the active subspace vector v, $n_v = 60$. It should be noted that the choice of active subspace dimension using the covariance matrix should be as high as possible to minimize the loss in closed-loop performance.

In Figure 4.3, we present the closed-loop trajectories for the reduced NMPC with the covariance matrix $C = H$, using the same set of initial conditions as in the full condensed NMPC in active subspaces.

We see that our dimensionality-reduced controller was able to stabilize the Cart-Pole system for all initial conditions, just like the full NMPC. Next, in Figure 4.4, we present the closed-loop trajectories for the reduced NMPC with the covariance matrix $C = I$ under the same set of initial conditions. We see that the same conclusion holds as with the NMPC with $C = H$. This

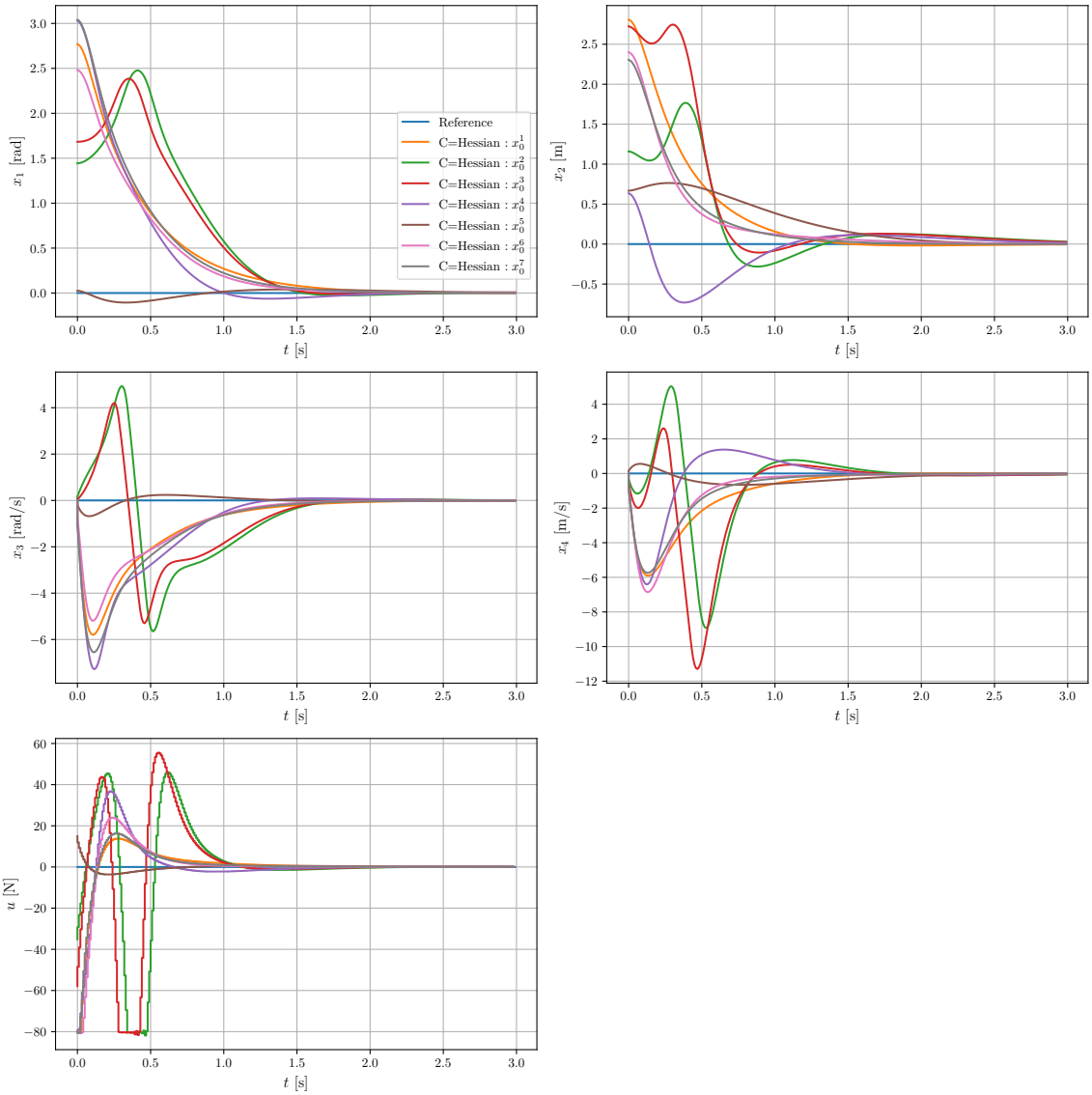

Figure 4.3: Closed-loop trajectories of NMPC with $C = H$ and $n_v + 1 = 61$, for different initial conditions

shows that stabilizing the system in the reduced input subspace is independent of the choice of covariance matrix.

Figure 4.5 shows the closed-loop trajectories for the initial condition $x_0 = [1.296, 2.858, 0, 0]^\top$ for all three controllers. As shown by the results in Figure 4.5, the closed-loop trajectories from both dimensionality-reduced controllers match very well with those of the full problem. To further assess the performance of the reduced controllers, we sample $N_b = 30$ batches of initial conditions from X_0, with each batch containing $N_{X_0} = 130$ different initial conditions, and compute the relative closed-loop performance with respect to the full problem, as defined in (3.26).

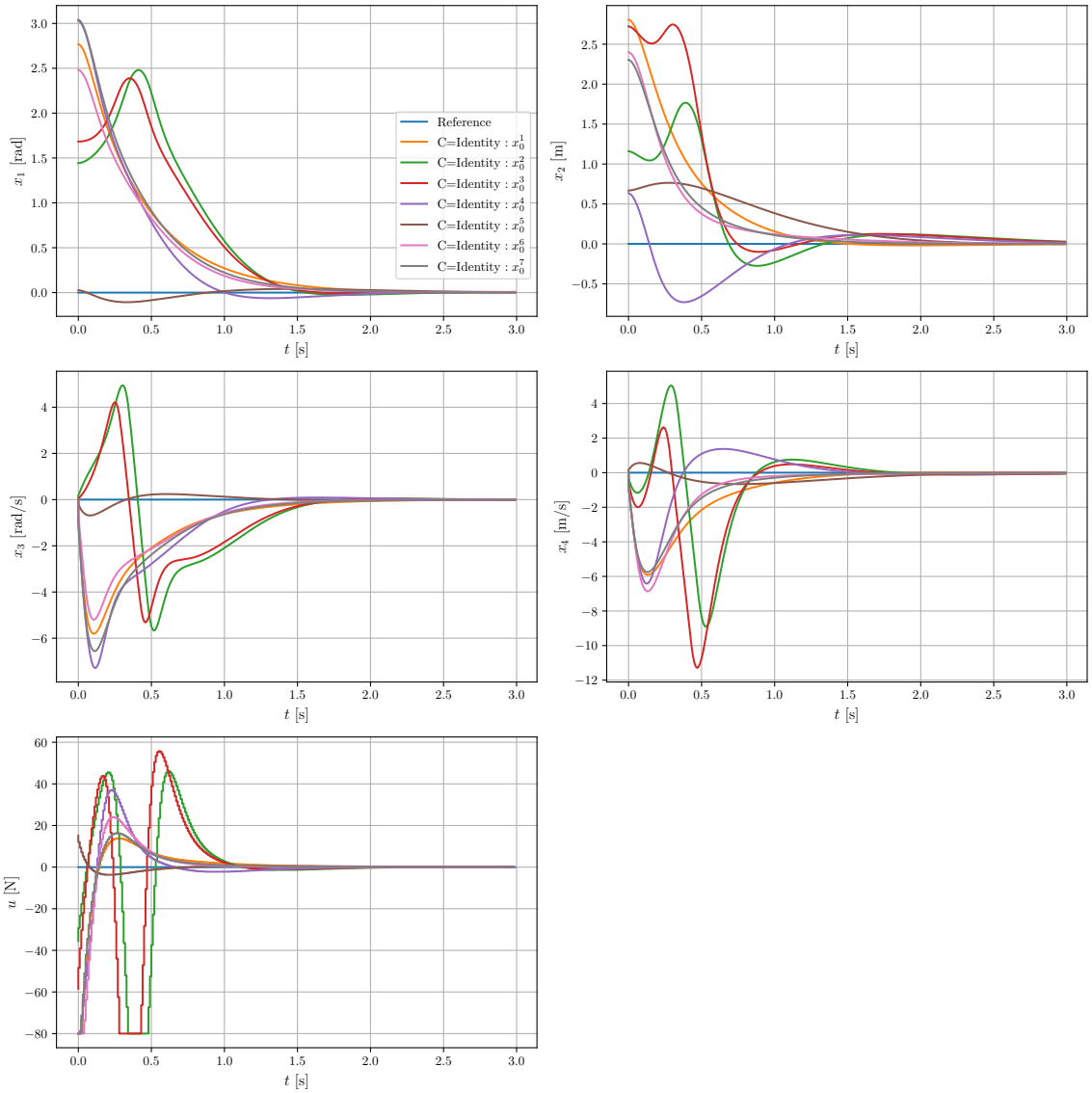

Figure 4.4: Closed-loop trajectories of NMPC with $C = I$ and $n_v + 1 = 61$, for different initial conditions

Implementation	**Number decision variables**	**Mean** [%]	**Std.** [%]
NMPC(C=H)	61	0.16	0.36
NMPC(C=I)	61	8.65×10^{-3}	0.065

Table 4.1: Relative performance difference δ (3.26) with $N_b = 30$, $N_{X_0} = 130$, and $N \cdot n_u = 100$.

In order to assess the computational complexity of the different controllers, we computed the mean computation time for $N_{X_0} = 130$ initial conditions across all three different NMPC schemes. The result is shown in Table 4.16.

From the results presented in Table 4.1, it shows that both dimensionality-reduced controllers closely match the full NMPC, with a minimal mean closed-loop performance loss of less than

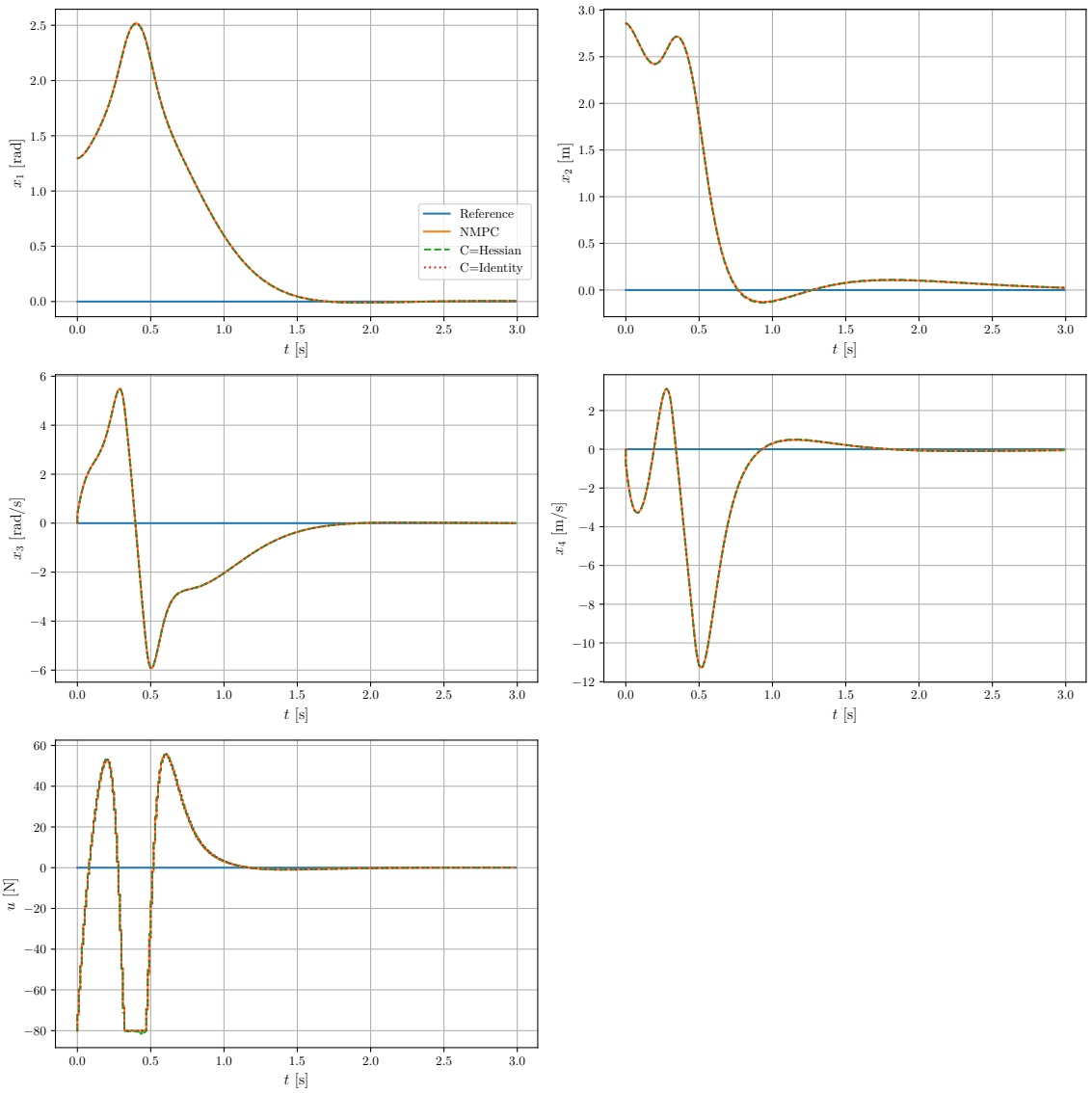

Figure 4.5: Closed-Loop trajectories for $x_0 = [1.296, 2.858, 0, 0]^\top$ $n_v + 1 = 61$ and $N \cdot n_u = 100$

Implementation	**Number decision variables**	**Mean [s]**	**Std.[s]**	**max [s]**
NMPC	100	0.1732	0.0944	2.7014
NMPC(C=H)	61	0.3087	0.1370	4.2013
NMPC(C=I)	61	0.1369	0.0548	3.6415

Table 4.2: Mean computation time of different NMPC schemes with 130 initial conditions

2%. We also notice in Table 4.16 that the dimensionality-reduced NMPC scheme with $C = H$ becomes slower than the full problem, even though the size of the decision variables has been reduced.

This highlights an interesting concept: reducing the dimension of the input space does not always reduce the computational burden, especially when using single shooting. This is because

the Hessian matrix in the single shooting method is dense, and using a dense active subspace projector matrix T_1 adds to the computational burden through the additional input equality constraints $\boldsymbol{u_k} = T_1 v_k + \mu_k T_2 \tilde{\boldsymbol{w}}_k$. The computational burden depends on the sparsity of T_1 and the dimension of v_k.

This concept is demonstrated by considering the NMPC scheme with the block identity matrix. Here, the projector matrix T_1 is sparse, so even though the active subspace dimension is the same as that of the NMPC with the Hessian, it performs better in terms of computational complexity than both controllers.

Implementation of Algorithm 5

After designing and validating our NMPC controllers in simulation, we turn our attention to implementing Algorithm 5 in Python. The aim of the algorithm is to determine the optimal active subspace dimension using the concept of PCA, as discussed in Section 3.4. The main components required to implement Algorithm 5 are: the choice of the covariance matrix C, the distribution of initial conditions X_0, the variance threshold for determining the principal eigenvalues, and the active subspace dimension $d = n_v$ for the given covariance matrix.

In the case of our Cart-Pole system, we have already observed that with the choices $C = H$ and $C = I$, our dimensionality-reduced NMPC scheme successfully stabilized the system starting from any initial condition drawn from X_0. For the active subspace dimension, we retained $n_v = 60$, as its closed-loop performance closely matched that of the full-order problem. It should be noted that, in order for PCA to capture significant variance, it is advisable to choose the active subspace dimension for the covariance matrix as large as possible, minimizing the loss in closed-loop performance.

Regarding the variance threshold for selecting the principal eigenvalues, it is always a trade-off between performance and computational complexity. For our problem, we chose a variance threshold of 2%, meaning that we retained only eigenvalues whose cumulative variance contribution exceeded 2%. With our design choices of $C = I$, $C = H$, $d = n_v = 60$, and $\eta = 2\%$, we successfully implemented Algorithm 5 using Python and the CasADi framework.

Validation of Algorithm 5 in Simulation

In Figure 4.6, we present the closed-loop trajectories of the full-order NMPC and the dimensionality-reduced NMPC scheme with the active subspace projector matrix obtained from PCA, where the covariance matrices are chosen as $C = H$ and $C = I$. We note that after running algorithm 5, the optimal active subspace dimension obtained was $n_v = 5$ for both choices of covariance matrices. From the closed-loop trajectories presented in Figure 4.6, the dimensionality reduced NMPC with PCA active subspace projector matrices was still able to stabilize the system for

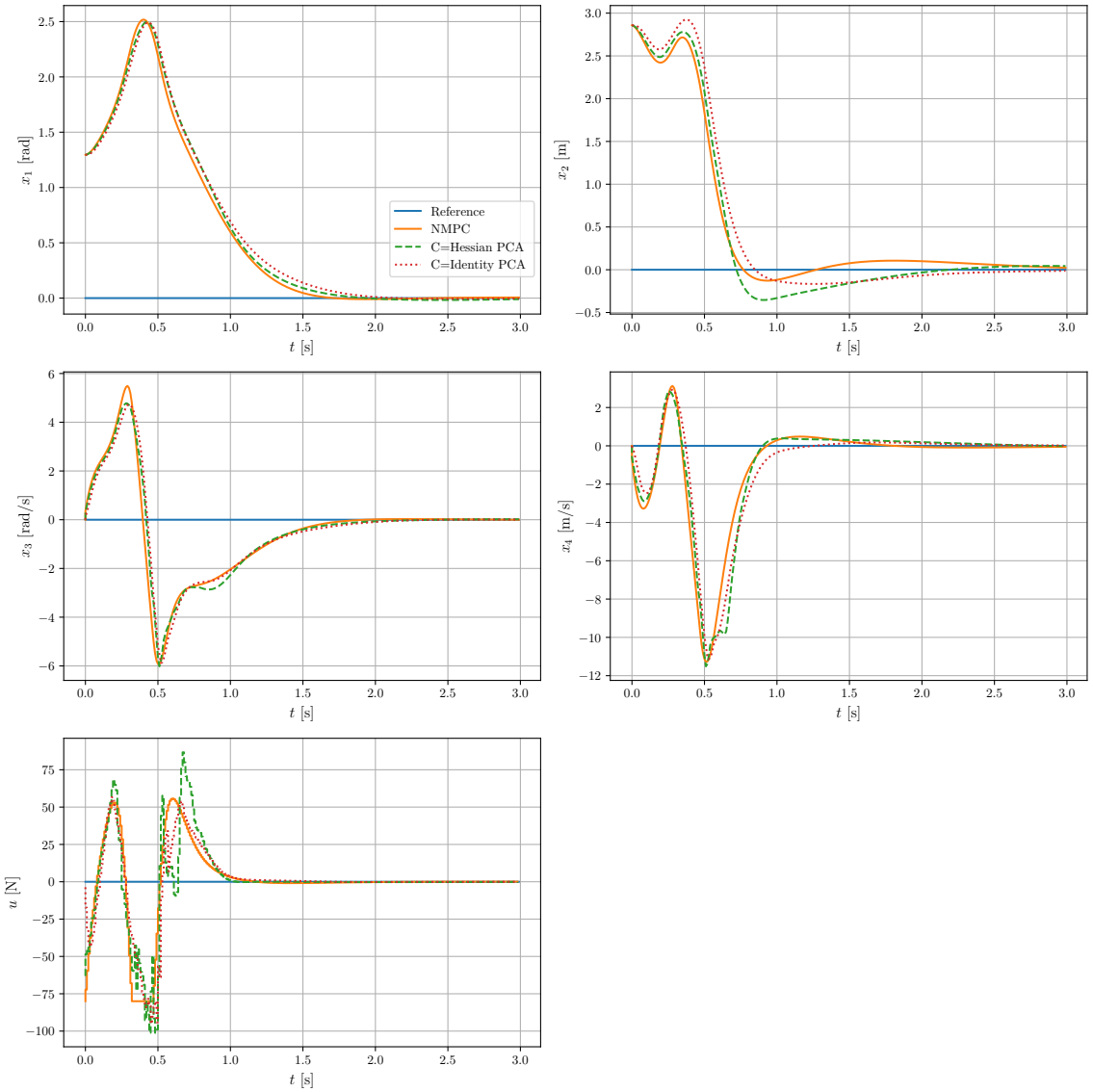

Figure 4.6: Closed-loop trajectories for $x_0 = [1.296, 2.858, 0, 0]^\top$ after PCA with $n_v + 1 = 6$ and $N \cdot n_u = 100$

the given initial conditions. In order to futher asses the performance of the active subspace projector obtain from PCA we use the same sample initial conditions as in Table 4.1

Implementation	**Number decision variables**	**Mean [%]**	**Std. [%]**
NMPC(C=H)	6	6.03	5.82
NMPC(C=I)	6	2.647	3.66

Table 4.3: Relative performance difference δ (3.26) with $N_b = 30$, $N_{x_0} = 130$, and $N \cdot n_u = 100$ after PCA.

From Table 4.3, we see that although the dimensionality-reduced NMPC with PCA active subspace projector matrices was able to stabilize the Cart-Pole system, its relative loss in closed-

Implementation	Number decision variables	Mean [s]	Std.[s]	max [s]
NMPC	100	0.1732	0.0944	2.7014
NMPC(C=H)	6	0.0976	0.0476	0.9410
NMPC(C=I)	6	0.072	0.0296	0.456

Table 4.4: Mean computation time of different NMPC schemes with 130 initial conditions after PCA

loop performance with respect to the full problem has increased. However, the computational time in both cases is now better than that of the full-order NMPC scheme.

Our next objective is to implement the MPC-Q learning algorithm to find an active subspace projector that achieves both improved closed-loop performance and reduced computational time.

Implementation of MPCQ-learning Algorithm

So far, we have implemented both the full-order and dimensionality-reduced NMPC controllers and validated them in simulation. Subsequently, we implemented the PCA algorithm and validated it through simulation as well. As discuss in the theory we aim at using RL to tune the active subspace projector matrix. To this end let us turn our attention in implementing the RL framework.

The first step was to implement a reinforcement learning environment using OpenAI Gym [38]. OpenAI Gym is an open-source Python library that provides a predefined framework for developing RL environments, making it highly suitable for our implementation. The Cart-Pole dynamics and reward function were implemented based on this framework. The reward function was the same as the stage cost of our NMPC scheme

The next step was to implement the parametrize dimensionality reduce NMPC agent defined as follow: NMPC scheme reads:

$$\mathscr{P}_\theta(x_k, \tilde{\boldsymbol{w}}_k) : \min_{\boldsymbol{v}_k, \mu_k} \; \mathcal{J}_\theta(x_k, \boldsymbol{v}_k, \mu_k, \tilde{\boldsymbol{w}}_k), \tag{4.16a}$$

$$\begin{aligned}\mathcal{J}_\theta(x_k, \boldsymbol{v}_k, \mu_k, \tilde{\boldsymbol{w}}_k) = \boldsymbol{g}_\theta(x_k, \boldsymbol{p}_\theta(v_k, \mu_k, \tilde{w}_k))^\top \boldsymbol{Q} \boldsymbol{g}_\theta(x_k, \boldsymbol{p}_\theta(v_k, \mu_k, \tilde{w}_k)) \\ + \boldsymbol{p}_\theta(v_k, \mu_k, \tilde{w}_k)^\top \boldsymbol{R} \boldsymbol{p}_\theta(v_k, \mu_k, \tilde{w}_k)\end{aligned}$$

$$-10 \leq g_{\theta,i}^{(2)} \leq 10, \quad -80 \leq p_{\theta,i} \leq 80. \tag{4.16b}$$

$$x_k = s, \tag{4.16c}$$

where

$$\boldsymbol{p}_\theta : \mathbb{R}^{n_v} \times \mathbb{R} \times \mathbb{R}^{n_{\tilde{w}}} \to \mathbb{R}^{N \cdot n_u}$$

and

$$\boldsymbol{g}_\theta : \mathbb{R}^{n_x} \times \mathbb{R}^{N \cdot n_u} \to \mathbb{R}^{(N+1) \cdot n_x}$$

are the parameterized predicted input and state trajectories of the reduced NMPC scheme. Specifically,

$$\boldsymbol{p}_\theta = \begin{bmatrix} p_{\theta,0}^\top & p_{\theta,1}^\top & \cdots & p_{\theta,N-1}^\top \end{bmatrix}^\top,$$

where each $p_{\theta,i} \in \mathbb{R}^{n_u}$, and

$$\boldsymbol{g}_\theta = \begin{bmatrix} g_{\theta,0}^\top & g_{\theta,1}^\top & \cdots & g_{\theta,N}^\top \end{bmatrix}^\top,$$

where each $g_{\theta,i} \in \mathbb{R}^{n_x}$, with $g_{\theta,i}^j$ representing the jth column of $g_{\theta,i}$. The implementation was done using Python and the CasADi framework. It should be noted that we did not have to implement this from scratch, as we had already implemented the dimensionality-reduced NMPC scheme in (4.16). We only needed to set up the learnable parameter, which is the active subspace projector T_1. Additionally, the size of T_1 is $Nn_u \times n_v = 100 \times 5$, giving a total of 500 parameters to learn. Once we have set up our dimensionality-reduced NMPC agent, the next step is setting up the parameterized state-action value function $Q_\theta(x_k, a_k, \tilde{\boldsymbol{w}}_k)$ where

$$V_\theta(x_k, \tilde{\boldsymbol{w}}) = \min_{p_k} \; Q_\theta(x_k, p_k, \tilde{\boldsymbol{w}}_k) \tag{4.17}$$

and $V_\theta(x_k, \tilde{\boldsymbol{w}}_k)$ is the value function approximation of the solution to $\mathscr{P}_\theta(x_k, \tilde{\boldsymbol{w}}_k)$

The final implementation to be done is the RL update rule in (3.24). As discussed in Chapter 3, in order to preserve the orthonormality constraints of the active subspace projector, we need to reformulate the Q-learning TD update rule as an optimization on the Stiefel manifold. Python offers toolboxes like Pymanopt [39], which support optimization on Riemannian manifolds with automatic differentiation capabilities. It should be noted here that manifold optimization problems are mostly non-convex smooth optimization problems, so the guarantee of finding a global minimum for the problem is always challenging. Our approach, therefore, is to find the best suboptimal solution among our set of possible solutions.

After reformulating the update rule problem in Pymanopt, the next step was to decide which optimizer is best suited for our problem. Pymanopt offers a series of optimizers, both gradient-based and non-gradient-based, including Steepest Descent, Conjugate Gradients, Particle Swarms, Trust Regions, and others. We decided to use the Trust Regions optimizer. It should be noted that our choice of the Trust Regions algorithm was motivated by [40].

The final step was integrating all the code to form the MPC-Q-learning algorithm. The workflow of this simulation environment is quite straightforward. Essentially, on top of the dimensionality-reduced NMPC simulation, a RL update rule on the manifold is initiated after the completion of each stabilization of the unstable equilibrium experiment. Parameters are updated accordingly, and a new experiment can be undertaken. Thus, the flow consists of the interleaving of experimentation and training, with each mutually influencing the other.

4.2 Results

After extensive tuning of the RL hyperparameters (specifically the learning rate α) and the parameter for smoothing the manifold (ρ), we finally reached a point where we could obtain meaningful results from the learning process. We start by presenting the results before and after RL.

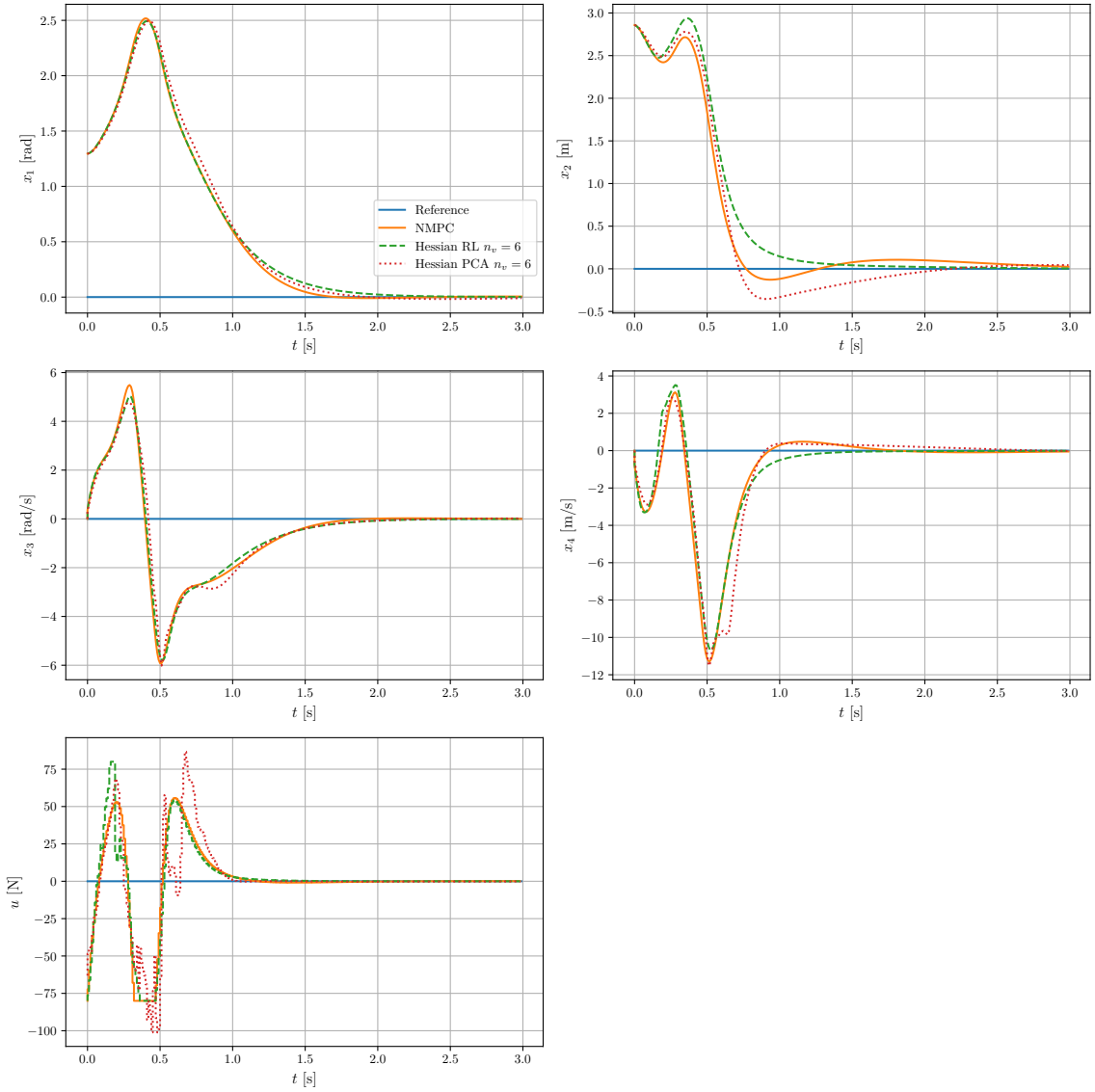

Figure 4.7: Closed-loop trajectories for $x_0 = [1.296, 2.858, 0, 0]^\top$ after RL with $n_v + 1 = 6$ and $N \cdot n_u = 100, K = 1300, C = H$

In Figure 4.7, we present the closed-loop trajectories of the dimensionality-reduced NMPC scheme using the trained active subspace projectors after 1300 episodes of learning. Overlaid on the plots are the closed-loop trajectories obtained from full order problem and dimensionality reduced NMPC in active subspace with PCA. By comparing the two plots, we observe an

improvement, indicating that our algorithm converges to a suboptimal active subspace projector matrix. Figure 4.8 with block identity matrix use for PCA as show a similar arguement.

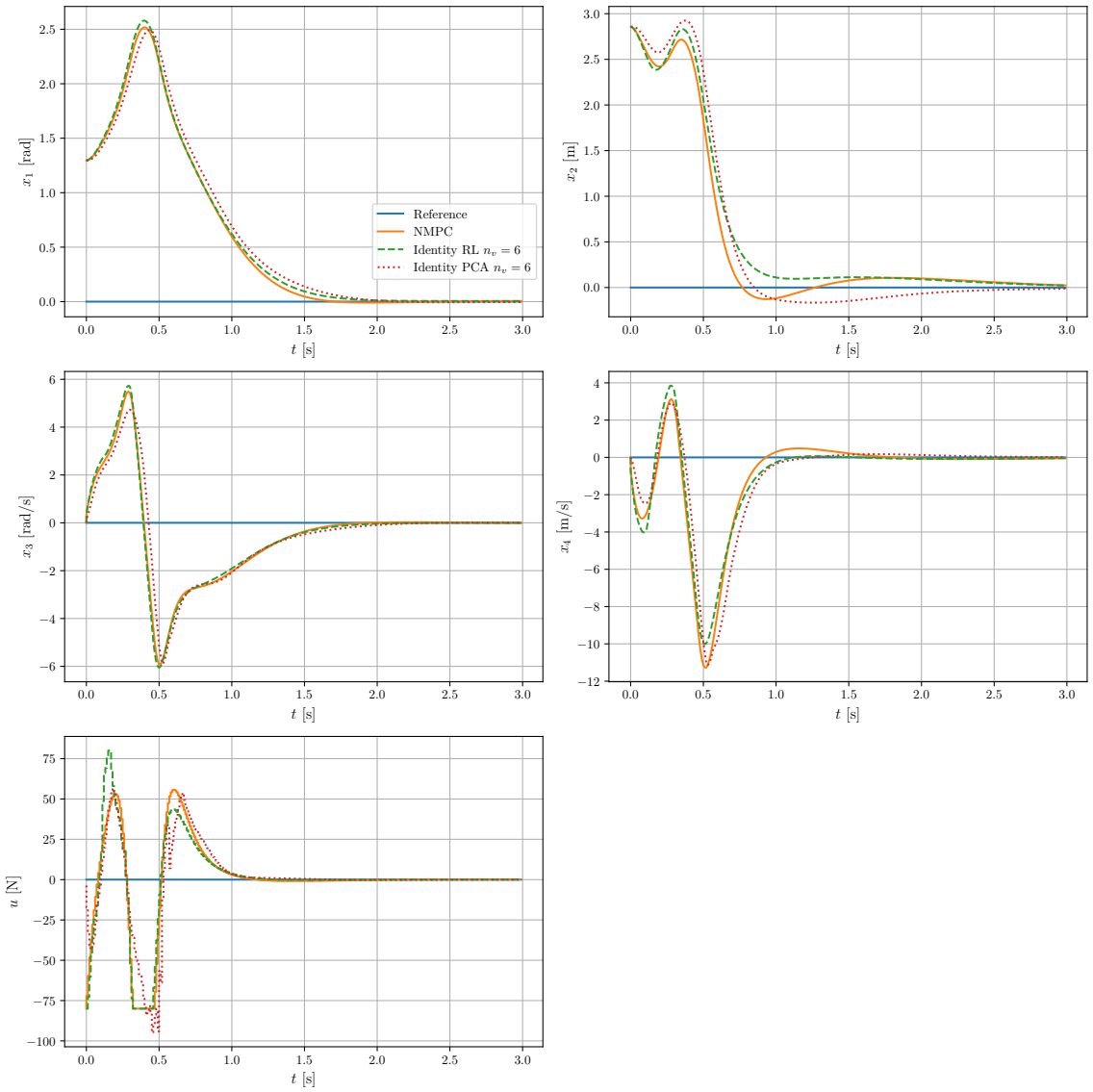

Figure 4.8: Closed-loop trajectories for $x_0 = [1.296, 2.858, 0, 0]^\top$ after RL with $n_v + 1 = 6$ and $N \cdot n_u = 100, K = 1300, C = I$

To further validate the results obtained, we compute the relative closed-loop performance from a sample batch of $N_b = 30$ drawn from X_0, with each batch containing $N_{X_0} = 130$ initial conditions.

In Table 4.5, we observe that after training, the relative closed-loop performance of both dimensionality-reduced NMPC schemes, compared to the full problem, has improved. The stopping criterion for our MPCQ-learning algorithm is based on a relative closed-loop performance threshold. In this project, it was set to less than 2%. This means that we only accept a suboptimal policy as optimal if its relative closed-loop performance, compared to

Implementation	Number decision variables	Mean [%]	Std. [%]
NMPC(C=H)	6	0.298	1.438
NMPC(C=I)	6	0.297	1.439

Table 4.5: Relative performance difference δ (3.26) with $N_b = 30$, $N_{x_0} = 130$, and $N \cdot n_u = 100$ after RL.

Implementation	Number decision variables	Mean [s]	Std.[s]	max [s]
NMPC	100	0.1732	0.0944	2.7014
NMPC(C=H)	6	0.0722	0.0307	0.4922
NMPC(C=I)	6	0.0699	0.0279	0.4626

Table 4.6: Mean computation time of different NMPC schemes with 130 initial conditions after RL

the best closed-loop performance, is less than 2%. Therefore, if we find a policy among the dimensionality-reduced policies that satisfies this closed-loop performance threshold, we consider that policy to be suboptimal.

Next, we run the NMPC scheme with both RL suboptimal active subspace projectors for seven different initial conditions outside the training set. Among these, we had cases where the input constraints were active.

Figures 4.9 and 4.9 show the closed-loop trajectories for both RL policies. We can see that even in situations where the input constraints were active, our reduced NMPC in active spaces controller could still match the full-order problem, albeit with some minimal loss in closed-loop performance.

So far, we have not observed much difference between the two trained policies. One might argue that the search for the suboptimal active subspace is independent of the choice of covariance matrix. However, our interest in training with the block identity matrix was to investigate whether it could lead to better performance than the Hessian. Specifically, we aimed to use the matrix to alleviate the computational burden and explore whether we could achieve faster performance than the state-of-the-art multiple shooting method, with minimal loss in closed-loop performance.

Next, we present in Figure 4.11 the closed-loop trajectories of the input at different training episodes. From these training episodes, it is evident that our safe RL update rule is working properly, as there were minimal violations of constraints and recursive feasibility was guaranteed. Even though some control trajectories were noisy, they always drove the system toward the set point.

The goal of the algorithm was achieved. We used our algorithm to find a suboptimal active subspace projector matrix for two different choices of covariance matrices. However, the question we ask ourselves is: Can we do even better? Can we reduce the dimension further and

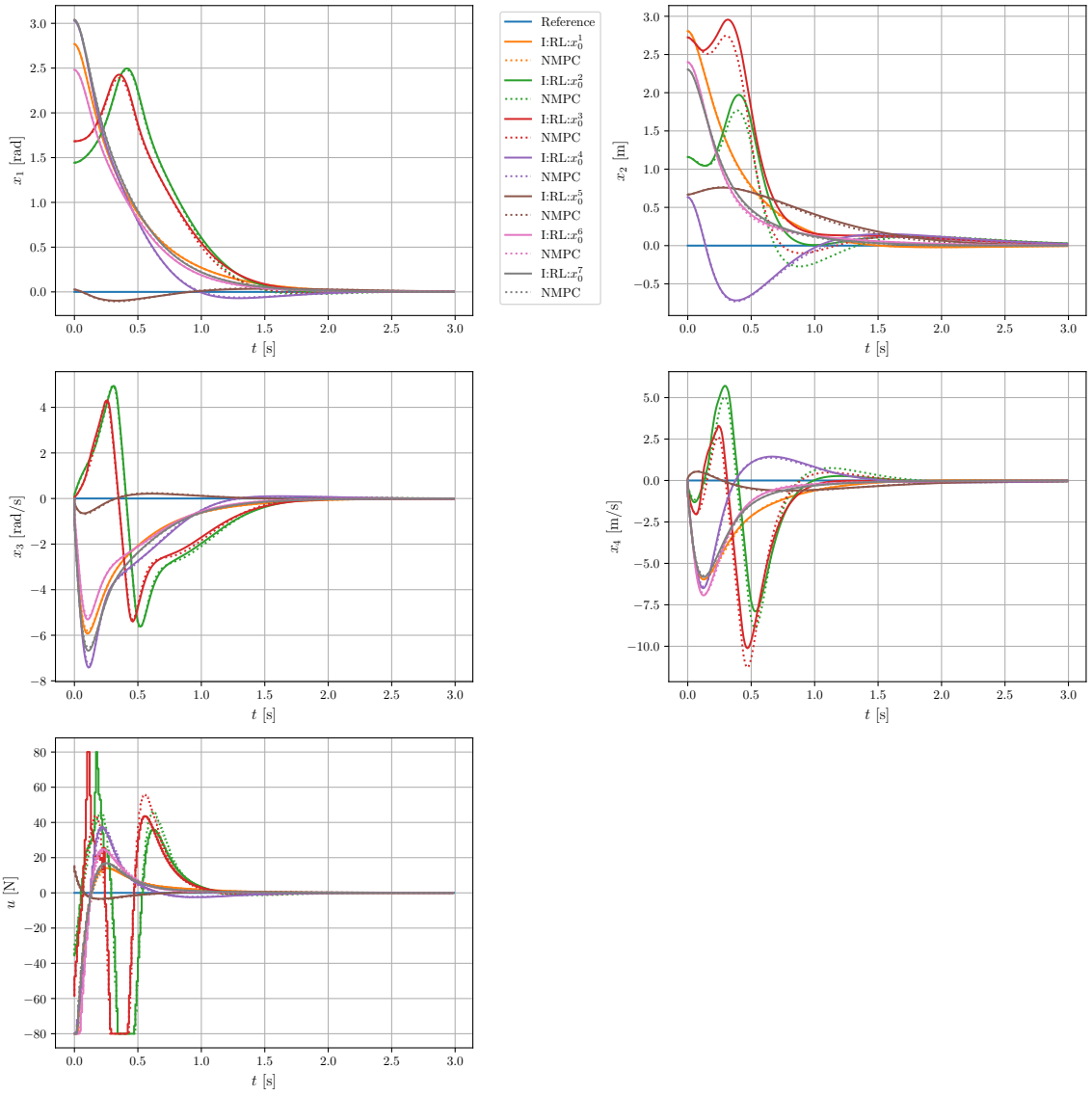

Figure 4.9: Closed-loop trajectories for different initial conditions with the RL Policy,$n_v + 1 = 6$ and $N \cdot n_u = 100$,$C = I$

still achieve better closed-loop performance? The answer to this question lies within the PCA method we use to determine the optimal active subspace dimension.

In order to investigate this, we ran the MPCQ-learning algorithm again with three different variance thresholds: $\eta = 0.001\%$, $\eta = 0.1\%$, and $\eta = 10\%$. It should be noted here that these values were chosen out of curiosity, with no specific claims behind them. Next, the PCA algorithm in 5 was executed with the same parameters as before, using the two choices of covariance matrices, C and H.

The first part of the results is presented in Figure 4.12, which shows the closed-loop trajectories for the dimensionality-reduced NMPC scheme using active subspace projector matrices obtained

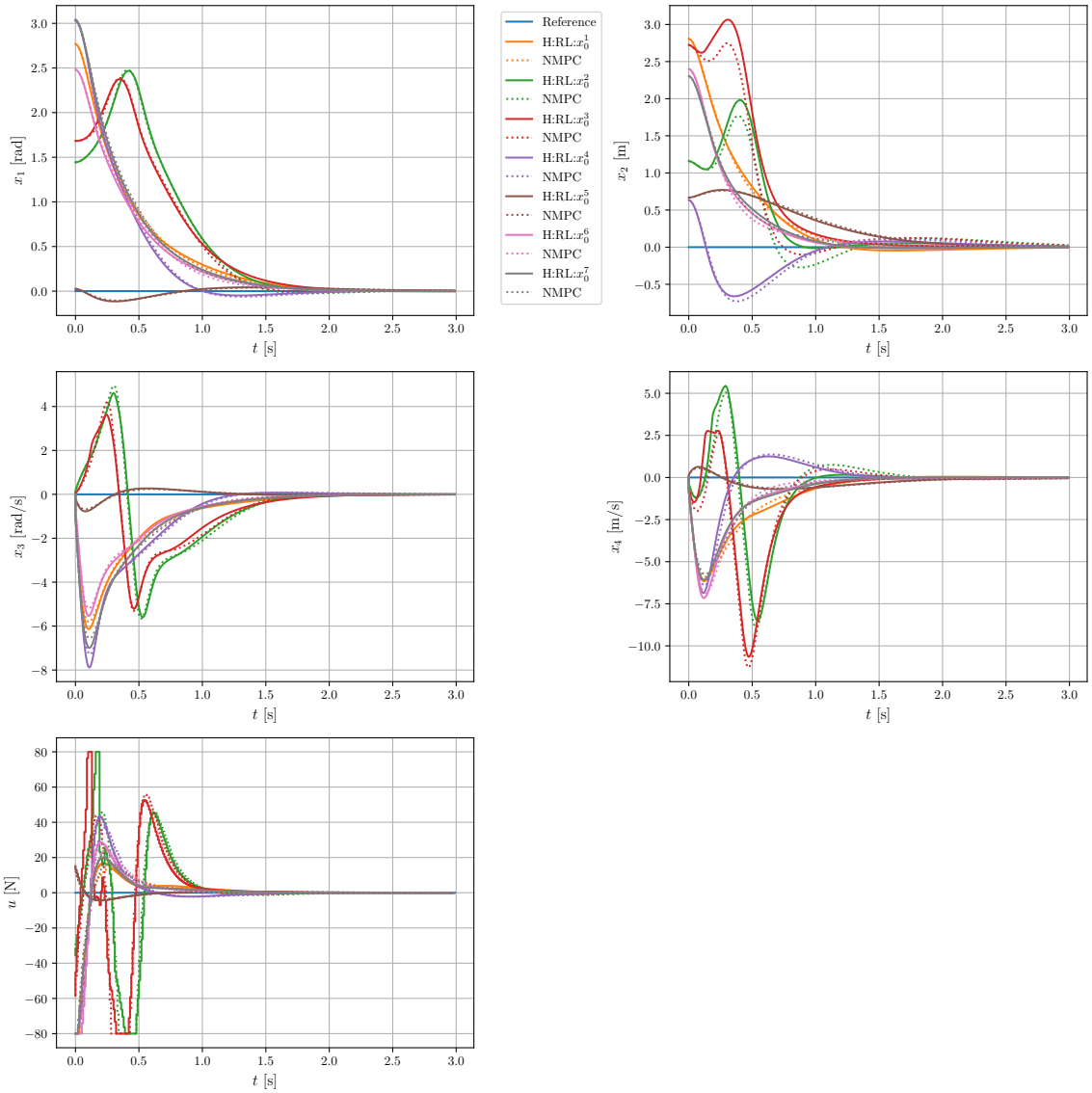

Figure 4.10: Closed-loop trajectories for different initial conditions with the RL policy $n_v+1 = 6$ and $N \cdot n_u = 100$, $C = H$

from different choices of variance thresholds. It should be noted that the covariance matrix used for running the PCA algorithm here was the Hessian.

From the different threshold values, it can be observed that as one tries to increases the variance threshold in an attempt to reduce the active subspace dimension, the closed-loop trajectories tend to deviate significantly from the full problem. In summary, the PCA approach was unable to capture sufficient variance as the threshold increased. But then the next question is: What if our MPCQ-learning algorithm is still able to tune the suboptimal active subspace? To this end, we ran the MPCQ-learning algorithm with these chosen active subspaces, and Figure 4.13 summarizes the results.

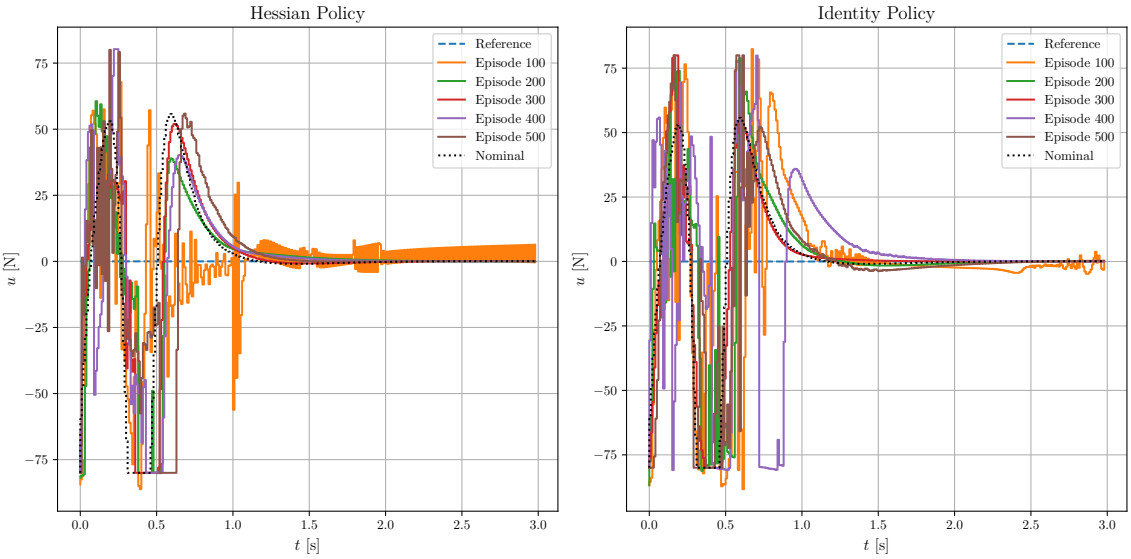

Figure 4.11: Closed loop trajectories of RL Policies for different training episodes $n_v + 1 = 6$ and $N \cdot n_u = 100$

We can see in Figure 4.13 that the claim appears to be true: we can always improve the active subspace projector matrix using our MPCQ-learning algorithm. We performed the same experiment, but this time using the block identity matrix. In Figure 4.14, we present the results of the closed-loop trajectories. One remarkable observation is that the choice of active subspace dimensions tends to differ at a variance threshold of $\eta = 0.001$. It is $n_v = 30$ for the identity matrix and $n_v = 23$ for the Hessian. The question that arise if,this might impact the result of the learning for the active subspace.

In Figure 4.15, we observe that, although the results from running the PCA algorithm with the block identity matrix were poor, its learning outcome was significantly better than that of its Hessian counterpart.

Next, to gain more insight, we draw a single batch of $N_{X_0} = 30$ different initial conditions from X_0. We then select ten active subspace dimensions, $n_v = \{1, \cdots, 10\}$. , From both covariance matrices H and I we choose ten active subspace projector matrices. Subsequently, we run the dimensionality-reduced NMPC in the active subspace and compute the relative closed-loop performance as defined in 2.19, along with the mean computation time.

Also for the computation complexity, the average time for the state of the art multiple shooting method. So give a good benchmark to our method. The results are presented in Figure 4.16. It was interesting to observe that our RL policy with $n_v = 6$ outperformed the state-of-the-art multiple shooting NMPC. This suggests that if we could train the active subspace projector matrix starting from the block identity matrix, with an active subspace vector dimension in $1 \leq x < 20$, there is a strong potential to consistently achieve a reduced NMPC in active subspace controller that is faster than multiple shooting, with minimal loss in closed-loop

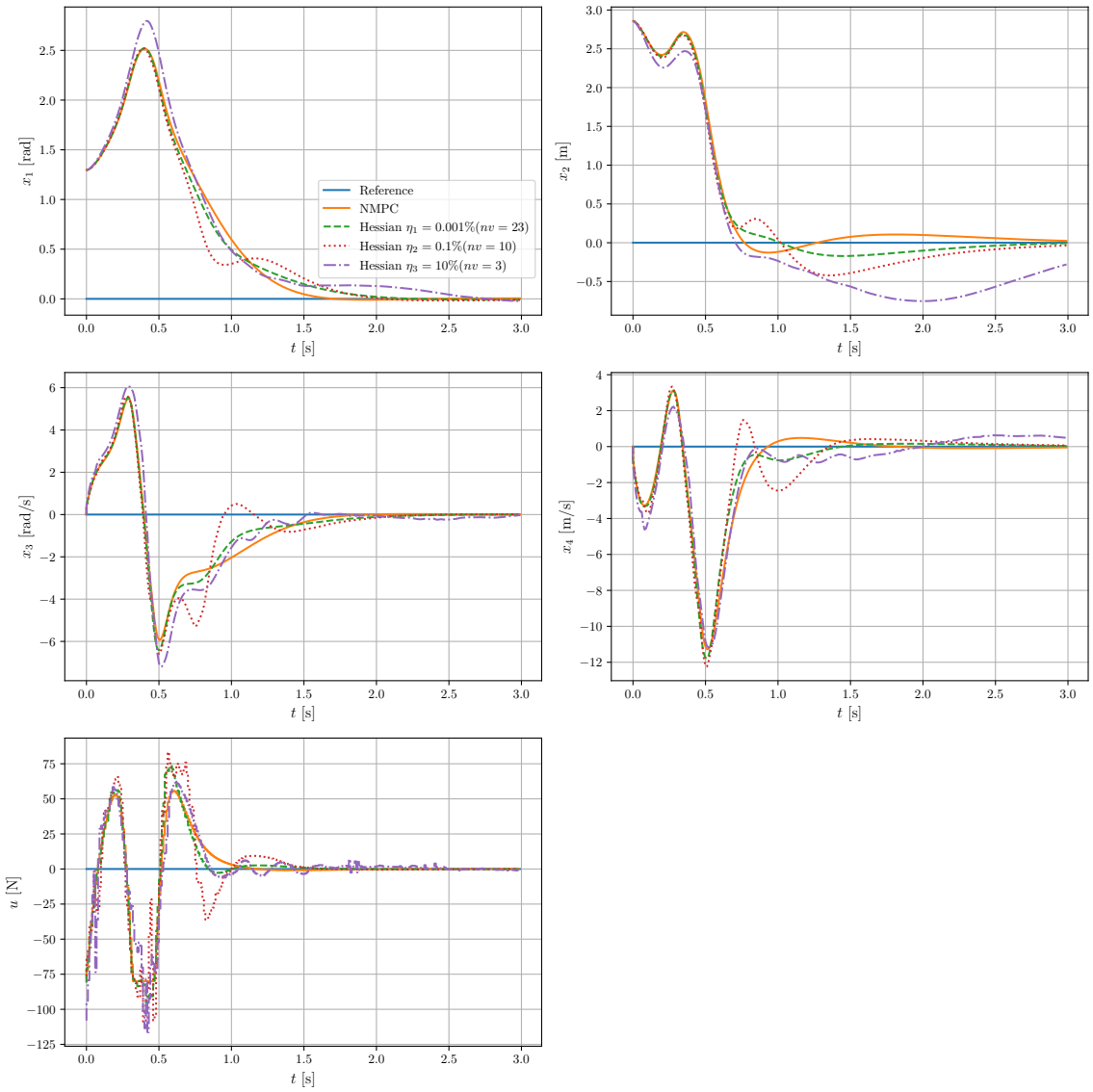

Figure 4.12: Closed-loop trajectories for $x_0 = [1.296, 2.858, 0, 0]^\top$ after PCA with $n_v + 1 = 23, n_v + 1 = 10, n_v + 1 = 4$ and $N \cdot n_u = 100$, $C = H$

performance. However, the key lies in formulating the RL update in such a way that it preserves the sparse nature of the block identity matrix active subspace projector matrix.

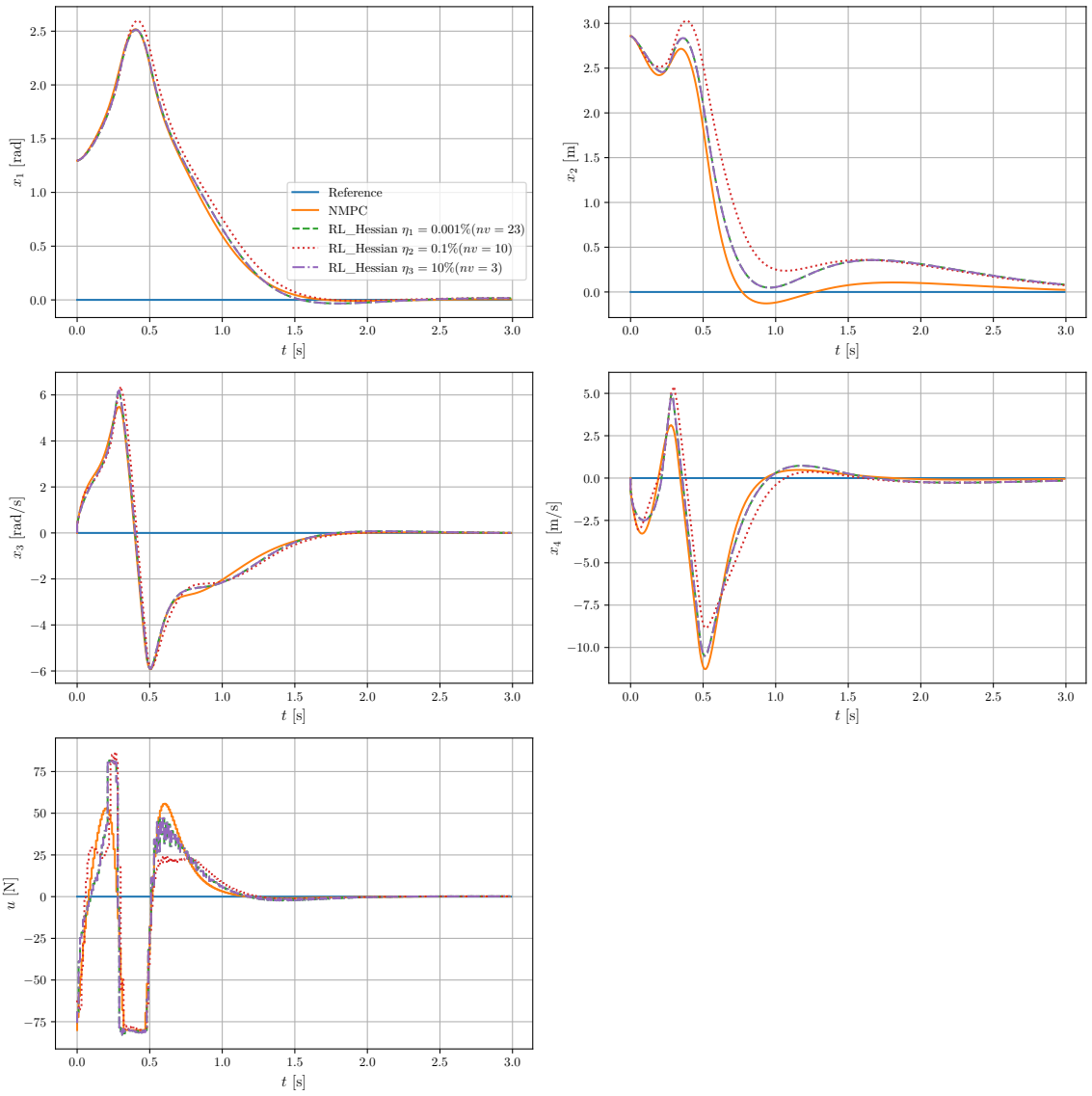

Figure 4.13: Closed-loop trajectories for $x_0 = [1.296, 2.858, 0, 0]^\top$ after RL with $n_v{+}1 = 23, n_v{+}1 = 10, n_v + 1 = 4$ and $N \cdot n_u = 100$, $C = H$

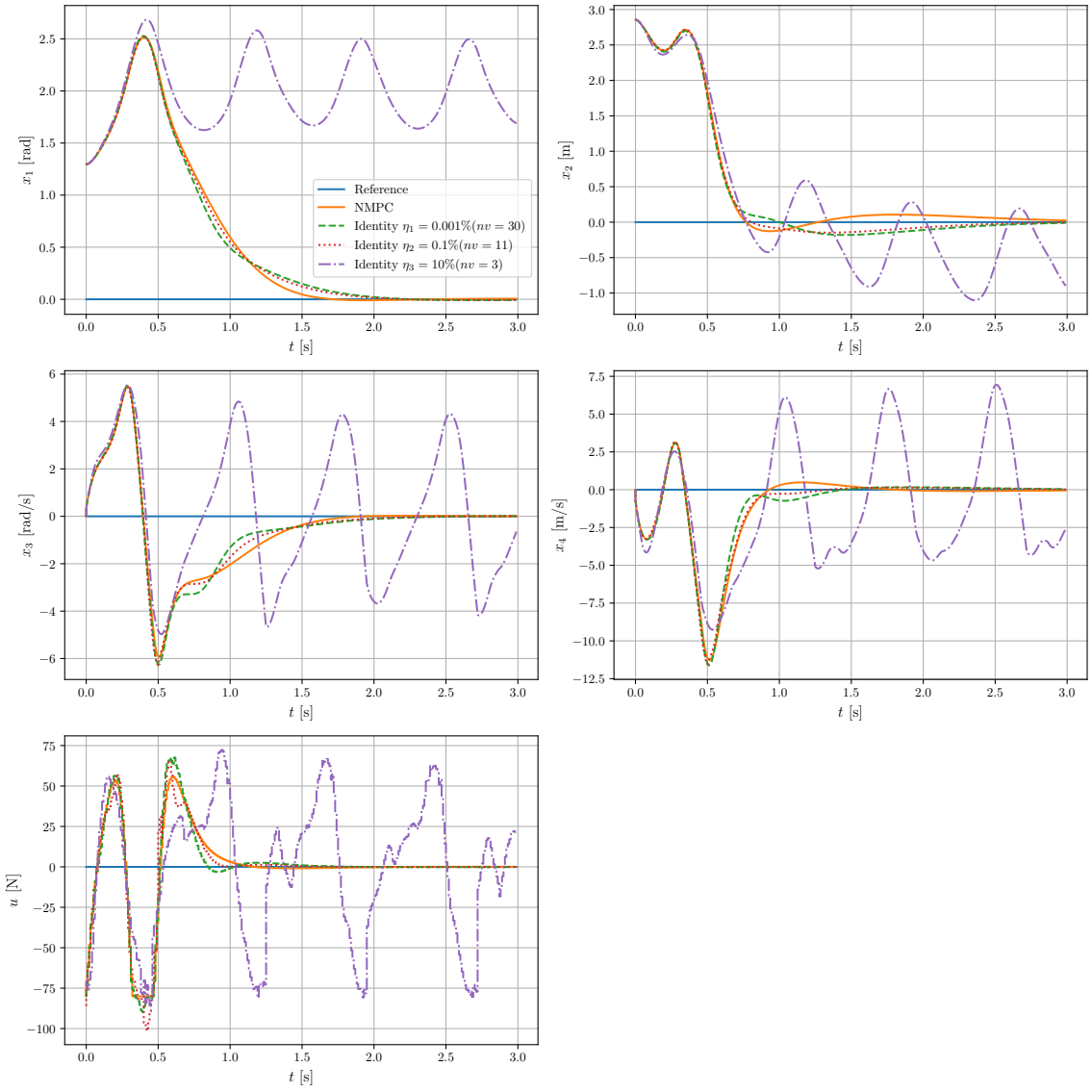

Figure 4.14: Closed-loop trajectories for $x_0 = [1.296, 2.858, 0, 0]^\top$ after PCA with $n_v + 1 = 23, n_v + 1 = 10, n_v + 1 = 4$ and $N \cdot n_u = 100$, $C = I$

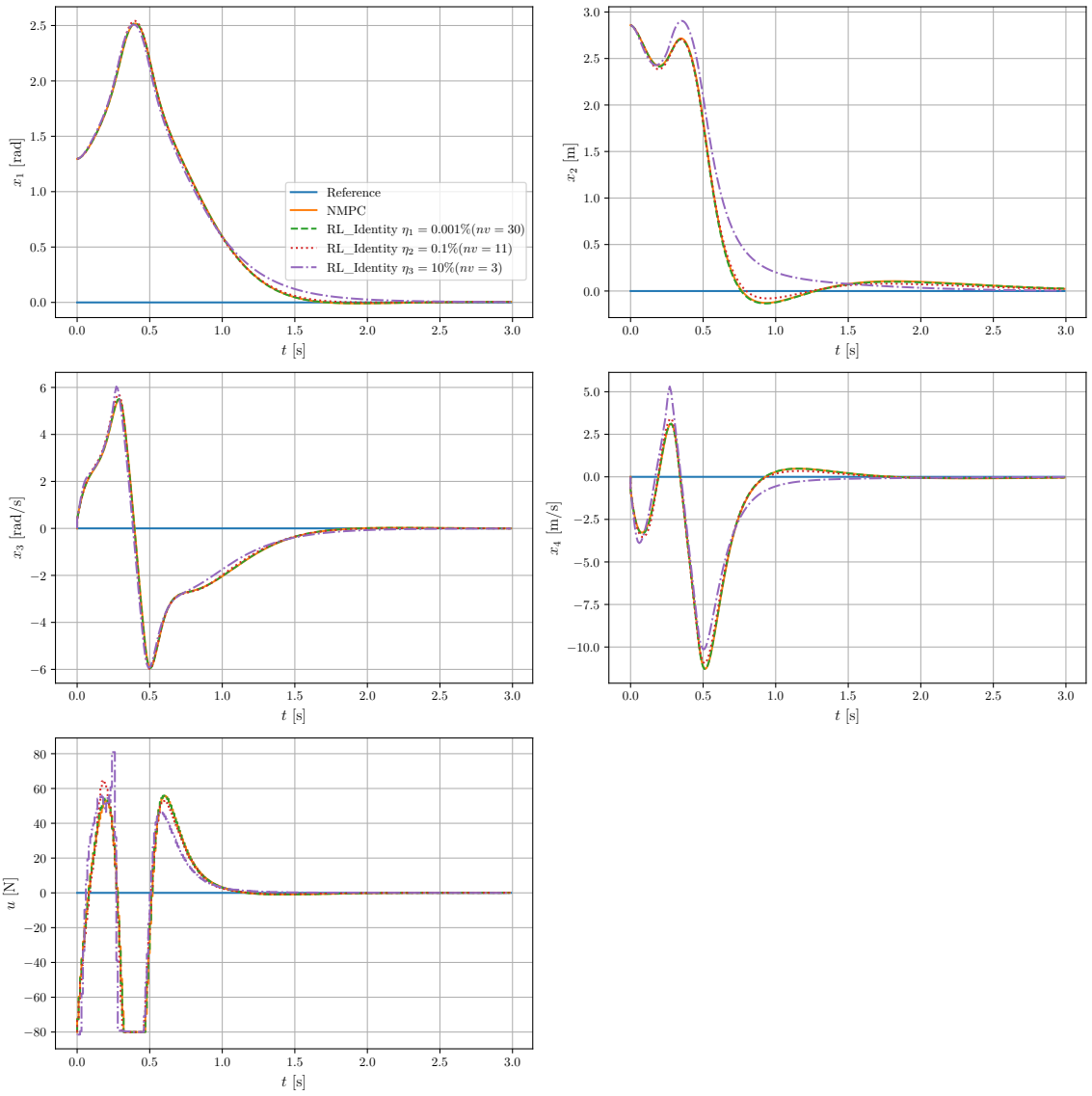

Figure 4.15: Closed-loop trajectories for $x_0 = [1.296, 2.858, 0, 0]^\top$ after RL with $n_v{+}1 = 23, n_v{+}1 = 10, n_v + 1 = 4$ and $N \cdot n_u = 100$, $C = I$

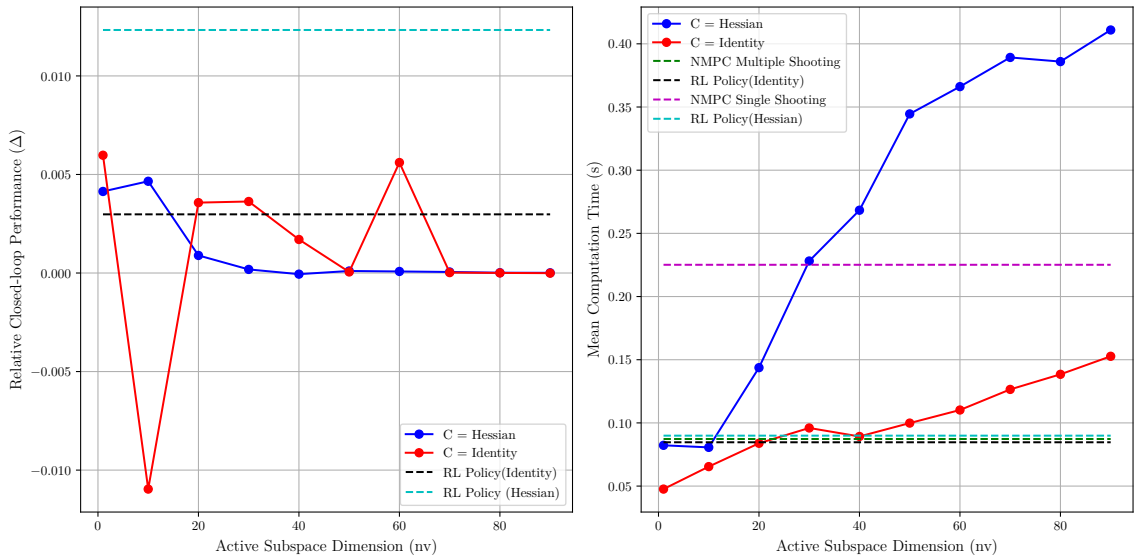

Figure 4.16: Computational complexity and relative closed -loop performance for $n_v = \{1, \cdots, 10\}$ with $n_v + 1 = 6$:RL Policy

5 Conclusion

NMPC is a powerful optimization tool for complex systems but is computationally expensive, as it typically requires a large N to guarantee stability and recursive feasibility. Dimensionality reduction of decision variables is a classic method to reduce the computational burden in both linear and nonlinear NMPC.

NMPC in active subspaces emerges as a method to solve the optimization problem in a lower-dimensional input subspace by decomposing the input decision variables into their active and inactive components. The mapping into the lower-dimensional subspace is achieved through the use of orthogonal projector matrices. However, no underlying theory exists so far on how to choose these projector matrices to guarantee closed-loop optimality.

Learning-based MPC has proven to be a powerful approach for addressing various control problems. The goal of this thesis is to use the RL with NMPC in active subspaces framework to choose the active subspace projector matrices in a way that improves closed-loop performance of the dimensionality reduced OCP. In this study, NMPC in active subspaces is combined with RL to address the problem. RL is used to offline tune the NMPC in active subspaces formulation based on data obtained from the system.

We have demonstrated the effectiveness of the NMPC in active subspaces with the RL framework through the design of our MPCQ-learning algorithm, which combines both frameworks to learn a suboptimal active subspace projector matrix with improved closed-loop performance. We were able to test and validate our design in simulations, which yielded good and satisfactory results.

One of the main concepts that struck us during this project was the block diagonal identity matrix. It was only out of curiosity, toward the end of the project, that we decided to explore the use of this blocking matrix. Surprisingly, it performed quite well as an orthogonal projector for NMPC in active subspaces. While we are uncertain how it would perform for systems with strongly active input constraints, we believe it could potentially serve as a solution to reduce the computational burden for single-shooting methods when use to formulate the dimensionality reduced OCP

As for future work, we suggest exploring the block diagonal matrix further to see if an update rule can be formulated to preserve its sparsity. By doing so, it may be possible to achieve both speed and improved performance, even in moderately reduced dimensional subspaces.

Bibliography

[1] Rojas, O. J. ; Goodwin, G. C. ; Serón, M. M. ; Feuer, A.: SVD-Based Strategy for Receding Horizon Control of Input-Constrained Systems. In: *International Journal of Robust and Nonlinear Control* 14 (2004), Nr. 13-14, S. 1207–1226

[2] Cagienard, R. ; Grieder, P. ; Kerrigan, E. C. ; Morari, M.: Move Blocking Strategies in Receding Horizon Control. In: *Automatica* 43 (2007), Nr. 1, S. 77–85

[3] Iqbal, K.: Use of Orthogonal Functions for Model Predictive Control of Biomechanical Sit-to-Stand. In: *2021 IEEE International Conference on Systems, Man, and Cybernetics (SMC)* 2283-2288 (2021), S. 2283–2288. http://dx.doi.org/10.1109/SMC52423.2021.9658642. – DOI 10.1109/SMC52423.2021.9658642

[4] Chen, Yutao ; Scarabottolo, Nicolò ; Bruschetta, Mattia ; Beghi, Alessandro: An Efficient Move Blocking Strategy for Multiple Shooting-based Nonlinear Model Predictive Control. In: *Proceedings of the 60th IEEE Conference on Decision and Control (CDC)*, IEEE, 2021, S. 2283–2288

[5] Kouzoupis, D. ; Quirynen, R. ; Houska, B. ; Diehl, M.: A block based ALADIN scheme for highly parallelizable direct Optimal Control. In: *American Control Conference* Bd. 2016, 2016, S. 1124–1129

[6] Pan, G. ; Faulwasser, T.: NMPC in Active Subspaces: Dimensionality Reduction with Recursive Feasibility Guarantees. In: *Automatica* 147 (2023), S. 110708

[7] Gros, Sébastien ; Zanon, Mario: Data-driven Economic NMPC using Reinforcement Learning. In: *IEEE Transactions on Automatic Control* PP (2019), Apr., S. 1–1. http://dx.doi.org/10.1109/TAC.2019.2913768. – DOI 10.1109/TAC.2019.2913768. – (cit. on pp. 12, 13, 26)

[8] Paulson, Joel A.: *Modern Control Methods for Chemical Process Systems*, Massachusetts Institute of Technology, Diss., 2016. – Submitted to the Department of Chemical Engineering in partial fulfillment of the requirements for the degree of Doctor of Philosophy in Chemical Engineering

[9] Diehl, Moritz M. ; Rawlings, James B. ; Mayne, David Q.: *Model Predictive Control: Theory, Computation, and Design*. 2. Nob Hill Publishing, 2022. – cit. on pp. 5, 10

[10] Mehta, B. R. ; Reddy, Y. J.: Advanced Process Control Systems. Version: 2015. http://dx.doi.org/10.1016/B978-0-12-800939-0.00019-X. In: Mehta, B. R. (Hrsg.) ; Reddy, Y. J. (Hrsg.): *Industrial Process Automation Systems*. Butterworth-Heinemann,

2015. – DOI 10.1016/B978–0–12–800939–0.00019–X, S. 547–557

[11] Qin, S. J. ; Badgwell, Thomas A.: A Survey of Industrial Model Predictive Control Applications. In: *Control Engineering Practice* 11 (2003), S. 733–764

[12] Ding, Ying ; Wang, Liang ; Li, Yongwei ; Li, Daoliang: Model Predictive Control and Its Application in Agriculture: A Review. In: *Computers and Electronics in Agriculture* 151 (2018), S. 104–117. http://dx.doi.org/10.1016/j.compag.2018.06.004. – DOI 10.1016/j.compag.2018.06.004

[13] Chen, H. ; AllgÃ¶wer, F.: A quasi-infinite horizon nonlinear model predictive control scheme with guaranteed stability. In: *Automatica* 34 (1998), Nr. 10, S. 1205–1217

[14] Rawlings, J. B. ; Mayne, D. Q. ; Diehl, M. M.: *Model Predictive Control: Theory, Computation, and Design*. 2nd. Nob Hill Pub, 2020

[15] Bemporad, Alberto ; Cimini, Giulio: Reduction of the number of variables in parametric constrained least-squares problems. In: *arXiv preprint* arXiv:2012 (2020). – Available at: https://arxiv.org/abs/2012

[16] Ghodsi, Ali: *Dimensionality Reduction: A Short Tutorial*. Waterloo, Ontario, Canada, 2006. – Â© Ali Ghodsi, 2006

[17] Xu, X. ; Lian, C. ; Zuo, L. ; He, H.: Kernel-based approximate dynamic programming for real-time online learning control: an experimental study. In: *IEEE Transactions on Control Systems Technology* 22 (2014), Nr. 1, S. 146–156. http://dx.doi.org/10.1109/TCST.2013.2257772. – DOI 10.1109/TCST.2013.2257772

[18] Watkins, C. ; Dayan, P.: Q-learning. In: *Machine Learning* 8 (1992), May, Nr. 3–4, S. 279–292. http://dx.doi.org/10.1007/BF00992698. – DOI 10.1007/BF00992698

[19] Sutton, Richard S. ; Barto, Andrew G.: *Reinforcement Learning: An Introduction*. The MIT Press, 2015

[20] Wei, Qinglai ; Lewis, Frank L. ; Sun, Qiuye ; Yan, Pengfei ; Song, Ruizhuo: Discrete-Time Deterministic Q-Learning: A Novel Convergence Analysis. In: *IEEE Transactions on Cybernetics* 47 (2017), Nr. 5, S. 1223–1234. http://dx.doi.org/10.1109/TCYB.2016.2555198. – DOI 10.1109/TCYB.2016.2555198

[21] Kordabad, Arash B. ; Gros, Sebastien: Q-learning of the Storage Function in Economic Nonlinear Model Predictive Control. In: *Department of Engineering Cybernetics, Norwegian University of Science and Technology (NTNU)* (2024). – Technical report or unpublished paper, if applicable

[22] Kordabad, A. B. ; Gros, S.: Q-learning of the storage function in Economic Nonlinear Model Predictive Control. In: *Engineering Applications of Artificial Intelligence* 116 (2022), S. 105343. http://dx.doi.org/10.1016/j.engappai.2022.105343. – DOI 10.1016/j.engappai.2022.105343

[23] Sutton, Richard S. ; Barto, Andrew G.: *Reinforcement Learning: An Introduction*. 2nd. Cambridge, MA : MIT Press, 2018. – ISBN 978–0262039246

[24] Slimeni, F. ; Scheers, B. ; Chtourou, Z. ; Nir, V. L.: Jamming Mitigation in Cognitive Radio Networks Using a Modified Q-Learning Algorithm. In: *Proceedings of the International Conference on Military Communications and Information Systems (ICMCIS)*, IEEE, 2015, 1–7

[25] Bahari Kordabad, Arash ; Gros, Sebastien: Q-learning of the storage function in Economic Nonlinear Model Predictive Control. In: *Engineering Applications of Artificial Intelligence* 116 (2022). http://dx.doi.org/https://doi.org/10.1016/j.engappai.2022.105442. – DOI https://doi.org/10.1016/j.engappai.2022.105442. – ISSN 0952–1976

[26] Mnih, Volodymyr ; Kavukcuoglu, Koray ; Silver, David ; Rusu, Andrei A. ; Veness, Joel ; Bellemare, Marc G. ; Graves, Alex ; Riedmiller, Martin ; Fidjeland, Andreas K. ; Ostrovski, Georg ; Petersen, Stig ; Beattie, Charles ; Sadik, Amir ; Antonoglou, Ioannis ; King, Helen ; Kumaran, Dharshan ; Wierstra, Daan ; Legg, Shane ; Hassabis, Demis: Human-level control through deep reinforcement learning. In: *Nature* 518 (2015), S. 529–533

[27] Tsitsiklis, J. N. ; Roy, B. V.: An analysis of temporal-difference learning with function approximation. In: *IEEE Transactions on Automatic Control* 42 (1997), Nr. 5, S. 674–690. http://dx.doi.org/10.1109/9.585292. – DOI 10.1109/9.585292

[28] Mnih, V. ; Kavukcuoglu, K. ; Silver, D. ; Graves, A. ; Antonoglou, I. ; Wierstra, D. ; Riedmiller, M.: Playing Atari with deep reinforcement learning. In: *ArXiv preprint arXiv:1312.5602* (2013). https://arxiv.org/abs/1312.5602

[29] Tagare, Hemant D.: *Notes on Optimization on Stiefel Manifolds*. New Haven, CT, unpublished. – Department of Diagnostic Radiology, Department of Biomedical Engineering

[30] Wen, Zaiwen ; Yin, Wotao: A Feasible Method for Optimization with Orthogonality Constraints / Rice University. 2010. – Technical Report

[31] Liu, Changshuo ; Boumal, Nicolas: Simple Algorithms for Optimization on Riemannian Manifolds with Constraints. In: *Numerical Algorithms* 82 (2019), Nr. 3, S. 1189–1216. – Published online: 28 March 2019

[32] Chen, C. ; Mangasarian, O. L.: Smoothing methods for convex inequalities and linear complementarity problems. In: *Mathematical Programming* 71 (1995), Nr. 1, S. 51–69. – Publisher: Springer

[33] Mayataka: *maafa*. https://github.com/mayataka/maafa. Version: 2024. – Accessed: 2024-06-28

[34] Andersson, Joel ; Gillis, Joris ; Horn, Greg ; Rawlings, James ; Diehl, Moritz: CasADi: A software framework for nonlinear optimization and optimal control. In: *Mathematical Programming Computation* 11 (2018), July, S. pp. 6, 15, 16. http:

//dx.doi.org/10.1007/s12532-018-0139-4. – DOI 10.1007/s12532–018–0139–4

[35] Wächter, Andreas ; Biegler, Lorenz T.: On the implementation of a primal-dual interior point filter line search algorithm for large-scale nonlinear programming. In: *Mathematical Programming* 106 (2006), Nr. 1, S. 25–57

[36] Gill, Philip E. ; Murray, Walter ; Saunders, Michael A.: Userâ€™s Guide for SNOPT Version 7: Software for Large-Scale Nonlinear Programming / Department of Mathematics, University of California, San Diego, La Jolla, CA, USA. Department of Management Science and Engineering, Stanford University, Stanford, CA, USA, 2008. – Forschungsbericht. – Available at: http://www.sdpc.stanford.edu

[37] Linke, T. ; Wassel, D. L. ; Büskens, C.: Recent advances in the solution of large nonlinear optimisation problems with WORHP. In: *Engineering Optimization* 46 (2014), Nr. 9, S. 1125–1145

[38] Brockman, Greg ; Cheung, Vicki ; Pettersson, Ludwig ; Schneider, Jonas ; Schulman, John ; Tang, Jie ; Zaremba, Wojciech: *OpenAI Gym*. https://arxiv.org/abs/1606.01540, 2016. – OpenAI

[39] Townsend, James ; Koep, Niklas ; Weichwald, Sebastian: Pymanopt: A Python Toolbox for Optimization on Manifolds using Automatic Differentiation. In: *Journal of Machine Learning Research* 17 (2016), Nr. 137, 1–5. http://jmlr.org/papers/v17/16-177.html

[40] Absil, P.-A. ; Mahony, R. ; Sepulchre, R.: *Optimization Algorithms on Matrix Manifolds*. Princeton, NJ, USA : Princeton Univ. Press, 2008